

Федеральное государственное бюджетное учреждение науки «Институт проблем передачи информации им. А.А. Харкевича Российской академии наук»



Институт проблем
передачи информации
им. А. А. Харкевича
Российской академии
наук

На правах рукописи

Чеканов Михаил Олегович

**Алгоритмы комплексирования разнородных данных при
построении модели окружающей сцены беспилотного
транспортного средства**

Специальность 2.3.8 —

«Информатика и информационные процессы»

Диссертация на соискание учёной степени
кандидата технических наук

Научный руководитель:
кандидат физико-математических наук
Шипитько Олег Сергеевич

Москва — 2026

Оглавление

	Стр.
Введение	6
 Глава 1. Обзор методов построения моделей окружающего пространства БТС на основе комплексирования данных разной природы	
	11
1.1 Современное состояние беспилотных транспортных средств . . .	11
1.2 Обзор используемых в колёсной робототехнике датчиков и их ограничения	14
1.3 Модели окружающего пространства БТС	16
1.4 Методы комплексирования данных	20
1.5 Метрики для оценки качества моделей окружающего пространства	22
1.5.1 Метрики для оценки объектно-ориентированных представлений	23
1.5.2 Метрики для оценки сеточных представлений	24
1.5.3 Метрики для оценки непрерывных представлений	27
1.6 Выбор модели окружающего пространства и метрик её оценки . .	28
1.6.1 Выбор метрик, используемых в работе	29
1.7 Обзор методов решения задач, возникающих при построении выбранной модели	30
1.7.1 Алгоритмы проецирования данных камеры на плоскость движения БТС	30
1.7.2 Влияние числа источников данных на производительность алгоритмов построения карты проходимости пространства	34
1.8 Требования к разрабатываемым модели окружающего пространства и алгоритмам её построения	37
1.9 Заключение к главе 1	38
 Глава 2. Алгоритмы проецирования визуальных данных на плоскость движения БТС	
	40

2.1	Алгоритм проекции визуальных детекций на плоскость движения БТС на основе модели L-Shape	40
2.1.1	Этап 1. Проекция двумерной детекции изображения на вид сверху	41
2.1.2	Этап 2. Вписывание проекции объекта на виде сверху в ориентированную ограничивающую рамку	44
2.2	Исследование точности алгоритма проекции визуальных детекций на основе модели L-Shape	45
2.2.1	Набор данных	46
2.2.2	Сравниваемые алгоритмы	47
2.2.3	Описание эксперимента	49
2.2.4	Результаты экспериментов	50
2.3	Алгоритм проекции визуальных детекций на плоскость движения БТС на основе комплексирования визуальных и лидарных данных	51
2.3.1	Разделение облака точек на точки земли и препятствий	56
2.4	Исследование эффективности алгоритма проекции визуальных детекций на основе комплексирования визуальных и лидарных данных	60
2.4.1	Описание эксперимента	60
2.4.2	Результаты экспериментов	61
2.4.3	Вычислительная эффективность	62
2.5	Алгоритм проекции сегментации проезжей части на изображении на плоскость движения БТС	63
2.5.1	Вычисление функции $z(x,y)$	68
2.5.2	Комплексирование полученной проекции с базовой картой проходимости пространства	70
2.6	Исследование эффективности алгоритма проекции сегментации проезжей части на основе комплексирования визуальных и лидарных данных	72
2.6.1	Набор данных	72
2.6.2	Сравниваемые алгоритмы и метрики	73
2.6.3	Результаты экспериментов	74

2.6.4	Вычислительная эффективность	76
2.7	Заключение к главе 2	77

Глава 3. Программный комплекс построения карты

проходимости пространства на основе

комплексирования разнородных данных 80

3.1	Общие сведения	80
3.2	Назначение и функциональные возможности	80
3.3	Структура программного комплекса	81
3.3.1	Библиотека «occipancy_map_fusion»	85
3.3.2	Программа «evo_occipancy_map_fusion»	95
3.4	Заключение к главе 3	97

Глава 4. Параллельный алгоритм построения карты

проходимости пространства на основе

комплексирования разнородных данных 98

4.1	Производительность алгоритма построения карты проходимости пространства	98
4.2	Постановка задачи построения карты проходимости пространства	99
4.3	Модификации базовой реализации	103
4.3.1	Базовая реализация	105
4.3.2	Расширенная карта проходимости пространства	108
4.3.3	Карта с разделяемыми блокировками	110
4.3.4	Карта проходимости пространства с полным обновлением без блокировок	114
4.4	Сравнение задержки обновления локальной карты проходимости пространства	117
4.4.1	Сдвиг карты	118
4.4.2	Добавление фрагмента в одном потоке	118
4.4.3	Добавление фрагмента в нескольких потоках	119
4.4.4	Получение карты	121
4.4.5	Стандартный цикл «публикации» карты	122
4.5	Реализация финальной модификации для двумерной карты проходимости пространства	124

4.6	Заключение к главе 4	124
Заклучение		127
Список сокращений и условных обозначений		129
Список литературы		130
Список рисунков		142
Список таблиц		145
Приложение А. Вывод решения задачи вписывания в L-Shape модель		146
Приложение Б. Свидетельства о государственной регистрации программ для ЭВМ		148
Приложение В. Патенты		151
Приложение Г. Акты о внедрении результатов диссертационного исследования		152

Введение

Последние десятилетия характеризуются активным внедрением и распространением беспилотных транспортных средств (БТС) в различных сферах человеческой деятельности. Примеры таких сфер – логистика, промышленность, сельское хозяйство, горное дело, городская инфраструктура. Развитие беспилотного транспорта играет важную роль в достижении национальных целей и задач развития Российской Федерации. Так, «Транспортные технологии для различных сфер применения (море, земля, воздух), в том числе беспилотные и автономные системы» входят в перечень критических технологий Российской Федерации. Транспортная стратегия РФ на период до 2030 года включает организацию движения БТС на автомобильной дороге общего пользования. В рамках Стратегии развития автомобильной промышленности Российской Федерации до 2035 года создана инициатива социально-экономического развития «Беспилотные логистические коридоры», предусматривающая проведение мероприятий по внедрению на автомобильных дорогах Государственной компании интеллектуальных транспортных систем, ориентированных в том числе на обеспечение движения высокоавтоматизированных транспортных средств.

Для осуществления успешной навигации и планирования безопасного маршрута БТС требуется модель окружающего пространства. В зависимости от выполняемых задач к данным моделям предъявляются различные требования, включая степень детализации описания пространства, обеспечиваемой моделью, вычислительную сложность её построения, объём памяти, занимаемой моделью и алгоритмами её построения и уточнения. В частности, для осуществления безопасного передвижения БТС, используемая модель должна корректно описывать проходимость пространства для БТС, т.е. позиции и форму окружающих препятствий и областей, в которых БТС может находиться. Построение модели производится на основании показаний различных датчиков, которыми оснащается БТС. Расположение, количество и тип используемых датчиков зачастую уникальны для каждого типа БТС. Эти характеристики должны быть учтены в выбираемой модели окружающего пространства для обеспечения своевременного учёта показаний всех доступных регистрируемых данных и, как следствие, достижения качественной и безопасной навигации.

Задача построения модели окружающего пространства в робототехнике широко исследована, однако не решена в полной мере и сегодня. Исследование этой задачи нашло отражение в работах М. Павоне, А. Элфеса, Х. Моравека, С. Труна, Б.М. Миллера, Д.А. Юдина и Л.М.Г. Гонсалвеса. Для построения модели, как правило, используется не один датчик, а их набор. Использование нескольких датчиков позволяет, во-первых, уменьшить слепую зону БТС, а во-вторых, при использовании датчиков разного типа, компенсировать ограничения каждого типа датчика по отдельности. Поэтому важным аспектом при разработке моделей окружающего пространства является способ комплексирования разнородных данных, регистрируемых датчиками, для построения полного и непротиворечивого описания окружения. Среди исследований задачи комплексирования данных стоит выделить работы Б. Халеги, П.П. Николаева, Ю.В. Визильтера. Однако вопрос применимости алгоритмов комплексирования при увеличении числа и/или типов комплексируемых датчиков не был в достаточной степени исследован к настоящему времени. Увеличение числа датчиков требует большего объёма вычислительных ресурсов бортовых вычислителей, устанавливаемых на БТС, и может приводить к увеличению времени построения модели окружения, тем самым снижая безопасность движения БТС.

Таким образом, с учётом вышесказанного, актуальной является разработка вычислительно эффективных методов комплексирования данных с датчиков БТС для построения карты проходимости пространства.

Целью данной работы является разработка методов построения моделей, описывающих проходимость окружающего пространства, предназначенных для работы на бортовом вычислителе БТС, обладающем ограниченными вычислительными ресурсами, в режиме реального времени и способных единовременно обрабатывать данные с большого количества бортовых датчиков, для повышения качества и актуальности модели окружающего пространства.

Для достижения поставленной цели необходимо было решить следующие **задачи**:

1. Провести обзор существующих методов построения моделей окружающего пространства БТС для выбора базовой модели, описывающей проходимость пространства, и метода её построения, которые будут использованы при разработке дальнейших алгоритмов.
2. Разработать алгоритмы оценки положения объектов в окружающем пространстве БТС, а также областей проезжей части, доступных для

движения БТС, на основе визуальных и лидарных данных, предназначенные для работы на бортовом вычислителе БТС в режиме реального времени.

3. Разработать программный комплекс, обеспечивающий построение модели окружающего пространства БТС на основе комплексирования разнородных данных с бортовых датчиков БТС.
4. Разработать параллельный алгоритм построения модели окружающего пространства, осуществляющий одновременное обновление модели несколькими источниками данных.

Научная новизна:

1. Предложен новый алгоритм оценки положения окружающих ТС на плоскости движения БТС на основе визуальных данных с изображения монокулярной камеры, использующий модель L-образной формы видимой границы транспортного средства.
2. Предложен новый алгоритм оценки положения окружающих ТС на плоскости движения БТС на основе комплексирования визуальных и лидарных данных, учитывающий рельеф поверхности дороги при сегментации лидарных данных.
3. Предложен новый алгоритм проекции семантической сегментации проезжей части на изображении на плоскость движения БТС на основе комплексирования данных изображения и лидарного облака точек, учитывающий рельеф поверхности дороги в проецируемой области.
4. Впервые показано, что учёт визуальных данных с камер БТС для построения карты проходимости пространства позволяет повысить её качество по сравнению с картой, строящейся исключительно по лидарным данным.
5. Предложен новый метод представления карты проходимости пространства, позволяющий снизить время задержки между моментом регистрации данных сенсором и моментом обновления карты за счёт параллельного обновления модели несколькими источниками данных.

Практическая значимость. Разработанные алгоритмы и комплекс технических средств, обеспечивающие построение модели окружающего пространства БТС, реализованные в данной работе, внедрены в беспилотные транспортные средства «EVO.1» и «EVO.3», разработанные компанией ООО «ЭвоКарго». Внедрение разработанных в данной диссертации методов под-

тверждено соответствующими актами. На результаты, полученные в работе, получены свидетельства о государственной регистрации программ для ЭВМ:

1. № 2025617212 Программное обеспечение «Система восприятия N1 V3».
2. № 2025669744 Программное обеспечение «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства».

На основные результаты работы получен патент № 2853062 «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства».

Методология и методы исследования. В диссертации используются методы вероятностной робототехники, методы байесовской оценки, вычислительной оптимизации, компьютерного зрения, анализ вычислительной сложности, а также численный эксперимент.

Основные положения, выносимые на защиту:

1. Предложенный алгоритм оценки положения объектов в окружающем пространстве БТС по изображению с монокулярной камеры позволяет на 2,7 % увеличить коэффициент Жаккара по сравнению с общеизвестными методами.
2. Предложенный алгоритм комплексирования визуальных данных с камеры и облаков точек лидаров для проекции визуальных детекций на изображении на плоскость движения БТС позволяет на 30,3 % увеличить коэффициент Жаккара и на 120 % увеличить дальность обнаружения препятствий по сравнению с алгоритмом, использующим только визуальные данные.
3. Предложенный алгоритм комплексирования визуальных данных с камеры и облаков точек лидаров для проекции семантической сегментации проезжей части на изображении на плоскость движения БТС при сочетании с базовым методом построения карты проходимости пространства повышает качество создаваемой модели: на 5,6 % снижает среднеквадратичную ошибку относительно эталонной карты проходимости пространства и на 9,3 % снижает среднеквадратичную ошибку стоимости поиска пути (PathFinding Cost Mean Squared Error, PFC-MSE).
4. Предложенный метод построения карты проходимости пространства, позволяющий выполнять неблокирующие операции сдвига, получения

и обновления, позволяет в 13,89 раза уменьшить время обновления карты несколькими сенсорами за один цикл получения данных в одномомерном случае.

Достоверность полученных результатов обеспечивается использованием строгого математического аппарата и методов математической статистики. Помимо этого, достоверность подтверждается результатами вычислительных и натурных экспериментов и внедрением разработанных методов. Полученные результаты находятся в соответствии с результатами, полученными другими исследователями.

Апробация работы. Основные результаты работы докладывались на международных конференциях:

- 16th International Conference on Machine Vision (ICMV 2023, Ереван, Армения);
- 40th ECMS International Conference on Modelling and Simulation (ECMS 2026, Гримстад, Норвегия).

Личный вклад. Все результаты диссертации, вынесенные на защиту, получены автором самостоятельно. Постановка задач и обсуждение результатов проводились совместно с научным руководителем.

Публикации. Основные результаты по теме диссертации изложены в 5 печатных изданиях, 3 из которых изданы в журналах, рекомендованных ВАК, 2 — в периодических научных журналах, индексируемых Web of Science и Scopus. Зарегистрированы 1 патент и 2 программы для ЭВМ.

Объём и структура работы. Диссертация состоит из введения, 4 глав, заключения и 4 приложений. Полный объём диссертации составляет 152 страницы, включая 41 рисунок и 7 таблиц. Список литературы содержит 117 наименований.

Глава 1. Обзор методов построения моделей окружающего пространства БТС на основе комплексирования данных разной природы

В данной главе проводится обзор моделей окружающего пространства БТС; датчиков, по измерениям которых проводится их построение; методов построения и обновления моделей, а также метрик для их оценки. На основании проведённого обзора произведён выбор базовой модели, алгоритмов её построения и метрик для оценки их качества, которые будут использованы в следующих главах работы. Также формируются требования к разрабатываемым в данном исследовании моделям и алгоритмам их построения.

1.1 Современное состояние беспилотных транспортных средств

Развитие беспилотных транспортных средств является одним из наиболее активных направлений интеллектуальных роботизированных систем. Разрабатываемые решения в этой области значительно отличаются по функционалу, а также условиям, в которых они применяются. Для классификации автономного транспорта Обществом автомобильных инженеров (англ. Society of Automotive Engineers, SAE) в 2014 году были предложены шесть уровней автоматизации (SAE J3016) [1].

1. Уровень 0. Отсутствие автоматизации (No Automation)

На данном уровне водитель полностью контролирует движение транспортного средства. Рулевое управление, ускорение/торможение и выполнение манёвров осуществляются исключительно водителем. Автоматические системы, доступные на транспортных средствах данного уровня, осуществляют только информационные функции либо краткосрочно берут на себя вспомогательные функции управления транспортным средством. Примерами систем такого уровня являются системы предупреждения о сходе с полосы, СПСП (Lane Departure Warning, LDW) [2], системы обнаружения объектов в слепых зонах транспортного средства (Blind Spot Monitoring, BSM) [3], системы авто-

матического экстренного торможения, САЭТ (Autonomous Emergency Braking, AEB) [4] и др.

2. **Уровень 1. Вспомогательные системы (Driver Assistance)**

Автоматические системы автомобиля на данном уровне могут выполнять одну из основных функций управления транспортным средством в определённых условиях, например ускорение или торможение или рулевое управление. При этом ответственность за безопасность остаётся на водителе, который обязан контролировать окружающую ситуацию и быть готовым взять управление транспортным средством на себя. Типичные примеры систем этого уровня – адаптивный круиз-контроль (Adaptive Cruise Control, ACC) [5] – система, позволяющая поддерживать фиксированную скорость и дистанцию позади идущего впереди транспортного средства, а также система удержания полосы движения, СУПД (Lane Keeping System, LKS) [6].

3. **Уровень 2. Частичная автоматизация (Partial Automation)**

На данном уровне системы транспортного средства способны одновременно выполнять и рулевое управление, и торможение/ускорение. Однако, как и на предшествующих уровнях, водитель несёт полную ответственность за безопасность движения транспортного средства. К системам данного уровня относят системы удержания полосы, которые, в отличие от аналогичных систем первого уровня, контролируют не только рулевое управление, но и способны корректировать скорость транспортного средства. Также к этому уровню принадлежат системы автоматической парковки (Automatic Parking) [7].

4. **Уровень 3. Условная автоматизация (Conditional Automation)**

Автоматические системы автомобиля на этом уровне способны полностью осуществлять управление транспортным средством без участия водителя в ограниченных условиях: как правило, это определённый тип дороги (автомагистрали с отчётливо распознаваемой разметкой) и хорошие погодные условия. В случае возникновения нестандартной ситуации на дороге система должна своевременно передать управление транспортным средством водителю. Соответственно, в отличие от предшествующих уровней, от водителя не требуется постоянное слежение за окружающей обстановкой, но он должен быть готов к управлению

транспортным средством в нештатных ситуациях. Пример такой системы – ассистент движения в пробках (Traffic Jam Assist) [8].

5. **Уровень 4. Высокая автоматизация (High Automation)**

Аналогично предыдущему уровню, системы автомобиля уровня 4 способны осуществлять полное управление транспортным средством в чётко определённых сценариях. От систем 3-го уровня их отличает способность самостоятельного завершения поездки или возвращения в безопасное состояние в случае возникновения внештатной ситуации и отсутствия реакции от водителя. Как следствие, на автомобили данного уровня не накладывается требование к обязательному наличию водителя. К системам этого уровня относят беспилотное такси и автоматизированные грузовые транспортные средства.

6. **Уровень 5. Полная автоматизация (Full Automation)**

На данном уровне автономности системы автомобиля способны осуществлять полное управление транспортным средством без ограничения на условия эксплуатации. Автомобили данного уровня не требуют наличия механизмов ручного управления, таких как педали торможения/ускорения, рулевое колесо и пр. На сегодняшний день не существует транспортных средств, обладающих 5-м уровнем автоматизации, несмотря на большое количество работ, направленных на их создание.

В настоящее время идёт бурное развитие технологий, направленных на усовершенствование беспилотных транспортных средств, обладающих 4-м уровнем автоматизации, а также создание транспортных средств с 5-м уровнем автоматизации. Среди проектов по разработке таких транспортных средств можно выделить Waymo [9], Tesla [10], Яндекс [11], Baidu Apollo [12] и др. Несмотря на то, что данные проекты показывают высокую степень автономности [13], перед отраслью стоит множество задач, требующих решения для повышения уровня безопасности движения беспилотных транспортных средств, что будет способствовать внедрению данных технологий в повседневное использование [14]. Одна из таких задач – разработка надёжных систем восприятия транспортного средства, отвечающих за построение точной и полной модели окружающего пространства. В рамках данной работы исследуются и предлагаются алгоритмы построения таких моделей.

1.2 Обзор используемых в колёсной робототехнике датчиков и их ограничения

Обновление любой модели окружающей сцены производится на основании показаний датчиков, установленных на БТС. Основные датчики, применяемые в колёсной робототехнике для построения модели, – лидары, радары, сонары и камеры [15].

1. **Радары (Radio Detection and Ranging, RADAR).** Технология, основанная на создании радиоволн и регистрации их отражений от объектов. По времени регистрации и степени затухания отражённой волны датчик определяет расстояние до объекта и его скорость. Радары разделяют на радары дальнего диапазона (РДД) и короткого диапазона (РКД) [16]. Первые обладают наибольшей дальностью видимости (около 250 м) среди упомянутых датчиков и используются для обнаружения препятствий на пути движения машины. Вторые обладают меньшей дальностью, но лучшим разрешением и могут быть использованы для реализации систем парковки и перестроения между полосами движения. В то же время радары обладают низким угловым разрешением и могут давать ложные срабатывания из-за металлических конструкций, например, решёток ливневых канализаций в поле видимости, и переотражений сигнала излучателя.
2. **Лидары (Light Detection and Ranging, LiDAR).** Технология, начавшая развитие во второй половине XX века, в 1960-х годах, в космической и авиастроительной сферах [17]. Принцип работы схож с радаром, но вместо радиоволн использует пучки инфракрасного спектра. Существует два типа лидаров: механические и твердотельные. В механическом лидаре источники и приёмники сигнала вращаются на высокой (от 600 оборотов в минуту) скорости. В твердотельном лидаре источник и приёмник остаются неподвижными, а изменение направления сканирующего луча производится путём вращения призмы или зеркала, через которые он проходит. Помимо измерения расстояния до объекта лидары способны регистрировать его светоотражательную способность, что может быть применено, например, для детекции разметки. Лидары обладают высокой точностью и лучшим разрешением,

чем радары, и хорошей дальностью обзора. Однако, как и радары, лидары требовательны к погодным условиям: качество измерений снижается в условиях тумана и/или осадков. Также ограничивающим фактором для использования лидаров является их дороговизна, из-за этого многие компании, занимающиеся разработкой беспилотных автомобилей, вынуждены отказываться от лидаров в пользу альтернативных способов детекции местонахождения машины и построения маршрута движения. Лидары могут использоваться для распознавания и классификации объектов, создания трёхмерных карт.

3. **Камеры.** Наиболее распространённые датчики [18], используемые в колёсной робототехнике. В отличие от остальных описанных датчиков, камеры – пассивные датчики: они не излучают никаких сигналов. Для активных же датчиков отдельной проблемой является взаимная интерференция. Это позволяет без опасений использовать большое количество камер без необходимости синхронизировать их работу друг с другом. В основном в БТС используют камеры видимого (400–780 нм) спектра, но также встречаются модели, использующие камеры инфракрасного спектра. Камеры обладают наилучшим разрешением среди описанных датчиков, но не способны измерять расстояние до объектов. Использование нескольких камер в связке позволяет использовать алгоритмы стереозрения для измерения расстояния, однако точность и дальность этих измерений ниже, чем у лидаров и радаров. Камеры используются для захвата изображений, которые затем обрабатываются с помощью алгоритмов компьютерного зрения. Камеры в компьютерном зрении широко используются в различных областях, таких как робототехника, автоматическое управление производственными процессами, медицинская диагностика, безопасность, распознавание лиц, автоматическое управление транспортом, анализ изображений и многих других.
4. **Сонары.** Как лидары и радары, принцип работы сонаров основан на измерении времени регистрации отражения волн от объектов, однако в сонарах используются ультразвуковые (40–70 кГц) волны [19]. Данные волны не слышны и не могут нанести вред слуху человека, что немаловажно, поскольку громкость создаваемого импульса достигает 100 дБ. Сонары обладают наименьшей областью обзора (менее десятка метров) и потому, как правило, используются для систем парковки. Со-

нары используются для измерения расстояния до объектов и создания трёхмерной модели окружающей среды. Они работают на основе эхолокации, отправляя звуковые импульсы и затем регистрируя отражённые от объектов сигналы. Полученные данные могут быть обработаны компьютером для определения расстояния и формы объектов. Одним из наиболее распространённых применений сонаров является их использование при парковке БТС. Также сонары могут использоваться в системах обнаружения движущихся объектов, таких как пешеходов и других транспортных средств.

Из описания датчиков следует, что каждый тип датчика обладает рядом преимуществ и недостатков. Использование нескольких датчиков разных типов на одном транспортном средстве позволяет компенсировать недостатки отдельных датчиков и добиться повышения качества восприятия, тем самым повышая безопасность движения и открывая новые сценарии применения беспилотных транспортных средств. Сводное сравнение характеристик рассмотренных датчиков приведено в таблице 1.

Характеристика	Лидар	Радар	Камера	Сонар
Разрешение	Среднее	Низкое	Высокое	Низкое
Дальность области обзора	150 м	250 м	200 м	5 м
Устойчивость к погодным условиям	Средняя	Высокая	Низкая	Высокая
Чувствительность к условиям освещения	Нет	Нет	Высокая	Нет
Информация о дистанции	Да	Да	Нет	Да
Вычислительная нагрузка обработки	Средняя	Низкая	Высокая	Низкая
Стоимость	Высокая	Средняя	Низкая	Низкая

Таблица 1 — Сравнение характеристик датчиков, используемых в колёсной робототехнике

1.3 Модели окружающего пространства БТС

Данные, регистрируемые перечисленными датчиками, служат основой для построения модели окружающего пространства. Рассмотрим, какие модели применяются для этого в колёсной робототехнике.

В робототехнике под моделью окружающего пространства понимается приближённое описание окружающих БТС объектов: их наличие и положение в некоторой окрестности транспортного средства. Помимо информации о положении объектов, такая модель может описывать их скорость и предполагаемую траекторию движения. Также модель может описывать форму и расположение проезжей части, слепых зон транспортного средства (как вызванных окклюзиями, так и обусловленных отсутствием показаний датчиков для соответствующей области пространства), а также объектов транспортной инфраструктуры: полос разметки, дорожных знаков, бордюров, светофоров и т.д.

В рамках данной работы исследуются двумерные модели – модели, проецирующие объекты, окружающие БТС, на плоскость его движения. Такие модели не хранят информацию о высоте объекта и его положении над землёй, т.к., как правило, эти данные избыточны для решения задачи планирования движения колёсного транспорта. В то же время следует отметить, что среди предложенных на сегодняшний день моделей существуют и трёхмерные [20]. Так, в работе [21] окружающая сцена представлена совокупностью полигонов, в работе [22] пространство разбивается трёхмерной сеткой, каждому элементу которой сопоставляется вероятность нахождения в нём препятствия, а в работе [23] в отдельный класс выделены модели, описывающие окружающее пространство как облако точек.

Среди существующих двумерных моделей можно выделить три категории:

1. **Объектно-ориентированные представления.** В них окружающая сцена описывается как множество объектов. Как правило, объект представлен в виде ограничивающего параллелепипеда: положением его центра, ориентацией и габаритами [24]. Дополнительно описание объекта может содержать тип (транспортное средство, пешеход, животное и т.д.), а также линейные и угловые скорость и ускорение [25–27], предсказываемую траекторию перемещения [28]. Такие модели имеют достаточно краткую форму записи: множество векторов, описывающих отдельный объект, что делает их вычислительно эффективными. В то же время они ограничены в возможности описания объектов сложной формы: тягачи с полуприцепами, перевёрнутые машины, объекты нетипичной формы.
2. **Сеточные представления.** Модели этой категории призваны обойти ограничение предыдущей категории и описывать объекты с произволь-

ной геометрией. В них пространство вокруг транспортного средства разбивается на сетку фиксированного разрешения, каждая клетка которой хранит информацию о наличии (или вероятности наличия) препятствия в соответствующей области [29]. Такое представление получило название карты проходимости пространства (Occupancy Grid Map, OGM). К этой же категории относятся динамические карты проходимости пространства (Dynamic Occupancy Grid Map, DOGMa) [30], в них для клеток, в которых обнаружено препятствие, добавляется информация о его скорости и направлении движения. Модели этой категории также чаще, чем объектно-ориентированные модели, учитывают неклассифицируемые объекты: мусор, открытые люки, бордюры и т.п., что делает их более надёжным представлением окружающей сцены. Однако более детальное описание сцены приводит к большему размеру структуры данных в памяти компьютера, что негативно влияет на вычислительную сложность алгоритмов, работающих с моделями в этой категории. Ограниченная разрешающая способность данных моделей также может рассматриваться как недостаток для некоторых задач автономной навигации.

3. **Непрерывные представления.** Как и модели прошлой категории, модели непрерывного представления стремятся описывать объекты со сложной геометрией. Однако в отличие от сеточных представлений модели этой категории представляют пространство как непрерывное отображение $\mathbb{R}^2 \rightarrow O$, где каждой точке плоскости движения транспортного средства сопоставляется некоторая характеристика: бинарная классификация принадлежности точки проезжей части дороги ($O = \{0,1\}$), класс объекта ($O = \{0, \dots, N\}$), плотность вероятности нахождения препятствия в данной точке ($O = \mathbb{R}^+$) и т.п. Таким образом, модели данной категории имеют бесконечную степень детализации, что позволяет обойти главное ограничение сеточных представлений. Недостаток же моделей данной категории заключается в том, что алгоритмы для их построения и обновления сложнее в реализации, а вычислительная сложность может быть хуже, чем у предыдущих двух категорий (для сеточных представлений последнее зависит от требований к размеру и разрешению сетки). Примерами таких представлений служат

полигональные случайные поля (Polygonal Random Field, PRF) [31], мультимодальное нормальное распределение [32].

Наибольшее распространение среди перечисленных получила карта проходимости пространства, поэтому опишем эту модель и методы её построения подробнее.

Представление окружающего пространства в виде карты проходимости пространства было впервые предложено в работе Х. Моравека и А. Элфеса [33]. В рамках данного представления окружающее пространство разбивается на двумерную сетку M , каждой клетке которой m_i сопоставляется вероятность нахождения препятствия в пространстве $p(m_i)$, ограниченном этой клеткой. В работах [34; 35] те же авторы расширили модель карты проходимости пространства, включив в неё обратную модель наблюдения – вероятностную модель, моделирующую процесс измерений датчика и учитывающую принцип работы, а также особенности среды, в которой производятся наблюдения. Расширение модели позволило сформулировать алгоритм уточнения карты проходимости пространства на основании показаний датчика. В рамках данной модели дополнительно делается предположение, что каждая вероятность занятости клетки в карте проходимости не зависит от вероятностей занятости других клеток карты. Для каждой клетки вычисляется величина так называемого логарифма шансов:

$$l_{t,i} = \ln \frac{p(m_i|z_{1:t}, x_{1:t})}{1 - p(m_i|z_{1:t}, x_{1:t})},$$

где $p(m_i|z_{1:t}, x_{1:t})$ – вероятность занятости i -й клетки карты, при условии, что к моменту времени t БТС прошёл траекторию $x_{1:t}$, и его датчики давали показания $z_{1:t}$. Заметим, что здесь используется обратная причинно-следственная зависимость: измеряется вероятность причины (занятость пространства), при условии наступления её следствия (показания датчика), чем и обусловлено название модели. С получением новых данных значения в тех клетках, о которых появились новые данные (иными словами, которые попали в область видимости датчика), обновляются по следующей формуле:

$$l_{t,i} = l_{t-1,i} + \ln \frac{p(m_i|z_t, x_t)}{1 - p(m_i|z_t, x_t)} - l_{0,i}, \quad (1.1)$$

где $l_{0,i} = \ln \frac{p(m_i)}{1 - p(m_i)}$ – логарифм шансов, посчитанный для априорной вероятности занятости клетки. Зачастую о карте проходимости не делается априорных предположений и $p(m_i)$ устанавливают равным 0,5, что приводит к $l_{0,i} = 0$.

Такой формат обновления удобен, так как позволяет независимо друг от друга посчитать значение $\ln \frac{p(m_i|z_t, x_t)}{1 - p(m_i|z_t, x_t)} - l_{0,i}$, на основании полученных данных от каждого датчика, а затем просто добавить полученное значение к уже имеющейся карте проходимости (при условии атомарности данной операции).

1.4 Методы комплексирования данных

На сегодняшний день предложено множество алгоритмов построения карты проходимости пространства. Их реализация сравнительно проста в случае, если информация об окружающем пространстве поступает от единственного датчика. Примеры таких систем представлены в статьях [36] и [37]. Однако для большинства практических задач единственного датчика недостаточно из-за ограниченного поля зрения, невозможности полностью компенсировать зашумлённость входных данных и общих ограничений, обусловленных принципом его работы.

Чтобы получить более точную модель окружения и повысить безопасность движения БТС, на них устанавливают несколько датчиков. Примеры систем с несколькими датчиками описаны в работах [38; 39]. Встречаются как системы, использующие несколько датчиков одного типа (например, камеры, как в работах [40; 41]), так и комбинирующие различные датчики, например:

- лидары и радары. Примеры таких БТС представлены в [42–44];
- лидары и камеры [45–47];
- и другие комбинации [48–50].

Алгоритмы построения модели окружающего БТС пространства для таких систем в общем случае генерируют модель достаточно полную и точную, но вынуждены решать задачу комплексирования данных для разрешения противоречий в показаниях разных датчиков. Существуют разные алгоритмы, осуществляющие комплексирование данных, которые можно разделить на три группы [51]:

1. Раннее комплексирование – необработанные данные с датчиков объединяются в один пакет данных и передаются системе для последующего анализа. Примером такой агрегации является построение карты глубины в стереокамере, как например в работе [52].
2. Промежуточное комплексирование – по сырым данным с датчиков вычисляются отдельные признаки (в общем случае отличные для каждого из датчиков), которые затем объединяются для дальнейшего решения задачи. Так в работах [53; 54] данные с разных датчиков обрабатываются разными нейросетевыми моделями, после чего выходные тензоры конкатенируются.
3. Позднее комплексирование – для каждого датчика целевая задача (в нашем случае построение модели окружающего БТС пространства) решается отдельно, что даёт независимую гипотезу. На основе этих гипотез более высокоуровневый алгоритм вычисляет итоговое решение.

Для существующих алгоритмов комплексирования данных в контексте задачи построения карты проходимости пространства в работе [55] представлена классификация по трём группам:

1. Алгоритмы, использующие вероятностные модели. К ним относятся байесовский вывод, теория Демпстера—Шафера, робастные методы и т.д.
2. Алгоритмы, использующие метод наименьших квадратов (МНК): фильтр Калмана, методы оптимизации, эллипсоиды неопределённости (англ. *uncertainty ellipsoids*) и пр.
3. Алгоритмы интеллектуального комплексирования: методы нечёткой логики, нейросетевые и генетические алгоритмы.

В настоящее время продолжается активная разработка алгоритмов каждой из трёх групп. Примером актуальных исследований в первой группе является работа [56], в которой используется метод релевантных векторов (метод машинного обучения, использующий байесовский вывод) для построения непрерывной карты проходимости пространства. Во второй группе можно выделить работу [57], в которой положение динамических препятствий на карте обновляется с помощью фильтра Калмана. Стоит отметить, что в данной работе обновление информации о статических препятствиях происходит с помощью байесовского вывода, поэтому предлагаемый алгоритм является объединением методов из двух представленных групп. Наконец, не менее активно развивают-

ся алгоритмы третьей группы, одним из которых является модель глубокого обучения BEVFusion, представленная в работе [58].

Помимо уже приведённых классификаций, которые характеризуют то, как устроена обработка данных, также существует классификация по характеру взаимодействия комплексируемых данных. Такая классификация предложена в работе Дюррант-Уайта [59] и выделяет три типа комплексирования:

1. **Конкурирующее** – входные данные содержат оценки одного и того же свойства наблюдаемого объекта, которое, соответственно, может быть уточнено при их объединении. К этому же типу относится и выявление противоречий между независимо зарегистрированными измерениями.
2. **Дополняющее** – входные данные содержат различные свойства наблюдаемого объекта, которые при объединении дополняют друг друга, но не уточняются.
3. **Кооперативное** – входные данные объединяются для оценки свойств, недоступных ни одному из источников по отдельности.

Заметим, что некоторые алгоритмы комплексирования данных могут включать несколько этапов, для каждого из которых могут применяться отдельные типы комплексирования из представленных классификаций. Пример такого алгоритма приведён в разделе 2.5.

1.5 Метрики для оценки качества моделей окружающего пространства

Одной из ключевых задач при построении моделей окружающей сцены БТС является оценка качества таких моделей. Учитывая сложность сенсорного восприятия в условиях реального мира, построенные модели нередко подвержены искажению информации об окружающей сцене из-за неоднородности, шумов, неполноты и несогласованности данных, в частности при использовании разных типов датчиков, регистрирующих данные разных модальностей. Поэтому разработка и выбор адекватных метрик, способных количественно охарактеризовать точность, полноту, согласованность и устойчивость моделей

окружающей среды, становится неотъемлемым этапом при анализе и сравнении алгоритмов картографирования.

В данном разделе рассматриваются существующие подходы к оценке качества моделей окружающего пространства. Обсуждаются метрики, разработанные для моделей из классификации, представленной в разделе 1.3.

1.5.1 Метрики для оценки объектно-ориентированных представлений

Среди метрик для объектно-ориентированных представлений широкое применение получил коэффициент Жаккара [60]. Пусть p_a – полигон, полученный в результате работы алгоритма построения модели по полученным с датчиков данным, описывающий форму и положение одного из окружающих БТС объектов. Пусть также p_r – полигон, описывающий эталонные форму и положение того же объекта. Тогда коэффициент Жаккара равен:

$$J = \frac{S(p_a \cap p_r)}{S(p_a \cup p_r)},$$

где $S(p_a \cap p_r)$ – площадь полигона, полученного при пересечении p_a и p_r , $S(p_a \cup p_r)$ – площадь полигона, полученного при объединении p_a и p_r . Таким образом, значение J лежит в диапазоне $[0; 1]$. Чем больше значение J , тем точнее p_a совпадает с p_r и тем качественнее модель. Отметим также, что зачастую в качестве сравниваемых полигонов выступают произвольно ориентированные ограничивающие прямоугольники. Коэффициент Жаккара вычисляется для каждой пары сопоставленных вычисленного и эталонного полигонов, поэтому качество модели на наборе данных характеризуется выборочным распределением величины J по всем таким парам, а для краткого представления результатов приводятся выборочные статистики этого распределения: среднее значение, среднеквадратичное отклонение, минимум, квантили и максимум.

Помимо коэффициента Жаккара при оценке качества моделей можно встретить метрики, используемые при оценке решения задачи классификации. Например, F-score. Для этого вводится некоторый порог $j_{thr} \in (0,1)$. Корректно обнаруженными объектами (верными срабатываниями алгоритма – TP) считаются только те p_a , для которых $J > j_{thr}$; остальные p_a считаются ложными

срабатываниями алгоритма – FP . Наконец, эталонные p_r , для которых не нашлось p_a с достаточным коэффициентом Жаккара, считаются пропусками алгоритма – FN . Итоговый F-score равен:

$$\text{F-score} = \frac{2}{P_c^{-1} + P_a^{-1}},$$

где $P_c = \frac{|TP|}{|TP|+|FN|}$ – полнота, $P_a = \frac{|TP|}{|TP|+|FP|}$ – точность.

Существует также метрика локализационной точности (localization accuracy), которая, как и коэффициент Жаккара, оценивает качество вычисленных p_a , но только для верных срабатываний алгоритма:

$$\text{LocA} = \frac{1}{|TP|} \sum_{c \in TP} J_c$$

Отдельную группу образуют метрики, оценивающие не только положение объектов в сцене (задача детекции объектов), но и постоянство их идентификаторов между кадрами (задача отслеживания объектов). Исторически первой из них является МОТА (Multi Object Tracking Accuracy) [61], объединяющая в одном показателе доли пропусков, ложных срабатываний и переключений идентификатора; её недостатком является преимущественная чувствительность к качеству детекции, а не отслеживания. Метрика IDF1 [62] оценивает качество отслеживания непосредственно: она вычисляется по наилучшему сопоставлению вычисленных и эталонных идентификаторов на всей последовательности кадров. Метрика НОТА (Higher Order Tracking Accuracy) [63] представляет собой среднее геометрическое двух вспомогательных величин – точности детекции DetA и точности ассоциации AssA, проинтегрированное по порогу коэффициента Жаккара, что даёт сбалансированную оценку качества решения обеих задач. Поскольку в настоящей работе задача отслеживания объектов не решается, подробное описание перечисленных метрик здесь не приводится.

1.5.2 Метрики для оценки сеточных представлений

Для сеточных представлений было также предложено множество метрик для оценки их качества, как, например, метрики, представленные в работе Ши [64], обзорающей существующие сеточные представления, алгоритмы их

построения и методы оценки их качества. Многие метрики, применимые для объектно-ориентированных представлений, применимы и для моделей этой категории. Так, бинарные карты проходимости пространства, т.е. карты, клетки которых могут принимать только два значения, оцениваются коэффициентом Жаккара J : площадью пересечения при этом считается число клеток, отнесённых к занятому пространству и на оцениваемой, и на эталонной картах, а площадью объединения – число клеток, отнесённых к занятому пространству хотя бы на одной из них. Если клетки классифицируются более чем на два класса, коэффициент Жаккара вычисляется для каждого класса отдельно и усредняется по классам [65]:

$$mJ = \frac{1}{|K|} \sum_{k \in K} J_k, \quad (1.2)$$

где K – множество классов клеток, J_k – коэффициент Жаккара, вычисленный для класса k . В отличие от доли верно классифицированных клеток, такое усреднение не позволяет высокому качеству описания преобладающего по площади класса (как правило, свободного пространства) маскировать ошибки в описании остальных классов. Также встречается оценка с использованием F-score [66].

Если клетки карты хранят не класс, а непрерывную величину – вероятность занятости либо соответствующий ей логарифм шансов, – качество модели оценивают среднеквадратичной ошибкой относительно эталонной карты:

$$\text{MSE}(M, M_r) = \frac{1}{|M|} \sum_i (p(m_i) - p(m_i^r))^2, \quad (1.3)$$

где M – оцениваемая карта проходимости пространства, M_r – эталонная карта того же размера и разрешения, $p(m_i)$ и $p(m_i^r)$ – вероятности занятости i -й клетки оцениваемой и эталонной карт соответственно, $|M|$ – общее число клеток карты. В отличие от коэффициента Жаккара, эта метрика не требует бинаризации значений клеток и потому учитывает степень уверенности модели в занятости пространства. Однако, как и коэффициент Жаккара, она штрафует ошибку одинаково независимо от того, в какой части карты эта ошибка допущена.

Для некоторых задач важно оценить не только полноту и корректность отображения текущей сцены на карте проходимости пространства, но и качество предсказания её значений в будущем. Такая задача исследуется, например,

в работе Ху [67]: получая на вход набор изображений, зарегистрированных на камерах БТС, предлагаемая в работе модель строит экземплярную сегментацию для текущего момента времени t и предсказывает её для набора будущих моментов времени вплоть до некоторого горизонта H . Для оценки качества создаваемого решения этой задачи в работе использовалось паноптическое качество видео (англ. Video Panoptic Quality) [68]. Данная метрика задаётся следующим выражением:

$$VPQ = \sum_{t=0}^H \frac{\sum_{(x_t, y_t) \in TP_t} J(x_t, y_t)}{|TP_t| + \frac{1}{2}|FP_t| + \frac{1}{2}|FN_t|},$$

где TP_t – совпавшие с эталонными значениями экземпляры, FP_t – экземпляры предсказания, отсутствующие в эталонных данных, FN_t – эталонные экземпляры, отсутствующие среди найденных.

В контексте задачи планирования маршрута перечисленные метрики могут быть не показательны. Действительно, на рисунке 1.1 приведён пример эталонной карты проходимости и двух вариантов карт, которые с точки зрения приведённых метрик имеют одинаковое качество. В то же время легко видеть, что карта 1.16 потенциально опаснее, т.к. согласно ей БТС может продолжать движение вперёд, что в реальности приведёт к столкновению с препятствием.



Рисунок 1.1 — Пример эталонной карты и предсказанных карт с одинаковым качеством согласно метрикам, не учитывающим контекст решаемой задачи.

Чтобы учесть это, в работе [69] была предложена метрика PathFinding Cost Mean Squared Error (PFC-MSE). Для её подсчёта производятся следующие шаги:

1. Для каждого типа клетки выбирается стоимость её «проезда» $c > 0$. Для клеток, соответствующих занятым областям, эта стоимость должна быть значительно выше, чем для свободных клеток (в оригинальной работе стоимость занятых клеток составила $cost_{occ} = 100$, свободных: $cost_{free} = 1$).
2. Для эталонной и оцениваемой карт проходимости строятся ориентированные графы. Вершинами графа являются клетки карты, рёбрами соединены соседние клетки карты, вес ребра – стоимость проезда клетки, являющейся конечной вершиной ребра, умноженная на расстояние между центрами клеток $w(c_1 \rightarrow c_2) = cost(c_2) \cdot \|c_2 - c_1\|_2$ (в оригинальной работе клетки, расположенные по диагонали, также считаются соседними).
3. С помощью алгоритма Дейкстры [70] определяются пути с наименьшим весом $path_cost(c)$ от центральной клетки карты, соответствующей положению БТС на карте.
4. Для каждой клетки вес пути затем нормализуется:

$$path_cost_{final}(c) = \frac{path_cost(c) - L(c)cost_{free}}{cost_{occ} - cost_{free}},$$

где $L(c)$ – длина пути до вершины c .

5. Для построенных таким образом двух изображений (для эталонной и оцениваемой карт) находится среднеквадратичная ошибка, взвешенная вероятностью того, что соответствующая клетка является свободным пространством:

$$PFC-MSE(GT, I) = \frac{\sum_c w_c \|GT_{cost}(c) - I_{cost}(c)\|_2}{\sum_c w_c},$$

где GT_{cost} – построенная карта нормализованных весов путей для эталонной карты, I_{cost} – для оцениваемой карты, $w_c = p(c = free)$.

1.5.3 Метрики для оценки непрерывных представлений

Работы, представляющие модели из категории непрерывных представлений, часто сравниваются с моделями сеточных представлений [71–73]. Поэтому

типичный подход к их оценке следующий: разбить пространство на сетку (двумерную или трёхмерную в зависимости от размерности оцениваемой модели), вычислить преобладающее значение внутри ограниченного клеткой пространства и тем самым преобразовать оцениваемую модель в модель сеточного представления, которую можно сравнивать с аналогами из этой категории.

Для моделей, которые сопоставляют каждой точке пространства плотность вероятности нахождения в ней препятствия, стандартный метод к оценке качества – построение precision-recall (PR) кривой. Каждой клетке после дискретизации пространства соответствует вероятность занятости соответствующего пространства, для вычисления PR-кривой перебираются все возможные значения порога p , по которому бинаризуются значения клеток. Для полученной бинаризации вычисляется точность и полнота:

$$Precision_p = \frac{|TP_p|}{|TP_p| + |FP_p|}$$

$$Recall_p = \frac{|TP_p|}{|TP_p| + |FN_p|}.$$

Полученное множество точек $(Precision_p, Recall_p) \in [0; 1]^2$ образует PR-кривую, площадь под которой (англ. Area Under Curve, AUC) описывает качество построенной модели.

1.6 Выбор модели окружающего пространства и метрик её оценки

Проведённый обзор позволяет остановить выбор на модели окружающего пространства, которая будет использована в работе, и на метриках оценки её качества.

В качестве базовой модели в работе принята карта проходимости пространства из категории сеточных представлений (раздел 1.3). Выбор обоснован тремя обстоятельствами. Во-первых, модель описывает объекты произвольной геометрии и не требует их отнесения к заранее определённым классам, что существенно для БТС, движущегося в неструктурированной среде, где встречаются объекты, не представленные в обучающих выборках детекторов. Во-вторых, она допускает явное представление областей, о проходимости которых информация

отсутствует, что необходимо для безопасного планирования пути в условиях окклюзии. В-третьих, принятый метод её обновления – байесовский вывод с использованием обратной модели наблюдения (формула (1.1)) – сводит учёт данных отдельного источника к сложению слагаемых, вычисляемых по данным этого источника независимо от остальных. Тем самым выбранная модель реализует позднее комплексирование в классификации по стадии обработки данных раздела 1.4 и, в отличие от объектно-ориентированных и непрерывных представлений, принципиально допускает одновременное обновление несколькими источниками данных.

1.6.1 Выбор метрик, используемых в работе

Из рассмотренных метрик в настоящей работе используются следующие. Положение и форма объектов в окружающем пространстве БТС оцениваются коэффициентом Жаккара J , вычисляемым для каждой пары сопоставленных вычисленного и эталонного положений объекта. Обнаружение объектов на изображении выполняется существующей нейросетевой моделью и в число решаемых в работе задач не входит, поэтому предметом оценки является геометрическая точность описания объекта, а не полнота его обнаружения. По этой причине метрики F-score и LocA, характеризующие в первую очередь полноту обнаружения и требующие бинаризации значений J по порогу j_{thr} , не используются: непрерывная величина J позволяет сравнивать качество во всём диапазоне значений, а не только выше выбранного порога. Качество карты проходимости пространства, построенной с учётом визуальных данных, оценивается среднеквадратичной ошибкой относительно эталонной карты (1.3) и метрикой PFC-MSE: первая характеризует поклеточное расхождение моделей, вторая – влияние этого расхождения на решение задачи планирования пути, ради которой модель и строится. Метрики отслеживания объектов и метрики непрерывных представлений в работе не используются, поскольку соответствующие задачи в ней не решаются.

1.7 Обзор методов решения задач, возникающих при построении выбранной модели

В рамках данной диссертационной работы при разработке алгоритмов построения карты проходимости пространства помимо прочих необходимо решить две задачи. Первая связана с тем, что камера – наиболее распространённый датчик БТС – регистрирует данные в системе координат изображения, тогда как модель описывает плоскость движения: результаты обработки изображения требуется спроецировать в пространство модели. Вторая состоит в том, что данные всех источников должны быть учтены в единой карте, и вычислительная стоимость этого учёта определяет, сколько источников может быть задействовано без потери актуальности модели. Для каждой из задач рассмотрим существующие подходы, оценим их применимость в условиях, характерных для бортового программного обеспечения БТС, и выделим ограничения, которые будут положены в основу требований к разрабатываемым алгоритмам (раздел 1.8).

1.7.1 Алгоритмы проецирования данных камеры на плоскость движения БТС

Как отмечено в разделе 1.2, камера является наиболее распространённым датчиком в колёсной робототехнике, однако она не позволяет непосредственно измерить расстояние до наблюдаемых объектов. Результаты обработки изображения, например, визуальные детекции объектов и семантическая сегментация, получаются в однородной системе координат изображения, тогда как двумерные модели окружающего пространства, рассмотренные в разделе 1.3, описывают плоскость движения в трёхмерной системе координат БТС. Поэтому учёт визуальных данных в таких моделях требует отдельного алгоритма проецирования, от точности которого напрямую зависит корректность положения препятствий в итоговой модели.

Изображение задаёт лишь направление на наблюдаемый объект, поэтому для восстановления расстояния до него требуется либо дополнительное предположение о геометрии сцены, либо дополнительный источник данных.

Определяющим для точности проецирования является то, какая модель поверхности движения БТС при этом используется, и по этому признаку существующие подходы разделим на три группы:

1. **Проецирование с использованием геометрических предположений о поверхности движения.** Если внутренняя и внешняя калибровки камеры известны, а поверхность движения считается плоскостью, то соответствие между плоскостью изображения и плоскостью движения задаётся проективным преобразованием. Уравнение данной плоскости может быть фиксированным, либо пересчитываться на данных, регистрируемых датчиками, например, широкое применение получил способ поиска плоскости в облаке точек с лидаров с помощью метода RANSAC [74]. Проективное преобразование применяется как для проецирования всего изображения на вид сверху, так и для проецирования отдельных признаков: нижних границ детекций, разметки, границ проезжей части. Пример системы, использующей данный тип проекции, – система GOLD [40]. Достоинствами методов этой группы являются вычислительная простота и отсутствие необходимости в обучении; кроме того, при использовании единственной камеры иного источника информации о геометрии сцены попросту нет. Основной их недостаток – само предположение о плоскости поверхности движения. Оно нарушается как из-за рельефа местности, так и в силу требований к конструкции дорожного покрытия: согласно СП 34.13330.2012 [75] покрытие имеет двускатный поперечный профиль на прямых участках и односкатный на виражах. Дополнительным источником погрешности является отклонение фактической внешней калибровки от измеренной, вызванное загрузкой БТС, работой подвески и смещением датчиков при длительной эксплуатации БТС. Ошибка проецирования при этом растёт с удалением от камеры.
2. **Проецирование с восстановлением формы поверхности движения.** Если БТС оснащён дальномерным датчиком, предположение о плоскости можно заменить оценкой формы поверхности $z = z(x, y)$, полученной по измерениям, и выполнять проецирование уже на эту поверхность. В качестве примеров такого проецирования выступают методы, основанные на разбиении облака точек сеткой в полярных координатах с последующей аппроксимацией высоты земли в каждом

секторе, предложенные в работах Химмельсбаха [76] и Зермаса [77]; их развитием является алгоритм Patchwork [78], использующий кусочную аппроксимацию поверхности с оценкой достоверности каждого участка. Наиболее общим является представление поверхности земли как реализации гауссовского процесса [79]: данный метод не требует ни плоскостной, ни кусочно-плоскостной модели и позволяет оценить дисперсию предсказания, но обладает большой вычислительной сложностью по отношению к числу точек в обучающей выборке: $O(n^3)$, где n – число точек. Это затрудняет его применение в режиме реального времени. Применение регрессии на основе гауссовских процессов к выделению точек земли рассмотрено в работе Чена [80].

3. **Обучаемые преобразования в вид сверху.** Третью группу образуют нейросетевые методы, выполняющие преобразование признаков изображения в вид сверху без явно заданной модели поверхности: геометрия сцены восстанавливается неявно, в процессе обучения. В работе Филиона [81] для каждого пикселя изображения предсказывается распределение по глубине, после чего признаки проецируются в трёхмерное пространство на сетку вида сверху с учётом веса предсказанного распределения. В работе Ли [82] преобразование реализовано с применением механизма внимания над обучаемыми запросами, соответствующими клеткам сетки. Модель BEVFusion [58; 83] объединяет в едином представлении вида сверху признаки камер и лидаров. Методы этой группы демонстрируют высокое качество, однако требуют графического ускорителя для инференса, размеченных мультимодальных наборов данных для обучения и переобучения при изменении состава и расположения датчиков БТС, что ограничивает их применимость в условиях, рассматриваемых в настоящей работе.

Результатом проецирования любым из перечисленных способов является множество точек либо полигон на плоскости движения БТС. Потребители модели окружающего пространства, как правило, не работают с таким представлением непосредственно, и тому есть две причины. Во-первых, в измерениях датчиков присутствует только обращённая к ним часть объекта, поэтому проекция систематически занижает занимаемую объектом область: недоступная для наблюдения часть объекта не будет учтена ни в модели, ни, следовательно, при планировании пути. Во-вторых, сложившимся интерфейсом между подсисте-

мой восприятия и последующими подсистемами БТС служит описание объекта ориентированным ограничивающим параллелепипедом либо прямоугольником: в этой форме размечены общепринятые наборы данных [25], в ней формулируется задача в обзорной литературе по трёхмерной детекции [24], её же принимают на вход алгоритмы предсказания траекторий окружающих объектов [28] и алгоритмы планирования пути, работающие с представлением препятствий в виде фигур ненулевой площади [84]. Поэтому проецирование, как правило, дополняется этапом вписывания полученной проекции в ориентированную ограничивающую рамку.

Простейшим решением этой задачи является построение рамки минимальной площади, для которого существует алгоритм с линейной по числу вершин выпуклой оболочки вычислительной сложностью [85]. В работе Ванга [86] для кластеров лидарных точек предложено семейство методов, использующих главные компоненты множества точек, наибольшую диагональ и вписанный треугольник. В работе Чжана [87] для той же задачи предложено вписывание в L-образную модель: точки кластера считаются принадлежащими двум смежным ортогональным сторонам объекта, а ориентация рамки определяется перебором угла с фиксированным шагом. Общим ограничением перечисленных методов является то, что они разработаны для кластеров лидарных точек, природа которых отличается от таковых для проекции визуальной детекции; кроме того, определение ориентации рамки перебором угла требует вычислительных затрат, пропорциональных числу проверяемых ориентаций.

Таким образом, существующие подходы к проецированию данных камеры на плоскость движения БТС либо опираются на предположение о плоской поверхности движения и потому вносят растущую с расстоянием погрешность, либо требуют обучения и вычислительных ресурсов, недоступных на бортовом вычислителе при одновременной обработке данных нескольких датчиков. Методы вписывания проекции в ориентированную рамку, в свою очередь, разработаны для лидарных кластеров и не учитывают специфику проекции визуальной детекции. Разработка вычислительно эффективных алгоритмов проецирования визуальных детекций и семантической сегментации проезжей части, не использующих предположение о плоской поверхности движения, а также алгоритма вписывания полученной проекции в ориентированную ограничивающую рамку составляет содержание главы 2.

1.7.2 Влияние числа источников данных на производительность алгоритмов построения карты проходимости пространства

В разделах 1.2 и 1.4 показано, что увеличение числа и разнообразия датчиков повышает полноту и достоверность модели окружающего пространства. Однако каждый дополнительный источник данных увеличивает вычислительную нагрузку на бортовой вычислитель БТС, а следовательно, и задержку между моментом регистрации данных и моментом, когда эти данные учтены в модели. Рассмотрим, как эта задержка формируется и какие ограничения она накладывает.

Обозначим через Δt время между регистрацией данных датчиком и завершением обновления модели. Процесс обновления можно условно разделить на четыре составляющие: передачу данных от датчика к вычислителю; извлечение из них признаков (визуальных детекций, семантической сегментации изображения, семантической сегментации облака точек и так далее); преобразование признаков в общий для всех источников формат; добавление результатов измерения в итоговую модель. Первые три составляющие выполняются для каждого источника независимо и потому допускают параллельное вычисление. Последняя составляющая является операцией над разделяемым ресурсом – общей картой проходимости пространства – и потому, в общем случае, требует синхронизации потоков выполнения либо реализации всего алгоритма с помощью последовательных вычислений. Так, в библиотеке инструментов для навигации робототехнических систем `navigation2` [88] построение карты проходимости пространства реализовано в пакете `costmap_2d`. В данном пакете карта представлена набором слоёв, каждый из которых соответствует отдельному источнику данных. С заданной периодичностью доступ к слоям на запись блокируется для агрегации их в единую модель и передачи результата получателям. Другие библиотеки `grid_map` [89] и `OctoMap` [90], реализующие схожий функционал, не содержат встроенных средств блокировок и не являются потокобезопасными.

Параллельные вычисления при построении сеточных моделей окружающего пространства применяются, однако на других уровнях. Первый из них – вычисление одного обновления карты: в работе Хомма [91] на графическом ускорителе распараллелено построение карты занятости по данным лидара и

радар, а в работе Нусса [30] тем же способом реализовано построение динамической карты проходимости пространства в реальном времени; вопрос одновременного доступа к карте нескольких независимых источников данных в этих работах не рассматривается. Второй уровень – независимые слои карты, как в описанном выше пакете `costmap_2d`: слои, соответствующие разным источникам данных, обновляются независимо друг от друга, однако сведение их в общую сетку выполняется последовательно. Наконец, самостоятельное направление образуют неблокирующие структуры данных, допускающие одновременное изменение общего объекта несколькими потоками выполнения без использования примитивов взаимного исключения [92; 93].

В рассмотренных библиотеках одновременное обновление общей сетки несколькими источниками данных не поддерживается: доступ к ней либо сериализуется, либо потокобезопасность не обеспечивается вовсе и возлагается на пользователей. Неблокирующие структуры данных к разделяемой сетке карты проходимости пространства при этом не применялись. Причина этого состоит не в сложности их реализации: локальная карта, следующая за БТС, требует операций, затрагивающих её целиком, – сдвига вслед за перемещением БТС и приведения всех клеток к общему моменту времени. Такие операции по своей природе не сводятся к независимым изменениям отдельных клеток, поэтому применение неблокирующих структур данных требует изменения не механизма синхронизации, а самого представления модели.

Оценим количественно, к каким ограничениям приводит последовательный учёт данных в карте. Размер карты выбирается из требования безопасной остановки: если препятствие обнаружено на границе карты, БТС должен успеть плавно затормозить до его достижения. Пусть R – расстояние от БТС до границы карты, a – ускорение при плавном торможении БТС. Тогда, чтобы БТС успел остановиться перед обнаруженным на краю карты препятствием, должно выполняться неравенство:

$$v_{max} < \sqrt{2aR}. \quad (1.4)$$

Полученная оценка является оценкой сверху, так как не учитывает время реакции системы автопилотирования на обновление карты проходимости пространства. Пусть res – разрешение карты, тогда при фиксированном расстоянии до границы карты R число клеток в ней составляет $N = (2R/res)^2$. Подставив v_{max} из формулы (1.4), получаем $N \propto v_{max}^4$. В главе 4 будет показано,

что в базовой реализации учёт данных одного источника в карте проходимости пространства требует $O(N)$ операций. Условие работы алгоритма построения карты в режиме реального времени состоит в том, что все S измерений источников должны быть учтены до поступления следующих. Для последовательного обновления всеми имеющимися источниками данных это требование описывается в виде неравенства $ST(N) \leq 1/f$, где $T(N)$ – время учёта в карте с N клетками данных одного источника; f – частота работы источников данных. Совмещая эти зависимости, получаем, что предельное число одновременно учитываемых источников данных обратно пропорционально четвёртой степени рабочей скорости БТС:

$$\frac{S_{max}(v_2)}{S_{max}(v_1)} = \left(\frac{v_1}{v_2}\right)^4. \quad (1.5)$$

Результаты расчёта по (1.5) для скоростей 20, 40, 60 и 80 км/ч приведены в таблице 2. 40 км/ч используются в качестве референса. Можно видеть, что увеличение скорости с 40 до 80 км/ч сокращает число источников данных, которые могут быть учтены в модели, в 16 раз.

Таблица 2 — Изменение предельного числа одновременно учитываемых источников данных с ростом рабочей скорости БТС

Рабочая скорость БТС v_{max}	20 км/ч	40 км/ч	60 км/ч	80 км/ч
Расстояние до границы карты R , м	7,7	30,9	69,4	123,5
Размер карты N (относительно 40 км/ч)	−93,75 %	1	+406 %	+1500 %
Число источников S_{max} (относительно 40 км/ч)	+1500 %	1	−80,25 %	−93,75 %

Чтобы представить порядок абсолютных значений, отметим следующее. Используемая в настоящей работе карта проходимости пространства имеет размер 600×600 клеток при разрешении 0,1 м, что согласно (1.4) при $a = 2 \text{ м/с}^2$ соответствует рабочей скорости около 40 км/ч. Приняв для карты такого размера $T(N) \approx 3 \text{ мс}$ – порядок величины, полученный при профилировании в разделе 4.1, – и частоту работы источников $f = 10 \text{ Гц}$, получаем, что одновременно может быть учтено порядка трёх десятков источников данных. Для

БТС этого заведомо достаточно. Однако при попытке двигаться со скоростью 80 км/ч то же соотношение оставляет лишь два источника данных, чего недостаточно даже для описания окружающего пространства по данным одного датчика с несколькими обработчиками.

Один из способов увеличить число источников данных с сохранением работы в режиме реального времени – обеспечить возможность одновременного обновления карты несколькими источниками данных. Это требует, во-первых, программной архитектуры, позволяющей подключать произвольное число источников данных без изменения кода построения карты (глава 3), и, во-вторых, соответствующей модификации самого алгоритма построения карты (глава 4).

1.8 Требования к разрабатываемым модели окружающего пространства и алгоритмам её построения

Выбор модели окружающего пространства и обзор методов решения возникающих при её построении задач, проведённые в разделах 1.2–1.7.2, позволяют сформулировать требования к разрабатываемым модели окружающего пространства и алгоритмам её построения.

1. алгоритмы построения модели должны обеспечивать комплексирование данных камер и лидаров, поскольку ни один из этих типов датчиков в отдельности не позволяет одновременно получить высокое разрешение и непосредственное измерение расстояния (раздел 1.2);
2. алгоритмы построения модели не должны опираться на предположение о совпадении поверхности движения с плоскостью;
3. разрабатываемые алгоритмы должны выполняться в режиме реального времени на бортовом вычислителе БТС при ограниченных вычислительных ресурсах, что исключает применение методов, требующих инференса крупных нейросетевых моделей для каждого источника данных (разделы 1.7.1, 1.7.2);
4. модель должна допускать подключение произвольного числа источников данных;

5. модель окружающего пространства и алгоритм её построения должны предоставлять возможность одновременного обновления модели несколькими источниками данных.

Разработке моделей и алгоритмов, удовлетворяющих данным требованиям, посвящены последующие главы данной работы.

1.9 Заключение к главе 1

Итак, в рамках данной главы было выполнено следующее:

1. определены типичные виды датчиков, используемых на БТС для построения и уточнения моделей окружения, выделены преимущества и ограничения каждого из них. Сделан вывод о необходимости комплексирования данных с датчиков для построения более полной и точной модели окружения;
2. проведён обзор существующих моделей представления окружающего пространства вокруг БТС и существующих методов комплексирования данных;
3. определены основные метрики, используемые для оценки качества различных моделей представления окружающего пространства;
4. в качестве базовой модели окружающего пространства выбрана карта проходимости пространства и обоснован выбор метрик, применяемых в настоящей работе для оценки качества предлагаемых алгоритмов;
5. проведён обзор алгоритмов проецирования данных камеры на плоскость движения БТС. Показано, что существующие алгоритмы либо опираются на предположение о плоской поверхности движения, либо требуют вычислительных ресурсов, недоступных на бортовом вычислителе при одновременной обработке данных нескольких датчиков;
6. рассмотрено влияние числа источников данных на производительность алгоритмов построения карты проходимости пространства. Показано, что в существующих реализациях данные источников учитываются в карте последовательно, и получена оценка, связывающая предельное число одновременно учитываемых источников данных с рабочей скоростью БТС;

7. сформулированы требования к разрабатываемым в последующих главах работы алгоритмам.

Глава 2. Алгоритмы проецирования визуальных данных на плоскость движения БТС

В главе 1 была показана необходимость комплексирования данных для построения более точной модели окружающего пространства. В данной главе предлагаются алгоритмы, реализующие комплексирование данных с RGB-камер и лидаров, установленных на БТС. Вначале предложен алгоритм оценки положения объектов, обнаруженных на изображении с камеры, в трёхмерном пространстве с использованием только данных с камеры и показана ограниченность качества его работы. Затем предложен алгоритм для решения той же задачи, но с использованием комплексирования данных с камеры и лидаров. Наконец, предложен алгоритм проецирования информации о свободной проезжей части с изображения на плоскость движения БТС и учёта полученной проекции для улучшения базового алгоритма построения карты проходимости пространства.

2.1 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе модели L-Shape

В данном разделе описан метод определения положения транспортных средств (наиболее распространённого типа объектов окружения в городских условиях) в виде произвольно ориентированных ограничивающих рамок на виде сверху (bird's-eye view) по изображению, полученному с одной монокулярной камеры. Этот метод состоит из двух этапов. На первом этапе вычисляется проекция видимой границы транспортного средства в виде сверху на основе двумерных детекций (2D-детекций) препятствий и сегментации проезжей части на изображении. Предполагается, что полученная проекция представляет зашумлённые измерения двух ортогональных сторон ТС. На втором этапе строится ориентированная ограничивающая рамка вокруг полученной проекции. Для этого этапа будет предложен новый алгоритм построения рамки на основе предположения об L-образности проекции: L-Shape алгоритм.

2.1.1 Этап 1. Проекция двумерной детекции изображения на вид сверху

Пусть имеется изображение, зарегистрированное камерой БТС, с известной внутренней и внешней калибровкой. На этом изображении производится двумерное детектирование ТС и семантическая сегментация проезжей части. Используя полученные данные, мы хотим достроить положение, ориентацию и габариты обнаруженных ТС на виде сверху. Далее описаны два подхода к определению границ окружающих ТС на плоскости движения БТС.

Наивный подход

Пусть дан набор ограничивающих рамок на зарегистрированном изображении, полученный с помощью нейросетевой модели YOLOP [94], осуществляющей детекцию ТС на изображении. Для дальнейшего определения положения обнаруженных ТС вводятся два предположения:

1. Все объекты находятся на земле (проезжей части дороги).
2. Поверхность дороги плоская. Далее в работе используется система координат, в которой уравнение плоскости, в которой лежит данная поверхность, имеет вид $z = 0$.

Для оценки 3D-координат ТС, мы проецируем два нижних угла ограничивающей рамки на плоскость дороги (рис. 2.1). Мы считаем известными параметры внутренней и внешней калибровок, поэтому данная операция происходит в два шага: сперва мы находим однородные координаты p_u интересующих нас точек в локальной системе координат камеры, а затем, используя внешнюю калибровку, проецируем полученные точки на плоскость движения БТС: $z_l = 0$ в системе O_l , получая окончательный результат p_l . Алгоритмы планирования пути зачастую работают с представлением объектов в виде геометрических фигур с ненулевой площадью или объёмом [84]. Для таких алгоритмов мы производим достраивание полученного отрезка до прямоугольника, добавлением к нему ортогональных отрезков фиксированной длины.

Данный алгоритм достаточно хорошо справляется с проецированием небольших объектов, например, людей, а также объектов, которые находятся в центре кадра и ориентированы вдоль оптической оси камеры. Однако, если объект большой и его границы не параллельны осям изображения, алгоритм проецирует его неверно, что может вызвать в алгоритмах планирования

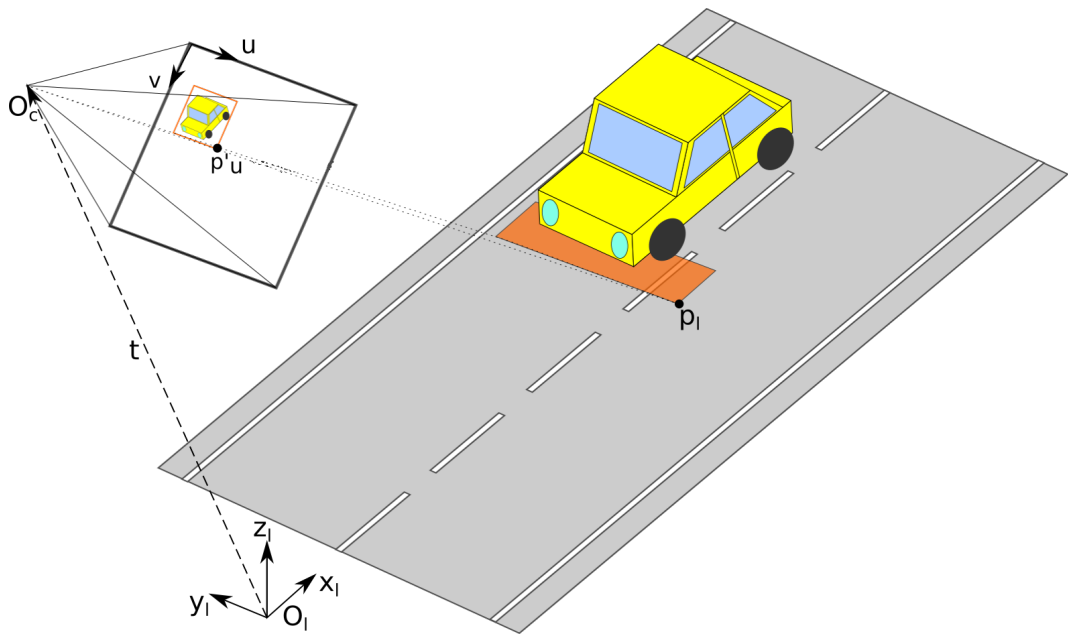


Рисунок 2.1 — Наивный подход к проецированию ограничивающей рамки в 2D на вид сверху.

движения, использующих выходные данные из нашего алгоритма, решение произвести остановку там, где она не требуется, т.к. возможен объезд. Пример такой ситуации показан на рис. 2.2.

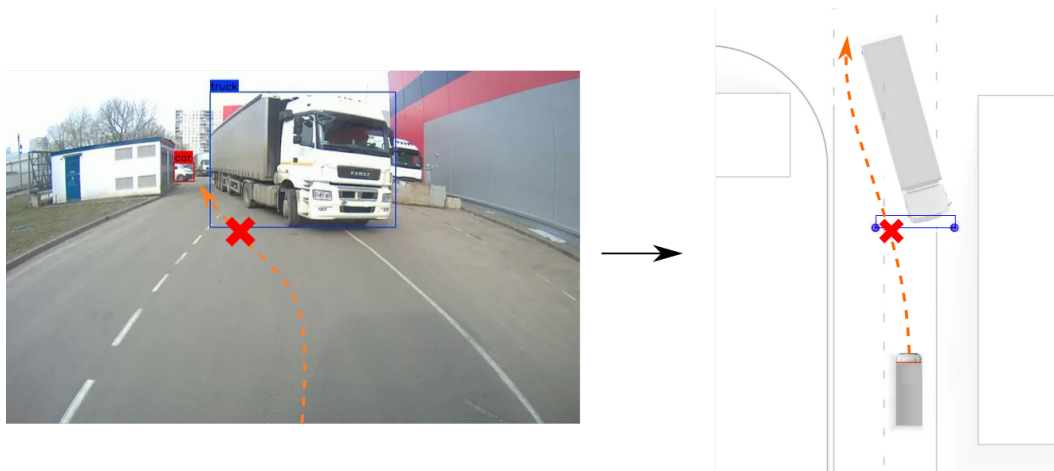


Рисунок 2.2 — Неверная проекция грузовика. Зона слева от грузовика воспринимается, как часть объекта, и мешает объехать его.

Уточнение положения объекта с помощью семантической сегментации проезжей части

Для решения продемонстрированной проблемы, мы дополнительно уточняем положение детектируемых объектов с учётом сегментации проезжей части (также полученной с помощью нейросетевой модели YOLOP). В результате наш алгоритм выглядит следующим образом:

1. Берём N равномерно распределённых точек на нижней границе ограничивающей рамки.
2. Каждая полученная точка сдвигается к верху изображения пиксель за пикселем, пока она не выйдет за пределы проезжей части либо не достигнет верхнего края ограничивающей рамки.
3. Выпуклая оболочка полученных точек проецируется на дорогу аналогично наивному подходу.

На рис. 2.3 приведена демонстрация принципа работы алгоритма.

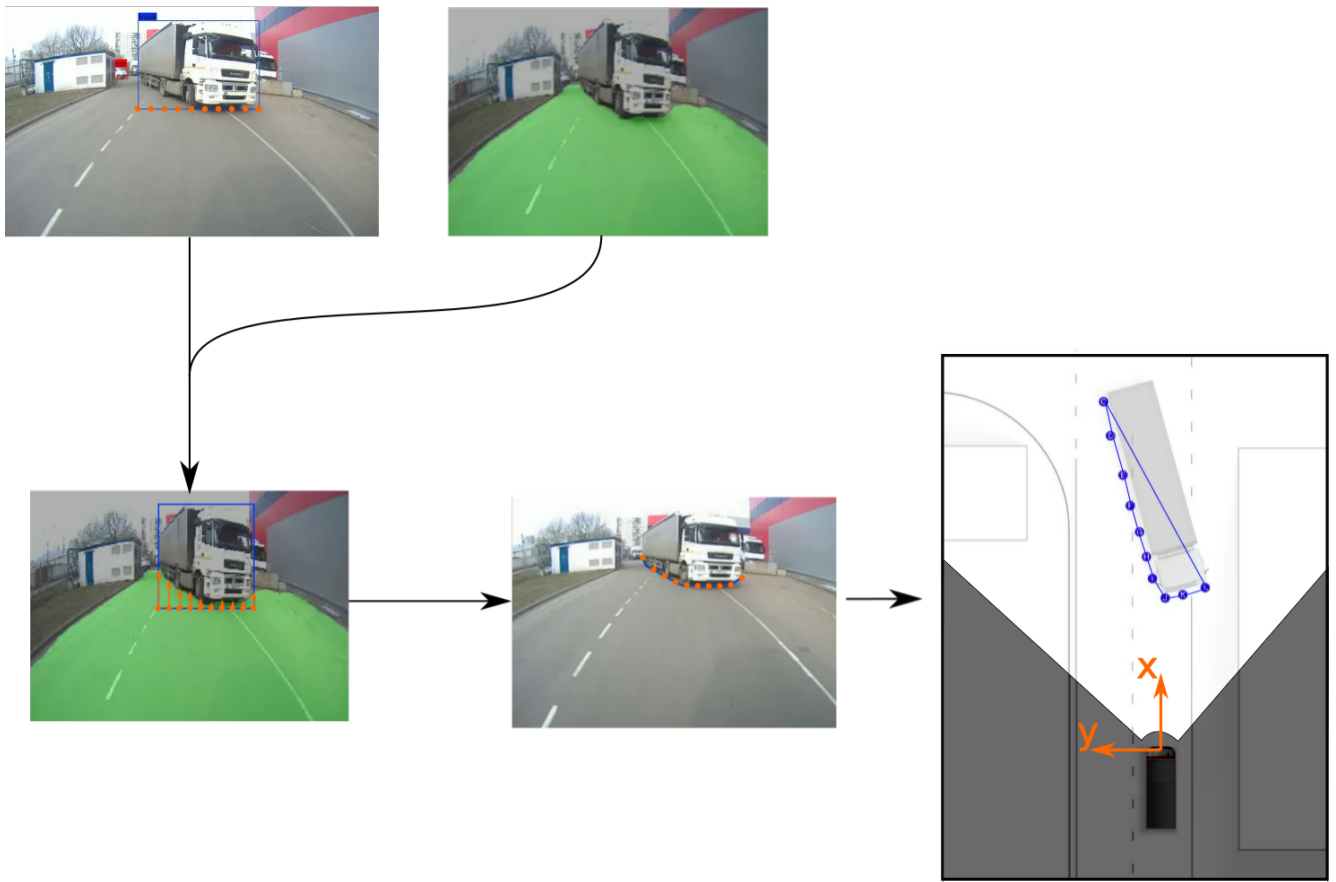


Рисунок 2.3 — Проекция двумерной ограничивающей рамки на вид сверху с учётом сегментации проезжей части.

Несмотря на недостатки этого метода, включая ошибки, вызванные изменяющимся зазором между ТС и дорогой, он может быть применим как способ построения одного из кандидатов для более сложных детекторов, учитывающих результаты работы нескольких алгоритмов, как это сделано в работе [86].

2.1.2 Этап 2. Вписывание проекции объекта на виде сверху в ориентированную ограничивающую рамку

На следующем этапе алгоритма полученная на прошлом шаге проекция вписывается в ориентированную рамку на плоскости дороги. Такое представление упрощает работу последующим алгоритмам, работающим с полученными трёхмерными детекциями, в частности некоторым алгоритмам планирования пути и многим алгоритмам отслеживания объектов. Помимо этого, из-за окклюзии нет возможности видеть некоторые части детектируемого ТС, из-за чего получаемая проекция покрывает только часть реально занимаемого им пространства.

Для построения ограничивающей рамки мы делаем дополнительные предположения об L-образной форме полученной проекции:

1. Вписываемые в рамку точки – зашумлённые точки, принадлежащие двум смежным и ортогональным сторонам ТС.
2. Точки упорядочены таким образом, что сперва идут точки, принадлежащие одной стороне ТС, затем точки второй стороны.

Далее в работе данную модель мы будем именовать L-Shape моделью. Используя данную модель, нахождение ограничивающей рамки сводится к поиску двух перпендикулярных прямых: $l : y = kx + d_1$ и $l' : x = -ky + d_2$, таких, что минимизируется сумма квадратов отклонения (Residual Sum of Squares, RSS) вписываемых точек от соответствующей прямой. Согласно выбранной модели:

$$\begin{aligned} \exists m \in \{1, \dots, N - 1\} \hookrightarrow \forall i \in \{1, \dots, m\}, p_i = (x_i, y_i) \in l; \\ \forall i \in \{m + 1, \dots, N\}, p_i = (x_i, y_i) \in l' \end{aligned}$$

При этом заранее значение m неизвестно. При фиксированном m задача минимизации приводится к стандартному виду

$$\operatorname{argmin}_a (Xa - y)^2,$$

где $a = (k, d_1, d_2)^T$ – вектор искомых параметров, строки матрицы X и столбца y составлены из координат вписываемых точек в зависимости от того, к какой из двух прямых точка отнесена (полный вид X и y приведён в приложении А). Решением данной задачи минимизации является:

$$a^* = (X^T X)^{-1} X^T y \quad (2.1)$$

$$\text{RSS} = a^{*T} X^T X a - 2a^{*T} X^T y + y^T y \quad (2.2)$$

Таким образом, неэффективная реализация модели L-Shape следующая:

1. Перебираем все значения $m \in \{1, \dots, N - 1\}$
2. Пересчитываем матрицы X, y
3. Находим a^* и среднеквадратичную ошибку для каждой задачи минимизации по формулам (2.1), (2.2)
4. В качестве финального решения для l, l' выбираем a^* с наименьшей среднеквадратичной ошибкой.
5. Обрамляем вписываемый набор точек линиями, параллельными l и l' . Область, ограниченная полученными линиями, – искомая ограничивающая рамка.

Вычисление шагов 2 и 3 занимает $\Theta(N)$ операций. Эти шаги повторяются в цикле $N - 1$ раз. Итоговая сложность такой реализации – $\Theta(N^2)$. Данный алгоритм допускает существенное ускорение. Как показано в приложении А, все элементы матриц $X^T X$ и $X^T y$ выражаются через семь сумм координат вписываемых точек, а при переходе от m к $m + 1$ каждая из этих сумм пересчитывается за $\Theta(1)$ операций: очередная точка «перемещается» с одной прямой на другую, и её вклад вычитается из одной суммы и добавляется в другую. Зная эти суммы, вычисление (2.1), (2.2) также занимает фиксированное, не зависящее от N , количество операций. Итоговая временная сложность алгоритма: $\Theta(N) + (N - 1)\Theta(1) = \Theta(N)$. Алгоритм, вычисляющий ориентированную ограничивающую рамку, описанным выше способом, мы далее будем называть L-Shape алгоритмом.

2.2 Исследование точности алгоритма проекции визуальных детекций на основе модели L-Shape

Для оценки качества второго этапа алгоритма мы сравнили его с подходом, минимизирующим площадь вычисляемой рамки, а также с алгоритмами построения рамок для кластеров точек из статьи [86], адаптированными под проекции на вид сверху.

2.2.1 Набор данных

Сравнение алгоритмов выполнялось на наборе данных, полученном с БТС. Данный набор включает в себя реальные сцены из четырёх различных локаций (складов открытого типа и закрытых территорий) и содержит 12 последовательностей кадров, снятых в разные сезоны и время суток. Суммарный размер набора составил 93 кадра. Примеры сцен из набора приведены на рис. 2.4. Каждая сцена, помимо RGB-изображений с монокулярной камеры содержит облака точек, полученные с двух 32-лучевых лидаров, в которых были вручную аннотированы (обрамлены ограничивающими параллелепипедами) все автомобили и пешеходы. Проекции полученных таким образом ограничивающих параллелепипедов автомобилей на вид сверху были взяты в качестве эталонных положений объектов.



Рисунок 2.4 — Примеры сцен из набора данных, использованного для сравнения алгоритмов.

2.2.2 Сравнимые алгоритмы

Для сравнения работы L-Shape алгоритма были взяты пять альтернативных подходов к решению задачи вписывания полигона в ориентированную ограничивающую рамку, рассмотренных в разделе 1.7.1: построение рамки минимальной площади [85] и четыре подхода из работы Ванга [86] – Rotating PCA, Diagonal PCA, Longest Diagonal и Rotated Triangle. Последние были разработаны для трёхмерной детекции в виде ориентированных ограничивающих параллелепипедов вокруг кластеров облаков точек: точки кластера проецировались на вид сверху, для полученной проекции строилась выпуклая оболочка, и далее выполнялось её вписывание в ориентированную рамку на виде сверху (которая позже может быть достроена до параллелепипеда минимальной высоты, включающего все точки исходного кластера). Так как в качестве входа алгоритма использовались не кластеры точек, а проекции объектов на вид сверху, полученные на первом этапе, эти подходы были адаптированы под такие входные данные. Отметим также, что в работе [86] получаемые перечисленными подходами детекции затем передавались в нейросетевую модель для создания итоговой трёхмерной детекции; в настоящей работе эта модель не рассматривается, поскольку исследуются вычислительно эффективные алгоритмы, не требующие наличия графических ускорителей.

Построение рамки минимальной площади

Тривиальным алгоритмом, решающим описанную задачу, является построение рамки, которая включает в себя все вписываемые точки и имеет наименьшую площадь. Пусть $(x_i, y_i), i \in \{1, \dots, N\}$ – вершины выпуклой оболочки, указанные в порядке обхода. Алгоритм построения рамки минимальной площади опирается на то, что хотя бы одна сторона выпуклой оболочки принадлежит одной из сторон рамки. Для двумерного случая существует алгоритм с линейной временной сложностью по числу вершин N [85]. Этот алгоритм правильно определяет положение транспортного средства (ТС) на дороге, только если среди точек оболочки есть хотя бы по одной точке каждой стороны ТС. На практике нам редко удаётся получить достаточное количество точек двух сторон, не говоря уже о трех сторонах. В таком случае, алгоритм выдаст неверное положение объекта, что может привести к нежелательным остановкам автономного ТС, использующего такой алгоритм, или, что ещё хуже, автономная

система может неправильно спланировать траекторию объезда препятствия. Пример такой ситуации представлен на рис. 2.5. Несмотря на недостатки метода, он может быть применен как способ построения одного из кандидатов для более сложных детекторов, учитывающих результаты работы нескольких алгоритмов, как это сделано в работе [86].

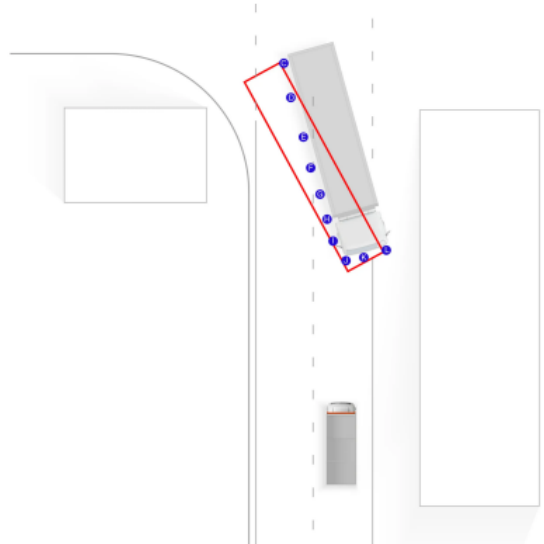


Рисунок 2.5 — Пример неправильно построенной ограничивающей рамки.

Алгоритмы построения рамок для кластеров точек, адаптированные под проекции на вид сверху

Из статьи Ванга [86] были взяты и адаптированы следующие алгоритмы вписывания полигона в ориентированную ограничивающую рамку:

- **Rotating Principal Component Analysis Algorithm.** Идея алгоритма заключается в том, чтобы взять собственные векторы, полученные в результате анализа главных компонент, и построить ограничивающую рамку минимального размера так, чтобы стороны этой рамки были параллельны полученным векторам.
- **Diagonal Principal Component Analysis Algorithm.** Этот алгоритм похож на предыдущий с той разницей, что собственный вектор, соответствующий наибольшему собственному значению, берётся за основу диагонали рамки, а не её сторон. Поэтому после того, как собственный вектор найден, алгоритм подбирает две точки оболочки, которые образуют прямую, наиболее близкую к направлению вектора. Затем алгоритм выбирает самую дальнюю от этой прямой точку и принимает эту точку за угол ограничивающей рамки. Ограничивающая рамка за-

тем подгоняется и расширяется таким образом, чтобы заключить в себе все точки оболочки.

- **Longest Diameter Algorithm.** И снова этот алгоритм похож на предыдущий, но на этот раз диагональ ограничивающей рамки строится по самым дальним точкам оболочки, а собственный вектор не принимается во внимание.
- **Rotated Triangle Algorithm.** В этом алгоритме перебираются пары соседних вершин выпуклой оболочки. Для каждой пары находится третья точка, максимально удалённая от прямой, проходящей через эту пару. Полученные три точки образуют треугольник. Среди всех построенных таким образом треугольников выбирается треугольник наибольшей площади, и соответствующая ему пара соседних вершин оболочки принимается за принадлежащую одной из сторон искомой рамки. Исходя из этого, можно достроить и расширить рамку, как это делалось при работе с другими алгоритмами.

2.2.3 Описание эксперимента

На изображениях из нашего набора данных запускался первый этап алгоритма, вычисляющий проекцию препятствия на вид сверху. Для сопоставления полученных проекций с эталонными данными вычислялся коэффициент Жаккара [60] для каждой пары проекция–эталон, после чего применялся Венгерский алгоритм [95].

Для каждой полученной проекции затем запускались все сравниваемые алгоритмы для второго шага алгоритма (L-Shape, рамка минимальной площади и алгоритмы из работы Ванга [86]), и для результирующего положения препятствия измерялся коэффициент Жаккара. Примеры полученных проекций, построенных ограничивающих рамок и эталонных положений представлены на рис. 2.6.

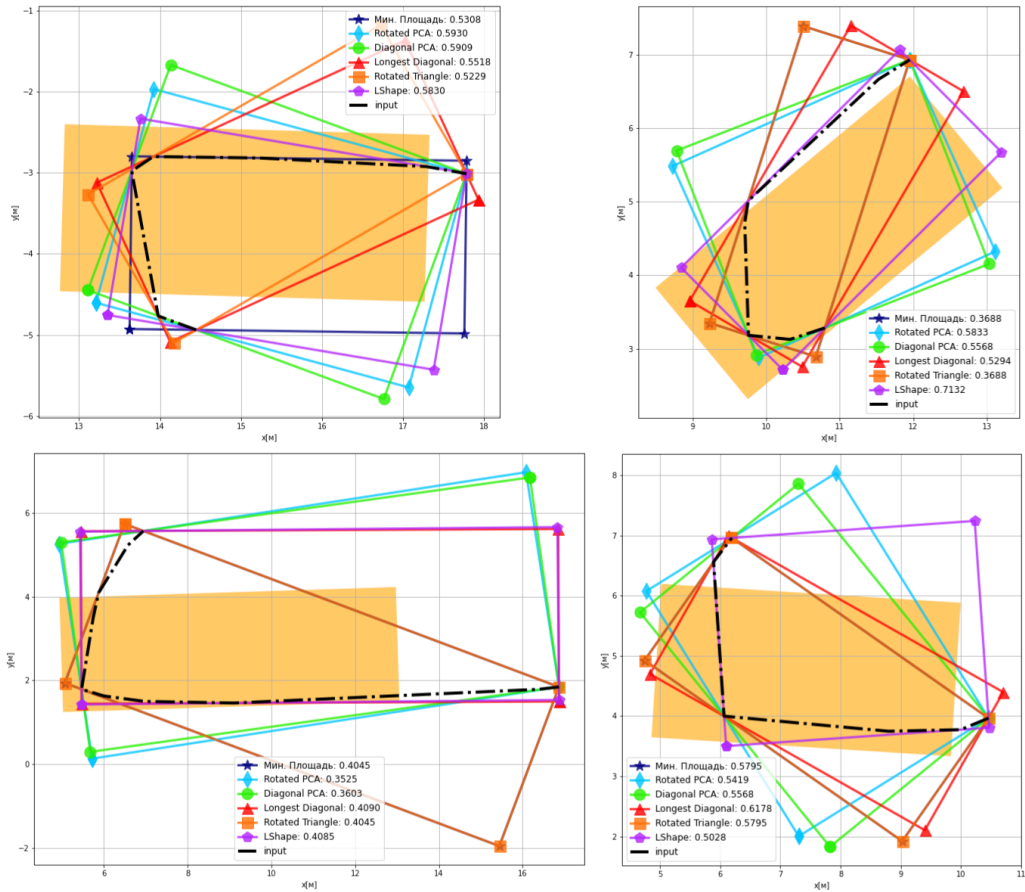


Рисунок 2.6 — Примеры работы сравниваемых алгоритмов построения ограничивающих рамок.

2.2.4 Результаты экспериментов

В таблице 3 представлена статистика измеренных коэффициентов Жаккара для каждого алгоритма. Можно видеть, что представленный алгоритм L-Shape превосходит по качеству другие исследуемые алгоритмы. Худшие результаты показал алгоритм построения рамки минимальной площади. Однако само значение среднего коэффициента Жаккара сравнительно невелико в том числе и для представленного алгоритма L-Shape. Объясняется это в первую очередь ограниченной применимостью предположения о плоскости поверхности дороги, а также неидеальностью внешней калибровки камеры, вызванной наклоном дороги, загруженностью транспортного средства, изменившимся объёмом шин и т.п. В результате предполагаемая плоскость дороги отклоняется от плоскости идеальной аппроксимации дороги и для удаляющихся препятствий наблюдается накопление ошибки. Помимо этого, погрешность первой части

алгоритма: вычисления проекции объекта на вид сверху, выше для более удалённых объектов из-за линейной перспективы изображения. Чтобы учесть это, было предложено ограничить рабочее расстояние алгоритмов: не учитывать препятствия далее некоторого фиксированного порога. На рис. 2.7 представлен график зависимости среднего коэффициента Жаккара от данного порога. Видно, что качество предсказываемого положения препятствия начинает падать после 10 метров для всех рассматриваемых алгоритмов. Также можно видеть, что при всех значениях L-Shape алгоритм сохраняет первенство по измеряемой метрике.

Характеристика	Мин. площадь	Rotating PCA	Diagonal PCA	Longest Diagonal	Rotated Triangle	L-Shape
Среднее	0,294	0,328	0,321	0,323	0,301	0,337
Среднеквадратичное отклонение	0,146	0,144	0,149	0,156	0,141	0,141
Минимум	0,059	0,075	0,078	0,074	0,059	0,078
25 %	0,198	0,219	0,202	0,199	0,208	0,230
50 %	0,257	0,316	0,316	0,287	0,264	0,328
75 %	0,383	0,448	0,445	0,442	0,425	0,451
Максимум	0,649	0,607	0,622	0,651	0,579	0,713

Таблица 3 — Статистики полученных коэффициента Жаккара для сравниваемых алгоритмов. Наилучшие значения выделены полужирным.

2.3 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе комплексирования визуальных и лидарных данных

Одним из фундаментальных ограничений описанного в прошлом разделе L-Shape алгоритма является предположение о том, что поверхность проезжей части лежит в плоскости. Как отмечено в разделе 1.7.1, в реальном мире это предположение не выполняется ни в силу рельефа местности, ни согласно требованиям к конструкции дорожного покрытия [75], а вносимая им погрешность растёт с удалением от камеры. Примеры оформления поперечного профиля дороги приведены на рис. 2.8.

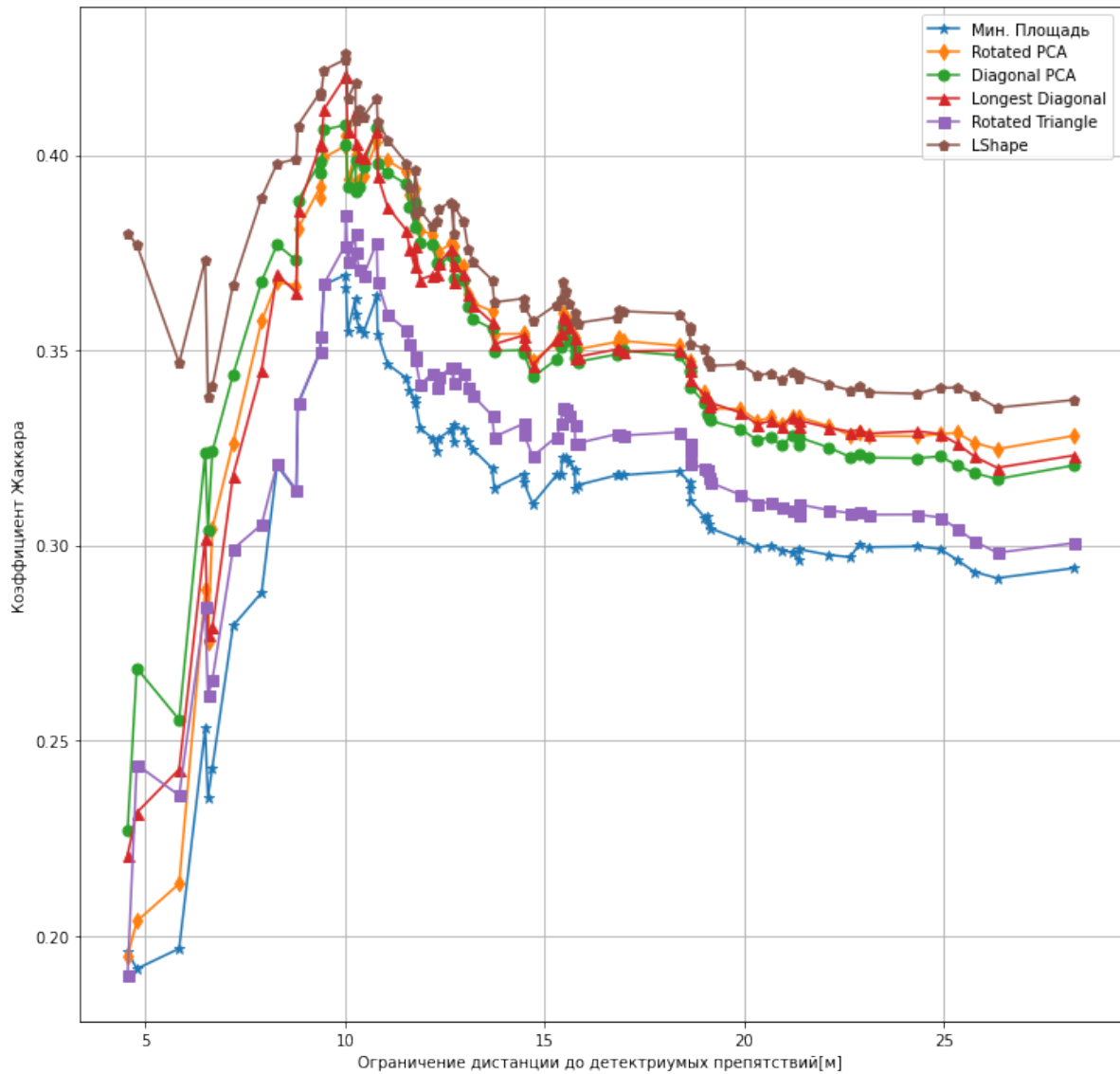


Рисунок 2.7 — График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий.

Для преодоления данного ограничения необходимо добавить источник информации, позволяющий оценить расстояние до наблюдаемых объектов без использования упомянутого предположения. В качестве такого источника информации были использованы механические лидары (лазерные дальномеры), установленные на БТС. Был разработан алгоритм комплексирования данных, полученных с камеры и лидаров, для оценки положения окружающих БТС объектов. Облако точек лидара представляет собой набор точек в трёхмерной системе координат лидара: $P_s = \{(x_i, y_i, z_i), i \in \{1, \dots, N_s\}\}$, где s — идентификатор лидара, N_s — количество зарегистрированных им точек. Все облака точек, а также изображения, полученные камерой, имеют временную метку момента регистрации датчиком. Сенсоры работают с фиксированной частотой и син-

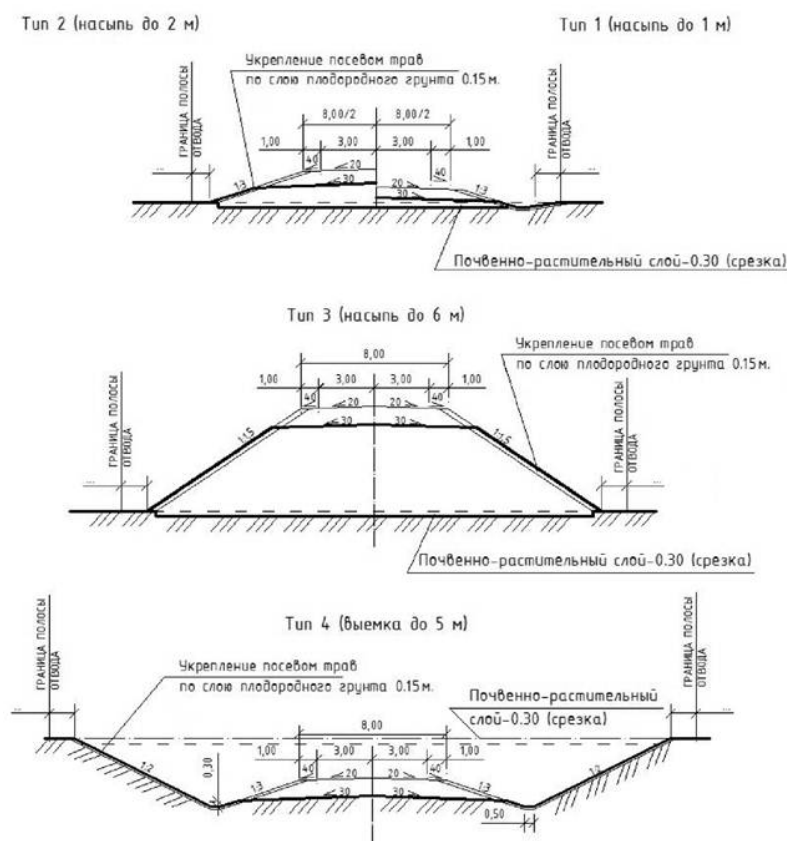


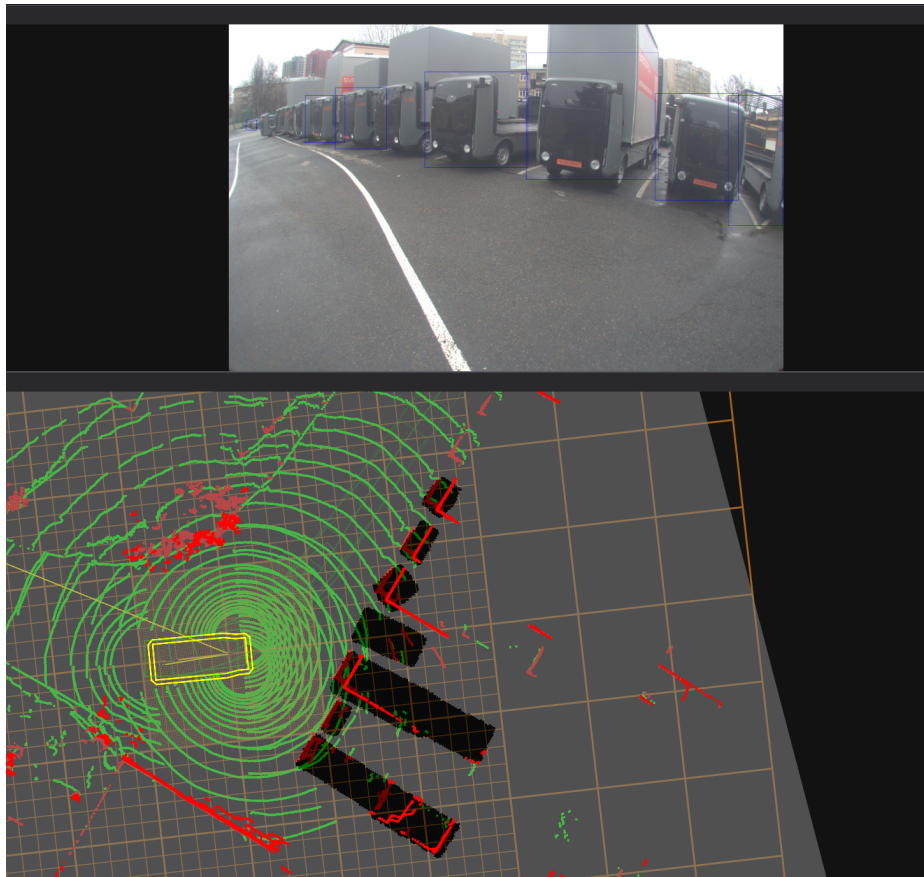
Рисунок 2.8 — Примеры оформления поперечного профиля из ГОСТ 21.701-2013 [96].

хронизированы друг с другом. Это позволяет однозначно сопоставить данные из последовательности измерений с датчиков в единые «кадры», описывающие окружающую БТС сцену в конкретный момент времени. Далее будем считать, что все облака точек с каждого лидара:

1. Приведены в локальную систему координат БТС O_l .
2. Точки, попадающие на БТС, удалены, что возможно при известной геометрии БТС.
3. Положение точек скорректировано и приведено к одному моменту времени, чтобы компенсировать собственное движение БТС во время регистрации облака точек.
4. После предыдущих манипуляций облака объединены в одно общее облако точек P .

После этих преобразований, один «кадр» представляет собой пару изображение–облако точек. В Листинге 1 представлен алгоритм осуществляющий комплексирование данных одного кадра для оценки положения препятствий в окружающей БТС сцене: из облака выделяется подмножество точек, не принадлежащих земле, производится объединение близко расположенных точек в

кластеры, которые затем сопоставляются с двумерными детекциями, найденными на изображении. Наконец, кластеры, которым нашлось сопоставление среди детекций, дополнительно фильтруются от точек, проецируемых за пределы соответствующей детекции, и находится ограничивающая рамка, описанная вокруг полученного набора точек. На рисунке 2.9 показан результат работы представленного алгоритма на данных, полученных с датчиков БТС. Отметим, что шаг 3 алгоритма 1 был реализован в виде алгоритма с использованием параллельных вычислений на платформе CUDA (англ. Compute Unified Device Architecture) [97]. Данная реализация была предложена в работе Нгуена [98].



На верхнем изображении с камеры БТС прямоугольниками выделены двумерные детекции, полученные моделью YOLOP. На нижнем красными точками отмечены точки, зарегистрированные лидаром и классифицированные как точки препятствия. Полученные алгоритмом детекции показаны в виде чёрных прямоугольников.

Рисунок 2.9 — Демонстрация работы алгоритма 1 на данных, полученных с БТС.

Input: I – изображение с RGB-камеры, P – облако точек.

Output: b_i – ориентированные ограничивающие рамки на виде сверху окружающих БТС объектов.

- 1 Произвести поиск двумерных детекций объектов на I : $\{d_1, \dots, d_n\}$;
- 2 Выделить в облаке точек P точки, не принадлежащие поверхности земли: P_o ;
- 3 Кластеризовать полученное облако точек P_o с расстоянием Евклида:

$$P_o = \bigcup_{i=1}^l C_i; C_i \cap C_j = \emptyset, i \neq j;$$
- 4 Спроецировать все кластеры на изображение I : $C_i \rightarrow \hat{C}_i$. Все точки, проецируемые за границы I , а также находящиеся позади оптического центра камеры отбрасываются. Пустые после проекции кластеры – удаляются;
- 5 Для каждого спроецированного кластера найти ограничивающую рамку минимального размера: d_{C_i} ;
- 6 Используя Венгерский алгоритм найти наилучшее сопоставление найденных детекций d_i с построенными ограничивающими рамками d_{C_i} , максимизирующее суммарный коэффициент Жаккара. Не сопоставленные кластеры – удаляются.;
- 7 В оставшихся кластерах удалить точки, не попадающие при проекции в сопоставленную детекцию d_i : $C_i \rightarrow C_i^-$.;
- 8 **foreach** C_i^- **do**
 - 9 | Используя RANSAC(RANdom SAMple Consensus) [74] найти прямую l_i , хорошо описывающую большую часть точек «очищенного» кластера C_i^- .;
 - 10 | Аналогично L-Shape алгоритму построить ограничивающую рамку b_i , со сторонами параллельными l_i и ортогональной ей l'_i .;
- 11 **end**
- 12 **return** $\{b_i\}$

Алгоритм 1: Алгоритм определения положения объектов на основе комплексов данных с камеры и лидаров.

2.3.1 Разделение облака точек на точки земли и препятствий

Отметим нетривиальность шага 2 алгоритма 1. Зачастую такое разделение производится поиском с помощью RANSAC плоскости, хорошо описывающей часть точек в облаке. Однако такой подход возвращает нас к предположению о совпадении поверхности дороги с некоторой плоскостью, от которого мы нацелены уйти. Поэтому в рамках данного алгоритма используется подход, не опирающийся на поиск плоскостей. В рамках данного подхода мы сперва производим поиск точек, для которых мы уверены, что они принадлежат земле. Далее мы решаем задачу регрессии: мы хотим найти зависимость высоты земли z_l от координаты в плоскости движения БТС (x_l, y_l) . В качестве обучающей выборки используются выбранные нами в начале точки. В качестве тестовых точек берутся координаты (x_l, y_l) входного облака точек. Мы решаем данную задачу регрессии на основе гауссовских процессов (Gaussian Process Regression, GPR) [79].

Регрессия на основе гауссовских процессов представляет собой байесовский непараметрический подход к аппроксимации функции, задающей зависимость значения целевой переменной от вектора признаков. В рассматриваемой задаче целью является аппроксимация функции $f : (x_l, y_l) \rightarrow z_l$, где (x_l, y_l) – координаты точки в плоскости движения БТС, а z_l – соответствующая высота земли.

Гауссовский процесс задаётся как распределение на множестве функций и полностью определяется средним значением $m(\mathbf{x})$ и ковариационной функцией (ядром) $k(\mathbf{x}, \mathbf{x}')$, где $\mathbf{x} = (x_l, y_l)$. В нашем сценарии мы использовали ядро Матерна ($\nu = 3/2$) [99], которое лучше справляется с аппроксимацией в точках разрыва функции (рисунок 2.10). Такие разрывы часто присутствуют в реальных сценах вокруг БТС, например, в местах соединения проезжей части и тротуаров.

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(1 + \frac{\sqrt{3}d}{l} \right) \exp \left(-\frac{\sqrt{3}d}{l} \right), \quad (2.3)$$

где $d = \|\mathbf{x} - \mathbf{x}'\|$ – евклидово расстояние между точками, σ_f^2 – дисперсия функции, а l – гиперпараметр масштаба длины.

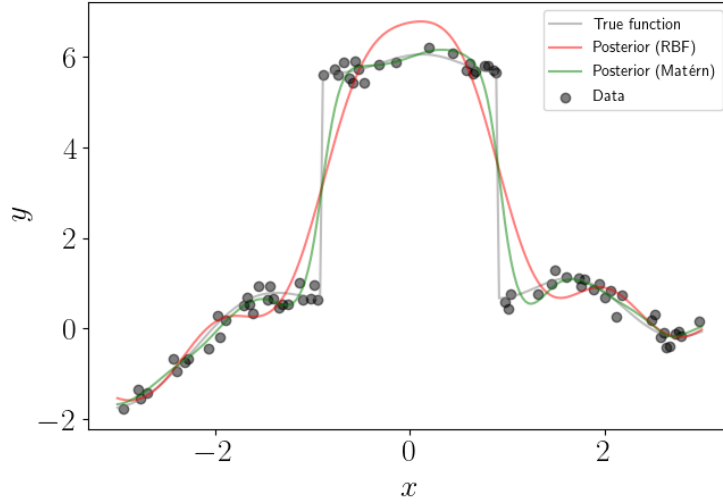


Рисунок 2.10 — Сравнение регрессии на основе гауссовских процессов с использованием в качестве ядра радиальной базисной функции и ядра Матерна [100].

Для обучающей выборки размером N_t с входами $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^{N_t}$ и соответствующими наблюдениями высоты $\mathbf{z} = \{z_i\}_{i=1}^{N_t}$ задача регрессии сводится к вычислению апостериорного распределения значений z_* для тестовых точек $\mathbf{X}_* = \{\mathbf{x}_*\}$.

Обозначим ковариационные матрицы:

$$K = [k(\mathbf{x}_i, \mathbf{x}_j)]_{i,j=1}^{N_t}, \quad K_* = [k(\mathbf{x}_*, \mathbf{x}_i)]_{i=1}^{N_t}, \quad K_{**} = [k(\mathbf{x}_*, \mathbf{x}_*)]_{i,j=1}^N, \quad (2.4)$$

тогда условное распределение предсказаний $p(z_* | \mathbf{X}, \mathbf{z}, \mathbf{x}_*)$ является гауссовским с параметрами:

$$\hat{z} = \mathbb{E}[z_*] = K_* K^{-1} \mathbf{z}, \quad (2.5)$$

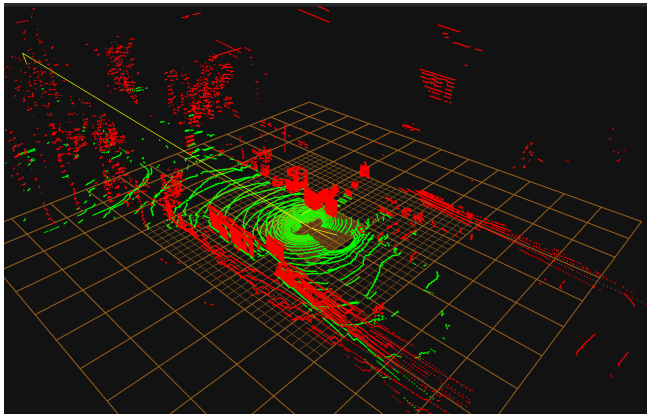
$$\text{Var}[z_*] = K_{**} - K_* K^{-1} K_*^\top.$$

Таким образом, предсказанное значение высоты земли для любой точки (x_l, y_l) получается как взвешенная сумма наблюдений обучающей выборки с учётом ковариации между точками.

Заметим, что регрессия на основе гауссовских процессов – вычислительно дорогая операция: в стандартных реализациях она занимает $O(N_t^3)$, где N_t – количество точек в обучающей выборке, т.к. включает разложение Холецкого. В реальных данных присутствуют десятки тысяч точек, принадлежащих земле.

Чтобы алгоритм имел возможность работать в режиме реального времени мы вычисляем усреднённые по областям значения координат точек, которые мы считаем принадлежащими земле и берём их подвыборку в размере нескольких сотен точек.

Наконец, для каждой точки входного облака мы сравниваем её измеренную координату z_l и предсказанную $\hat{z}_l$: если z_l больше $\hat{z}_l$ на некоторый порог z_{thr} , мы считаем данную точку не принадлежащей земле. Отметим, что ввиду принципа формирования лидарных данных плотность точек уменьшается при отдалении от машины, ввиду чего растёт и дисперсия предсказываемого значения $\hat{z}_l$. Чтобы учесть этот факт, мы делаем величину z_{thr} зависимой от расстояния до БТС: $z_{thr} = f(\|(x_l, y_l)\|_2)$. В Листинге 2 представлена реализация описанного алгоритма. На рисунке 2.11 представлен результат работы алгоритма.



а) Результат работы алгоритма. б) Изображение сцены с камеры БТС.
Рисунок 2.11 — Пример работы алгоритма разделения облака точек земли и препятствий.

Отметим, что данный алгоритм может быть реализован с помощью параллельных вычислений, в частности используя программно-аппаратную архитектуру CUDA, что позволяет снизить время выполнения данного алгоритма.

Input: P – облако точек.

Output: P_g – облако точек, принадлежащих земле, P_o – облако точек, не принадлежащих земле

```

1 Копируем облако  $P$  в  $P_c$ ;
2  $P_c$  разделяется на воксели  $V_i$  – области пространства, получаемые при
   накладывании трёхмерной сетки фиксированного разрешения;
3  $P_c$  разделяется иным образом сеткой в полярных координатах на сектора  $S_i$ ;
4 foreach  $V_i$  do
5     Для точек в вокселе вычисляется среднее  $m$  и среднеквадратичное отклонение
        $\sigma$  координаты  $z_l$  в локальной системе координат БТС  $O_l$ ;
6     Из  $P_c$  удаляются точки с  $|z_l - m| > \alpha\sigma$ , где  $\alpha$  – фиксированный параметр
       алгоритма;
7 end
8 foreach  $S_i$  do
9     Находится минимальное значение  $z_{min}$   $z_l$  точек в  $S_i$ ;
10    Из  $P_c$  удаляются точки, у которых  $z_l > z_{min} + T$ , где  $T$  – фиксированный
       параметр алгоритма;
11 end
12 Из вокселей  $V_i$  случайным образом выбираются  $N_{train}$  вокселей, где  $N_{train}$  –
    параметр алгоритма;
13 Для выбранных вокселей вычисляется центр масс  $(mx_i, my_i, mz_i)$  точек внутри
    вокселя. Полученные точки образуют множество  $P_{train}$ ;
14 Применяется регрессия на основе гауссовских процессов для оценки высоты  $\hat{z}_l$ , в
    точках  $(x_i, y_i)$ ;  $\forall p_i = (x_i, y_i, z_i) \in P$ , используя в качестве обучающей выборки
     $((x_i, y_i), z_i)$ ;  $\forall p_i = (x_i, y_i, z_i) \in P_{train}$ ;
15  $P_g = \{\}$ ;
16 foreach  $p_i = (x_i, y_i, z_i) \in P$  do
17      $S(p_i)$  – сектор, в котором находится точка  $p_i$ ;
18     if  $z_i < \hat{z}_i + z_{thr}(S(p_i))$  then
19         //  $z_{thr}(S(p_i))$  – массив порогов по высоте для каждого сектора
20          $P_g+ = p_i$ ;
21     end
22 end
23  $P_o = P \setminus P_g$ ;
24 return  $P_g, P_o$ ;

```

Алгоритм 2: Алгоритм фильтрации точек земли.

2.4 Исследование эффективности алгоритма проекции визуальных детекций на основе комплексирования визуальных и лидарных данных

Для оценки предложенного алгоритма [1](#) было проведено его сравнение с подходами, не использующими комплексирование: с L-Shape алгоритмом, опирающимся только на визуальные данные и описанным в разделе [2.1](#), и с четырьмя алгоритмами вписывания в ориентированную рамку, разработанными для лидарных облаков точек.

2.4.1 Описание эксперимента

Эксперимент выполнялся на том же наборе данных, что и сравнение в разделе [2.2](#). Как и прежде, эталонными положениями объектов служили проекции аннотированных параллелепипедов на вид сверху, а для сравнения алгоритмов измерялся коэффициент Жаккара для сопоставленных вычисленных и эталонных повернутых ограничивающих рамок на виде сверху.

Отметим существенное отличие нового эксперимента от эксперимента в разделе [2.2](#). В последнем алгоритмы Rotating PCA, Diagonal PCA, Longest Diagonal и Rotated Triangle применялись к проекциям нижнего контура объектов, найденных на изображении, а не к выпуклым оболочкам лидарных кластеров, как это было в оригинальной работе Ванга [\[86\]](#). Данное ограничение использовалось, поскольку в соответствующем разделе исследовались алгоритмы, использующие исключительно визуальные данные. В новом же алгоритме из-за появления комплексирования с лидарными данными появляются лидарные кластеры (шаг 6 алгоритма [1](#)), для работы с которыми и разрабатывались оригинальные алгоритмы. Поэтому в данном эксперименте перечисленные алгоритмы применялись к выпуклой оболочке получаемых лидарных кластеров, что отразится на полученных для них значениях коэффициента Жаккара в сравнении с результатами раздела [2.2](#).

Как и ранее для сравниваемых алгоритмов было проведено исследование зависимости среднего коэффициента Жаккара от максимальной дистанции до детектируемых объектов.

Сравниваемые алгоритмы реализованы на языке Python. Помимо этого, предложенный алгоритм реализован на языке C++ в виде узла ROS (англ. Robot Operating System) [101] и интегрирован в систему управления БТС, что позволило проверить его применимость в режиме реального времени.

2.4.2 Результаты экспериментов

В таблице 4 приведены статистики измеренных коэффициентов Жаккара. Предложенный алгоритм комплексирования показывает средний коэффициент Жаккара 0,439 против 0,337 у L-Shape алгоритма, использующего только визуальные данные, что соответствует росту на 30,3 %. Лучший из подходов, использующих исключительно лидарные данные, – Diagonal PCA – достигает 0,399, то есть предложенный алгоритм превосходит и его примерно на 10 %.

Характеристика	Rotating PCA	Diagonal PCA	Longest Diagonal	Rotated Triangle	Предложенный	L-Shape
Используемые данные	лидар	лидар	лидар	лидар	лидар и камера	камера
Среднее	0,396	0,399	0,335	0,339	0,439	0,337
Среднеквадратичное отклонение	0,222	0,222	0,188	0,215	0,256	0,141
Минимум	0,008	0,007	0,004	0,009	0,004	0,078
25 %	0,212	0,209	0,187	0,175	0,219	0,230
50 %	0,400	0,399	0,331	0,330	0,423	0,328
75 %	0,562	0,566	0,457	0,439	0,652	0,451
Максимум	0,872	0,871	0,734	0,915	0,925	0,713

Таблица 4 — Статистики полученных коэффициентов Жаккара для предложенного алгоритма и алгоритмов, не использующих комплексирование. Наилучшие значения выделены полужирным.

Вместе с тем по минимальному значению и по нижнему квартилю предложенный алгоритм уступает L-Shape алгоритму: 0,004 против 0,078 и 0,219 против 0,230 соответственно. Причина в том, что качество комплексирования

зависит от количества лидарных точек, попавших на объект. Для удалённых либо существенно заслонённых объектов кластер вырождается в несколько точек, и построенная по нему рамка может грубо расходиться с эталоном. В то же время L-Shape алгоритм строит рамку по проекции нижней границы объекта на плоскость движения с фиксированным количеством точек, что даёт устойчивую, хотя и невысокую в среднем, оценку.

Зависимость среднего коэффициента Жаккара от ограничения дальности до детектируемых препятствий представлена на рис. 2.12. Предложенный алгоритм превосходит остальные во всём диапазоне рассмотренных дальностей. Кроме того, на графике видно, что предельная дальность, на которой алгоритмы, использующие лидарные данные, сохраняют работоспособность, превышает соответствующую дальность для подхода, опирающегося только на визуальные данные. 62,36 метров у первых, против 28.24 метров у вторых: увеличение предельное дистанции на $\approx 120\%$.

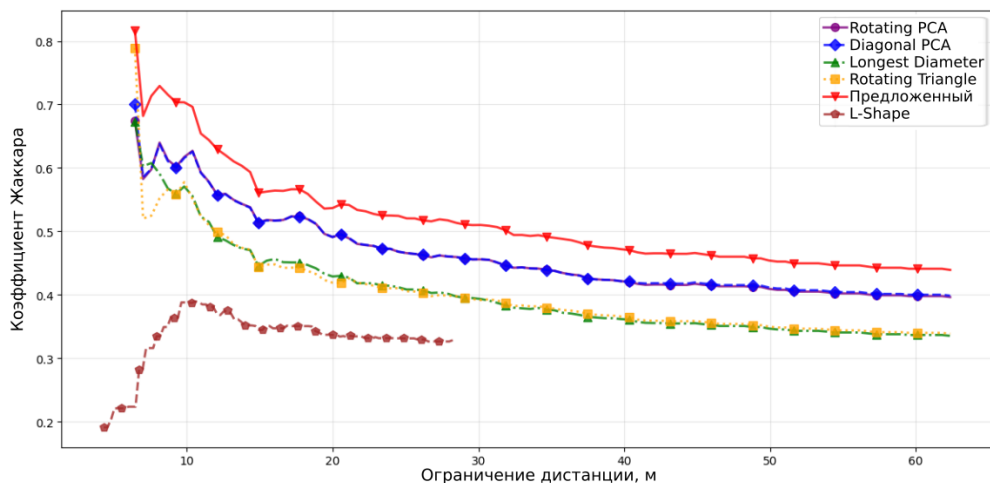


Рисунок 2.12 — График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий для алгоритма комплексирования и алгоритмов, не использующих комплексирование.

2.4.3 Вычислительная эффективность

Измерение времени работы производилось для реализации предложенного алгоритма на языке C++ на вычислительной платформе с процессором AMD Ryzen 5 5600X, ОЗУ объёмом 32 ГБ и ГПУ Nvidia GeForce RTX 3070. Среднее

время обработки одного кадра составило 66,7 мс при типичной частоте регистрации данных сенсорами БТС 10 Гц, то есть при допустимом времени обработки в 100 мс. Объём занимаемой памяти составил 178,8 МБ ОЗУ и 156 МБ памяти ГПУ. Полученные значения подтверждают применимость алгоритма на бортовом вычислителе БТС в режиме реального времени.

Время отдельных этапов алгоритма, измеренное с помощью библиотеки инструментального профилирования Trasy [102], приведено на рис. 2.13. Основную долю времени занимают шаги 4 и 5 алгоритма 1, в которых в текущей реализации дважды выполняется проецирование точек кластеров на плоскость изображения с коррекцией дисторсии. Эта операция выполняется на ЦПУ, хотя, как отмечено в разделе 2.3, хорошо поддаётся распараллеливанию на ГПУ, что оставляет запас для дальнейшей оптимизации.

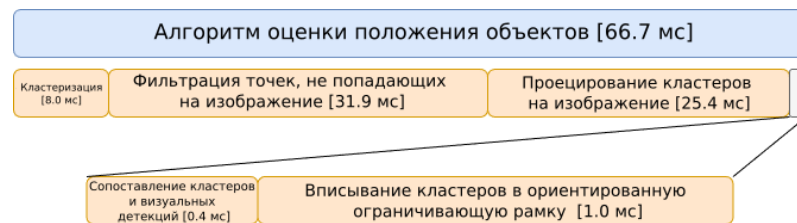
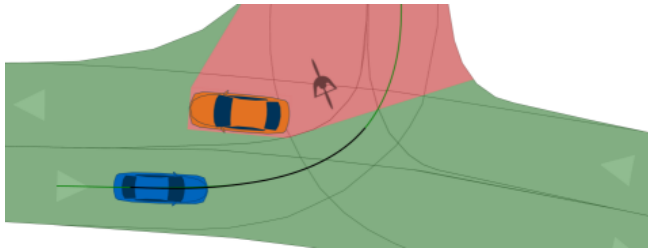


Рисунок 2.13 — Среднее время выполнения этапов алгоритма 1

2.5 Алгоритм проекции сегментации проезжей части на изображении на плоскость движения БТС

Помимо определения положения препятствий в окружающей БТС сцене важную роль в осуществлении безопасного решения задач планирования пути играет включение в модель описания областей, свободных для перемещения БТС. Это позволяет алгоритмам планирования пути отличать области, для которых достоверно известно отсутствие в них каких-либо объектов, от областей, для которых отсутствует информация, позволяющая судить об их проходимости ввиду окклюзии либо недостоверности регистрируемых данных. На рисунке 2.14 приведён пример, демонстрирующий важность этого различия. Мы можем обучить нейросетевую модель YOLOP или другую модель, осуществляющую семантическую сегментацию двумерных изображений, сегментировать проезжую часть на изображении. Однако аналогично задаче проецирования

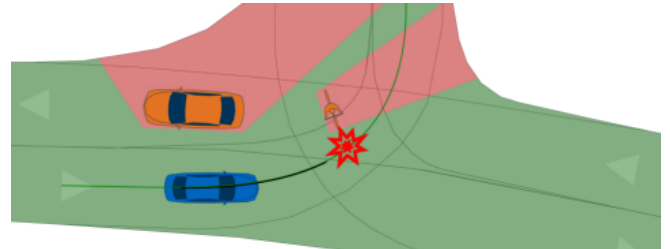
препятствий в трёхмерную систему координат БТС O_l , описанной в разделе 2.1, мы столкнёмся с необходимостью спроецировать полученную сегментацию на вид сверху. В данном разделе предложен алгоритм, осуществляющий данное проецирование на основе комплексирования данных с камеры и лидаров, установленных на БТС.



а) БТС (синий) не видит

велосипедиста позади автомобиля.

Зелёным выделена территория в поле видимости датчиков БТС, красным – области, о которых нет информации ввиду их окклюзии



б) При продолжении движения

велосипедист детектируется

датчиками БТС, но из-за близости

объекта, у систем управления остаётся

мало времени для реагирования.

Рисунок 2.14 — Демонстрация влияния окклюзии на безопасность движения БТС [103].

В отличие от типичных препятствий в окружающей БТС сцене: людей, автомобилей; размеры и положение которых с высокой точностью описываются ориентированными ограничивающими рамками, проезжая часть после проекции имеет сложную геометрию (рисунок 2.15). Из-за этого векторное представление проезжей части на виде сверху может быть неудобным для построения. Вместо него мы воспользуемся представлением в виде сетки фиксированного разрешения, где каждому элементу будет сопоставлен класс «свободно» – в случае если в соответствующей области БТС может безопасно проезжать, или «неизвестно» – в случае если информации об области недостаточно (а далее и дополнительный класс «занято» для областей, где обнаружен объект).

Прежде чем приступить к описанию алгоритма опишем интуицию, которая будет лежать в его основе без использования информации с лидаров. Воспользуемся предположением в наивном подходе проецирования препятствий из раздела 2.1 о совпадении поверхности проезжей части с плоскостью $z_l = 0$. В такой модели оценка проходимости элемента сводится к проекции центров сетки



Рисунок 2.15 — Пример семантической сегментации проезжей части моделью YOLOP на изображении с камеры БТС.

на плоскость изображения (как и прежде, мы считаем внутреннюю и внешнюю калибровки камеры известными): если при проецировании центр элемента попал на пиксель, классифицированный как проезжая часть, соответствующий элемент сетки классифицируется как «свободный», в противном случае – как «неизвестный» (рисунок 2.17 а). Отметим, что в последнем случае нельзя утверждать, что область не свободна для проезда, т.к. она может быть заслонена другим объектом – см. рисунок 2.16). Листинг 3 приводит описание данного алгоритма.

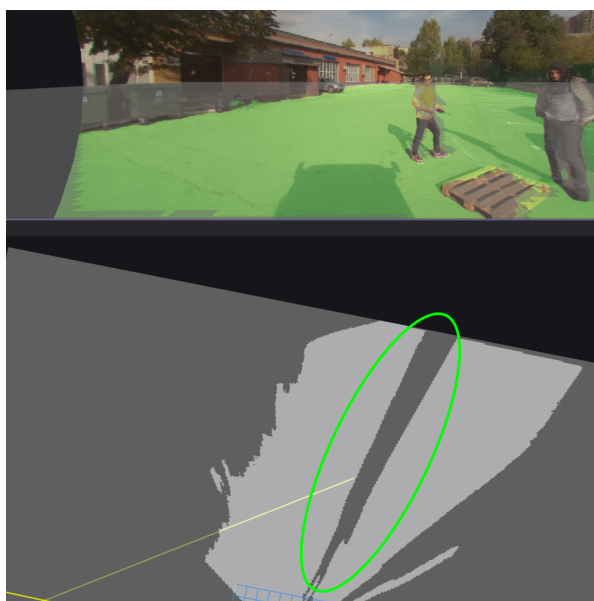


Рисунок 2.16 — Пример заслонённой области свободной для проезда.

Input: S_I – семантическая сегментация проезжей части на изображении I ; res – разрешение сетки [элементов на метр], на которую производится проецирование; l – ширина и высота сетки [м]

Output: S_{BEV} – проекция сегментации проезжей части на вид сверху

```

1   $s \leftarrow \frac{1}{\text{res}}$  ;
2   $L \leftarrow l \cdot \text{res}$  ;
3   $S_{BEV} \leftarrow$  матрица размера  $L \times L$ ;
4  for  $i \leftarrow 0$  to  $L$  do
5      for  $j \leftarrow 0$  to  $L$  do
6           $x \leftarrow -\frac{l}{2} + js$ ;
7           $y \leftarrow -\frac{l}{2} + is$ ;
8           $(u, v) \leftarrow$  проекция  $(x, y, 0)$  на изображение (с учётом дисторсии);
9          if  $(u, v)$  попадает на изображение AND  $S_I[u][v] == \text{«свободно»}$ 
10             then
11                  $S_{BEV}[i][j] \leftarrow \text{«свободно»}$ ;
12             else
13                  $S_{BEV}[i][j] \leftarrow \text{«неизвестно»}$ ;
14             end
15     end
16 return  $S_{BEV}$ ;

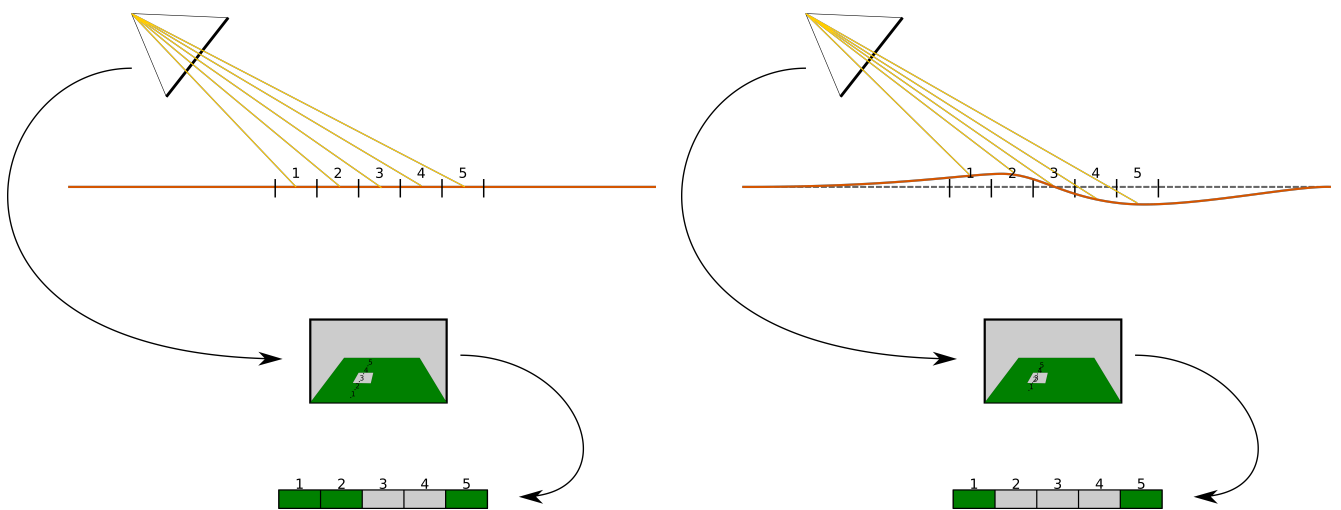
```

Алгоритм 3: Алгоритм проецирования проезжей части на вид сверху (наивный).

Однако, мы уже показали низкое качество моделей, использующих предположение о совпадении поверхности проезжей части с плоскостью. Ослабим это предположение и обобщим алгоритм на этот случай. Будем считать, что поверхность описывается некоторой известной функцией $z_l = z(x_l, y_l)$. Тогда в ранее описанный алгоритм достаточно добавить вычисление этой координаты перед проецированием на изображение (листинг 4). На рисунке 2.17 б представлена иллюстрация обобщённого алгоритма. Очевидно, для корректной работы алгоритма необходимо, чтобы более близкие к БТС участки проезжей части не перекрывали обзор камеры для более удалённых участков. При достаточной гладкости проезжей части, разумном разрешении итоговой сетки на виде сверху и достаточной высоте расположения камеры, это требование будет выполняться.

- 1 Аналогично шагам 1–7 из Листинга 3;
- 2 $(u; v) \leftarrow$ проекция $(x, y, z(x, y))$ на изображение (с учётом дисторсии);
- 3 Аналогично шагам 9–16 из Листинга 3;

Алгоритм 4: Обобщение алгоритма проецирования проезжей части на вид сверху.



а) Проезжая часть лежит в известной плоскости $z_l = 0$.

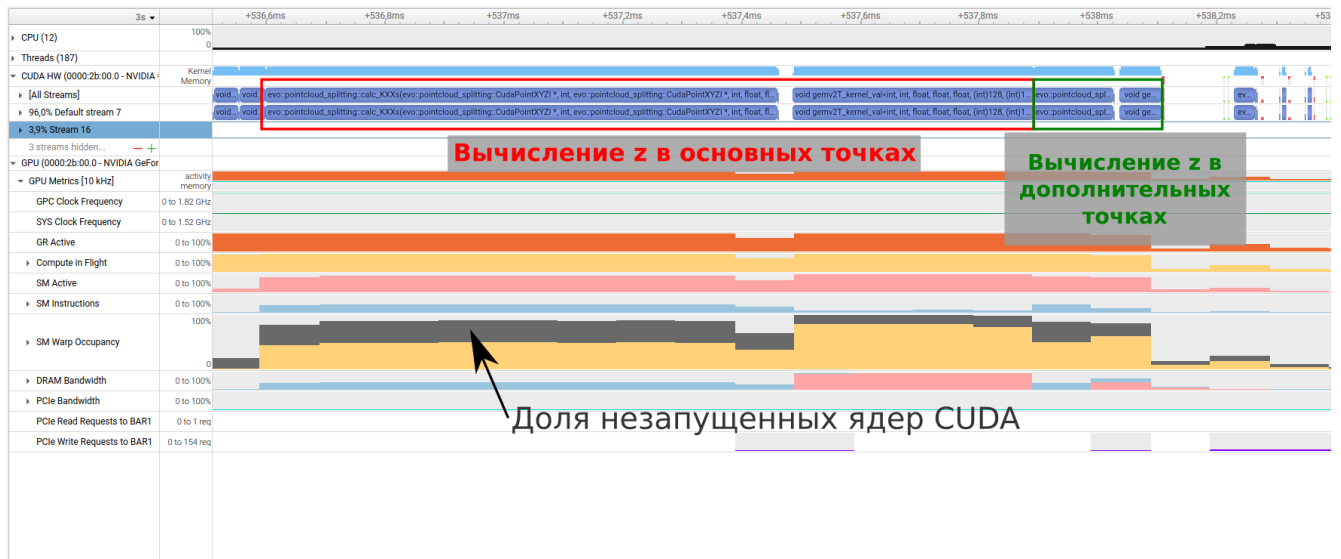
б) Поверхность проезжей части удовлетворяет уравнению $z = z(x, y)$.

Рисунок 2.17 — Иллюстрация алгоритма проецирования сегментации проезжей части на вид сверху. Для облегчения восприятия проецируемая сетка представлена одним рядом.

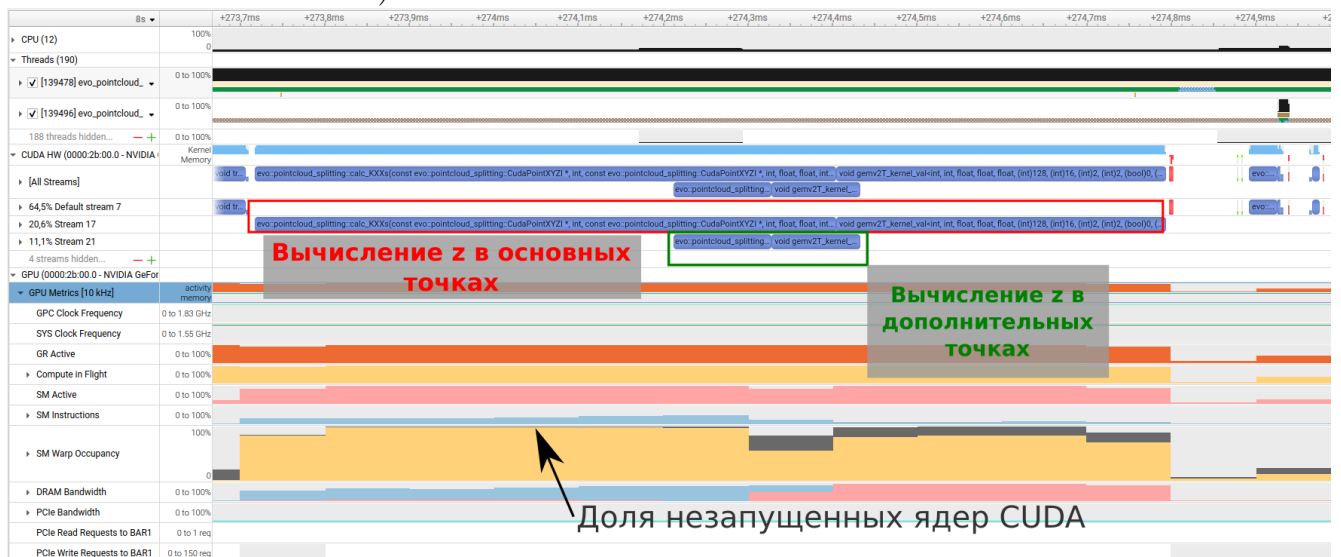
2.5.1 Вычисление функции $z(x,y)$

Опишем алгоритм оценки значений $z(x,y)$ в центрах сетки, на которую производится проекция сегментации. Для этого аналогично разделу 2.3 мы воспользуемся комплексированием данных, зарегистрированных лидарами. Как и прежде мы будем считать, что облака точек с лидаров прошли предварительную обработку и представлены единым облаком точек с компенсированным движением БТС во время регистрации облаков.

Чтобы оценить $z(x,y)$ в центрах сетки, в описанном ранее алгоритме 2 проводится отдельная итерация регрессии (шаг 14). В качестве обучающих точек используются те же точки, что и раньше, а в качестве тестовых – (x,y) центров сетки. Недостатком такого подхода станет увеличение времени работы алгоритма 2, что замедлит работу алгоритмов, использующих его результаты, в частности алгоритма 1. Однако при наличии вычислительных ресурсов на ГПУ вычисление регрессии можно распараллелить с использованием механизма мультипоточности CUDA Streams [104]. В этом случае в регрессии можно переиспользовать матрицу K из уравнения (2.4). Затем вычислить ковариационные матрицы $\widetilde{K}_*$, $\widetilde{K}_{**}$ для новых тестовых точек, и аналогично (2.5) вычислить аппроксимацию z в тестовых точках. На рисунке 2.18 представлены результаты профилирования ресурсов на ГПУ с помощью Nsight Systems [105] изменившейся части программы, реализующей алгоритм 2 при последовательном и параллельном решении задач регрессии. Программа реализована на языках C++ и CUDA C++. Профилирование происходило на ПК с процессором AMD Ryzen 5 5600X с 6 ядрами и тактовой частотой 4,6 ГГц, ОЗУ объёмом 32 ГБ, а также ГПУ Nvidia GeForce RTX 3070 с графической памятью 8 ГБ и тактовой частотой 1,5 ГГц. Количество основных точек составило порядка 100 000, количество дополнительных точек – 10 000. На рисунке 2.19 представлен график сравнения распределения времени работы алгоритма разделения при последовательном и параллельном вычислениях.



а) Без использования CUDA Streams



б) С исправленной конфигурацией запуска ядер CUDA и использованием CUDA Streams

Красной рамкой выделены вычисления аппроксимации прежних тестовых точек, в зелёной – дополнительных точек. Во вкладке «SM Warp Occupancy» жёлтым показана доля вычисляемых ядер относительно максимальной теоретической пропускной способности архитектуры в данный момент времени, серым – доля ядер, которым не хватило ресурсов для вычисления и в результате простаивающих в ожидании их появления.

Рисунок 2.18 — Профилирование в Nsight Systems изменившейся части программы, реализующей алгоритм 2, при добавлении дополнительных точек для аппроксимации.

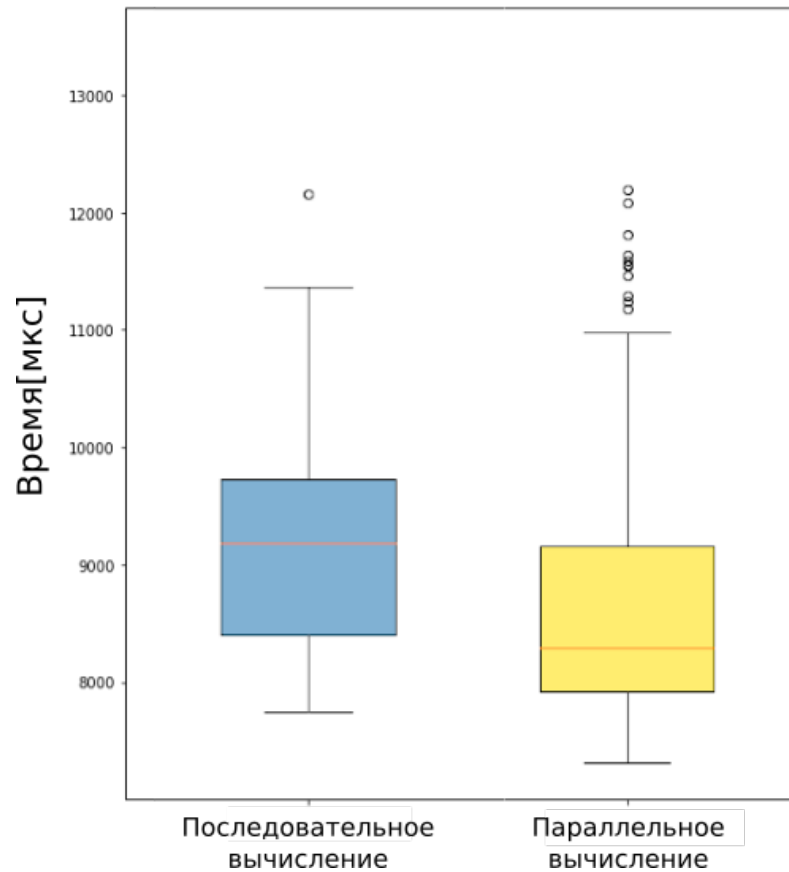


Рисунок 2.19 — График полного времени работы алгоритма 2 с дополнительными тестовыми точками при последовательном и параллельном вычислении.

2.5.2 Комплексирование полученной проекции с базовой картой проходимости пространства

Описанный алгоритм строит фрагмент карты проходимости пространства, опираясь на семантическую сегментацию изображения. Он не является самостоятельным способом построения модели окружения: сведения о препятствиях, не относящихся к проезжей части, в нём отсутствуют. Поэтому полученная проекция должна быть объединена с картой, построенной по лидарным данным.

В качестве базового прием алгоритм построения карты проходимости пространства по объединённому облаку точек, восходящий к работам Х. Моравека и А. Элфеса [33; 35] описанным в разделе 1.3. В нём из облака выделяются точки, лежащие выше поверхности земли и не превышающие габаритов БТС. Элементам сетки, в которые проецируются эти точки, присваивается отрицательный вклад l_-^b , а элементам, лежащим на отрезке между положением лидара

и зарегистрированной точкой, – положительный вклад l_+^b . Области, расположенные за наблюдаемыми препятствиями, считаются заслонёнными и сохраняют состояние «неизвестно». Учёт вкладов производится байесовским обновлением по формуле (1.1). Наконец, все элементы карты квантуются в три значения в зависимости от их положения относительно интервала $(-l_t, l_t)$, где l_t – некоторый фиксированный порог квантования.

Отметим, что базовый алгоритм предполагает статичность наблюдаемой сцены: вклады, полученные в разные моменты времени, накапливаются в элементе карты без ограничений. Для сцен, в которых работает БТС, это предположение не выполняется – в них присутствуют люди, автомобили и иные подвижные объекты, а также перемещается сам БТС. Чтобы это учесть, к карте применяется механизм затухания, возвращающий давно не наблюдавшиеся элементы в состояние «неизвестно»; подробно этот механизм рассматривается в главе 4.

Объединение проекции сегментации с картой базового алгоритма выполняется следующим образом. Элементам сетки, определённым как «свободно», присваивается значение l_+^c . Тем элементам, которые были определены как «занято» или «неизвестно» и которые попадают на изображение с камеры – l_-^c . Остальные элементы заполняются 0. Полученная таким образом карта проходимости пространства затем объединяется с картой базового алгоритма по формуле (1.1). Заметим, что все элементы в поле зрения камеры получают ненулевые значения, даже если алгоритм 4 определил их как «неизвестно». Это сделано намеренно, чтобы перевести в состояние «неизвестно» те области карты, которые базовый алгоритм выделил как «свободно», а предложенный алгоритм не смог это подтвердить. Для достижения этого необходимо выбрать параметры l_-^c и l_+^b таким образом, чтобы выполнялось неравенство:

$$l_-^c + l_+^b \in (-l_t, l_t).$$

В терминах классификации, приведённой в разделе 1.4, предложенный алгоритм задействует два типа комплексирования на разных своих этапах. На этапе проецирования применяется *кооперативное* комплексирование: лидар предоставляет геометрию поверхности движения, камера – семантическую разметку, и ни один из этих датчиков по отдельности не способен разместить семантическую метку в трёхмерном пространстве. На этапе объединения полученной

проекции с картой базового алгоритма применяется *конкурирующее* комплексирование: оба источника оценивают одно и то же свойство – проходимость элемента сетки, – и их расхождение переводит элемент в состояние неопределённости.

2.6 Исследование эффективности алгоритма проекции сегментации проезжей части на основе комплексирования визуальных и лидарных данных

Для оценки предложенного алгоритма было проведено сравнение качества создаваемой им карты проходимости пространства с картой базового лидарного алгоритма, не использующего комплексирование с визуальными данными. Помимо этого, чтобы раздельно оценить вклад двух составляющих предложенного алгоритма – учёта рельефа поверхности при проецировании и конкурирующего комплексирования при объединении карт, – было выполнено абляционное исследование.

2.6.1 Набор данных

Для проведения эксперимента использовались 5 записей показаний датчиков БТС, каждая из которых содержит объединённое облако точек с двух лидаров и изображения с трёх бортовых камер. Каждой записи соответствует аннотированный фрагмент длительностью от 4,9 до 9,9 секунд; фрагменты подобраны так, чтобы охватывать различные дорожные ситуации, характерные для эксплуатации БТС (таблица 5).

Каждый кадр аннотированных данных включает объединённое облако точек, в котором каждой точке присвоена метка принадлежности препятствию, множество спроецированных на плоскость движения БТС детекций людей и автомобилей, а также положение БТС в глобальной системе координат. По этим данным для каждого кадра формировалась эталонная локальная карта проходимости пространства с числом элементов $N = 600$ и разрешением

№	Описание сценария	Длительность, с	Число кадров
1	Движение по прямой с разворачивающимся впереди автомобилем	9,9	91
2	Начало движения при находящейся впереди группе пешеходов	9,9	90
3	Поворот и движение вдоль парковки автомобилей	9,9	93
4	Подъезд к перекрёстку	4,9	48
5	Объезд препятствий	9,9	87

Таблица 5 — Сценарии, представленные в наборе тестовых данных.

$res = 10$ элементов на метр. Помимо аннотаций кадра, в эталонную карту переносился соответствующий участок размеченной вручную глобальной карты, содержащей области, доступные для проезда БТС, и области статических препятствий. Участкам, выходящим за пределы размеченной глобальной карты, присваивалось состояние «неизвестно». Полученные эталонные карты для всех пяти сценариев приведены в левом столбце рис. 2.20.

2.6.2 Сравнимые алгоритмы и метрики

Для используемых данных запись велась с трёх синхронизированных бортовых камер БТС, проецирование сегментации выполнялось для изображения каждой камеры, а полученные фрагменты карты складывались с учётом взаимного расположения камер, после чего результат объединялся с картой базового алгоритма. В качестве источника семантической сегментации использовалась нейросетевая модель YOLOP [94], дообученная на данных, собранных с реальных БТС. Оценка высоты поверхности земли в центрах элементов сетки производилась добавлением этих центров к тестовым точкам алгоритма 2, как описано в разделе 2.5.1.

Сравнивались четыре алгоритма:

- **Б** — базовый лидарный алгоритм. Комплексование с визуальными данными отсутствует.

- **П** – предложенный алгоритм. Проекция сегментации выполняется с учётом рельефа поверхности дороги, оценённого по лидарным данным; объединение карт производится конкурирующим комплексированием ($l_-^c \neq 0$).
- **A1** – вариант предложенного алгоритма без учёта рельефа. Проецирование выполняется на фиксированную плоскость $z_l = 0$.
- **A2** – вариант предложенного алгоритма, в котором конкурирующее комплексирование заменено дополняющим. Карта базового алгоритма не содержит свободных зон ($l_+^b = 0$), а проекция сегментации до сложения с ней не содержит зон препятствий ($l_-^c = 0$). Тем самым лидар отвечает только за препятствия, а камера – только за проезжую часть, и объединяемые карты дополняют, а не уточняют друг друга.

Алгоритмы A1 и A2 добавлены для абляционного исследования предложенного алгоритма, для оценки эффекта отдельных этапов работы алгоритма.

Значения отличающихся параметров, использованных для каждого из алгоритмов, приведены в левой части таблицы 7. Остальные параметры были одинаковы: $z_{max} = 3,0$ м, $d_{max} = 40$ м, $res^b = 10$, $res^c = 2,5$, $l_-^b = -4,701$, $l_t = 5$. Механизм затухания также применялся одинаково во всех вариантах.

Качество построенных карт оценивалось тремя метриками, введёнными в разделе 1.5: среднеквадратичной ошибкой относительно эталонной карты (1.3), усреднённым по классам коэффициентом Жаккара (1.2) и метрикой PFC-MSE [69]. При вычислении mJ в качестве классов элементов карты рассматривались три возможных состояния: «занято», «свободно» и «неизвестно». Как уже обсуждалось в главе 1, в качестве основной метрики для сравнения была выбрана PFC-MSE, поскольку она наиболее показательна для решения задачи планирования движения БТС.

2.6.3 Результаты экспериментов

Значения PFC-MSE по каждому из тестовых сценариев приведены в таблице 6. Предложенный алгоритм превосходит базовый лидарный во всех пяти сценариях; в среднем PFC-MSE снижается с 1019,3 до 924,4, то есть на 9,3 %.

	Алгоритм	Сценарий					Среднее
		1	2	3	4	5	
Б	Базовый лидарный алгоритм	1218,3	1353,0	1087,5	932,0	505,9	1019,3
П	Предложенный алгоритм	1107,5	1213,9	949,0	850,9	500,5	924,4

Таблица 6 — Значения метрики PFC-MSE (меньше – лучше) для предложенного и базового лидарного алгоритмов. Наилучшие значения выделены полужирным.

Улучшение наблюдается и по двум другим метрикам: среднеквадратичная ошибка снижается с 0,231 до 0,218, то есть на 5,6 %, а усреднённый по классам коэффициент Жаккара возрастает с 0,225 до 0,238. Таким образом, учёт визуальных данных с камер БТС при построении карты проходимости пространства повышает её качество по сравнению с картой, строящейся исключительно по лидарным данным, причём как по метрикам поэлементного сходства, так и по метрике пригодности карты для планирования маршрута.

Результаты абляционного исследования приведены в таблице 7.

	Параметры				Метрики		
	l_+^b	l_-^c	l_+^c	рельеф	СКО ↓	mJ ↑	PFC-MSE ↓
Б	0,490	—	—	—	0,231	0,225	1019,3
П	0,490	−0,080	1,099	да	0,218	0,238	924,4
A1	0,490	−0,080	1,099	нет	0,236	0,230	1059,4
A2	0	0	1,099	да	0,183	0,205	1130,8

Таблица 7 — Параметры сравниваемых алгоритмов Б, П, A1, A2 (обозначения введены выше) и усреднённые по пяти сценариям значения метрик. Стрелка ↓ означает, что меньшие значения метрики соответствуют лучшему качеству, ↑ — большие. Наилучшие значения по каждой метрике выделены полужирным.

Вклад учёта рельефа поверхности дороги. Алгоритм A1 отличается от предложенного тем, что положение поверхности движения задаётся плоскостью, а не оценивается по лидарным данным. Учёт рельефа даёт преимущество по всем трём метрикам. Более того, по среднеквадратичной ошибке и по основной метрике алгоритм A1 уступает даже базовому лидарному алгоритму, вовсе не использующему визуальные данные. Причина в том, что в процессе эксплуатации БТС внешняя калибровка камер изменяется, а дорожное полотно отклоняется от плоскости, что приводит к неверному сопоставлению точки

плоскости с данными семантической сегментации. Тем самым эксперимент подтверждает на реальных данных ограничение предположения о плоскости поверхности дороги, отмеченное в разделе 2.3.

Сравнение конкурирующего комплексирования с дополняющим. Алгоритм A2 отличается от предложенного тем, что комплекслируемые карты перестают содержать оценки одного и того же свойства: свободные зоны формируются исключительно по данным камер, а лидар отвечает только за препятствия. По основной метрике такая замена приводит к ухудшению результата: PFC-MSE составляет 1130,8 против 924,4 у предложенного алгоритма.

Отдельно отметим, что алгоритм A2 показал наилучшую среди всех вариантов среднеквадратичную ошибку (0,183), одновременно оказавшись худшим по mJ (0,205) и по PFC-MSE (1130,8). Это наблюдение показывает, что среднеквадратичная ошибка и PFC-MSE оценивают карту по-разному: близость карты к эталонной по поэлементному сходству не влечёт её пригодности для планирования маршрута, поскольку связность свободных областей при этом может отличаться существенно. Примеры построенных карт проходимости пространства всеми сравниваемыми алгоритмами приведены на рис. 2.20.

2.6.4 Вычислительная эффективность

Измерение времени работы производилось на персональном компьютере с процессором Intel Core i7-13650HX, ОЗУ объёмом 32 ГБ и ГПУ Nvidia GeForce RTX 4070 с графической памятью 8 ГБ. Среднее время одной итерации построения карты проходимости пространства предложенным алгоритмом составило 1,63 мс. Указанное время относится к проецированию результатов семантической сегментации и обновлению значений элементов карты и не включает время инференса модели семантической сегментации. Полученное значение подтверждает возможность построения карты в режиме реального времени на бортовом вычислителе БТС.

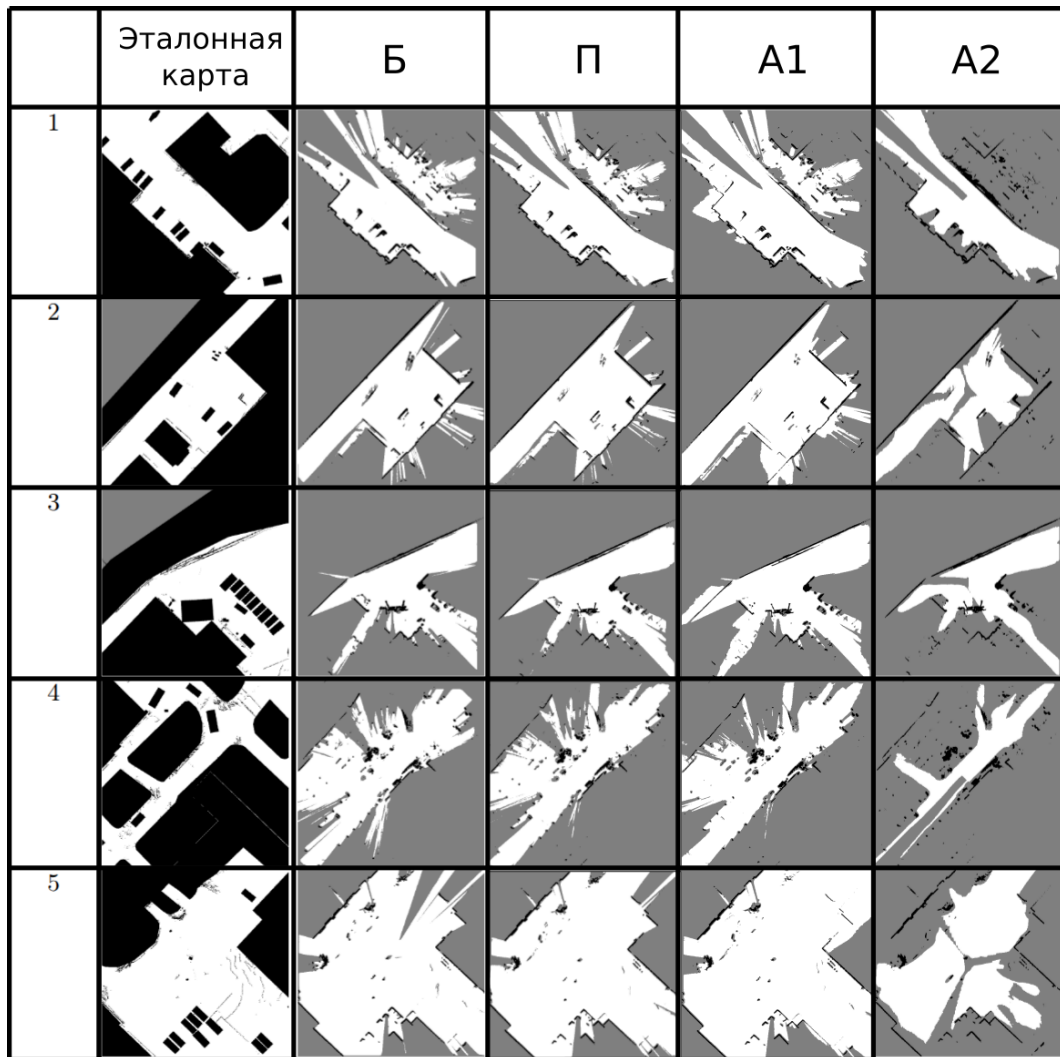


Рисунок 2.20 — Эталонные карты проходимости пространства и карты, построенные сравниваемыми алгоритмами на тестовых данных.

2.7 Заключение к главе 2

Итак, в данной главе получены следующие основные результаты:

1. Предложен алгоритм проекции визуальных детекций с изображения на плоскость движения БТС с использованием модели L-shape;
2. Проведено сравнение алгоритма L-Shape с альтернативными алгоритмами. Результаты экспериментов демонстрируют, что:

на реальных данных L-Shape алгоритм показал наивысший среди рассмотренных алгоритмов вписывания ограничивающей рамки средний коэффициент Жаккара (0,337);

алгоритм показывает наилучшее качество, если ограничить дальность проецируемых препятствий до 10 метров;

3. Предложен альтернативный алгоритм проекции визуальных детекций с изображения на плоскость движения БТС на основе комплексирования данных с камеры и лидаров, установленных на БТС, не использующий предположения о плоскости поверхности дороги;
4. Проведено сравнение предложенного алгоритма комплексирования с алгоритмами, не использующими комплексирование. Результаты экспериментов демонстрируют, что:

на реальных данных алгоритм комплексирования показал средний коэффициент Жаккара 0,439, что на 30,3 % превосходит алгоритм, использующий только визуальные данные, и примерно на 10 % – лучший из алгоритмов, использующих только лидарные данные;

предельная дальность обнаружения препятствий при использовании лидарных данных возрастает на 120 % по сравнению с подходом, опирающимся только на визуальные данные;

среднее время обработки одного кадра составляет 66,7 мс, что допускает работу алгоритма в режиме реального времени на бортовом вычислителе БТС;

5. Предложен алгоритм проекции семантической сегментации проезжей части с изображения на плоскость движения БТС на основе комплексирования данных с камеры и лидаров, установленных на БТС, а также способ его объединения с базовым алгоритмом построения карты проходимости пространства, использующий конкурирующее комплексирование;
6. Проведено сравнение полученной карты проходимости пространства с картой базового лидарного алгоритма. Результаты экспериментов демонстрируют, что:

учёт визуальных данных снижает среднеквадратичную ошибку относительно эталонной карты на 5,6 % и метрику PFC-MSE на 9,3 %, причём по последней предложенный алгоритм превосходит базовый во всех пяти тестовых сценариях;

согласно абляционному исследованию, замена лидарной оценки рельефа дороги плоскостным приближением ухудшает все три рассматриваемые метрики, а замена конкурирующего комплексирования дополняющим ухудшает основную метрику;

среднее время одной итерации построения карты составляет 1,63 мс.

Перечисленные основные результаты и другие авторские материалы главы опубликованы в работах [106—109]. Для разработанных алгоритмов было получено свидетельство о регистрации программы для ЭВМ: № 2025617212 Программное обеспечение «Система восприятия N1 V3».

Глава 3. Программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных

В предыдущей главе 2 были предложены алгоритмы извлечения информации о проходимости пространства вокруг БТС. Результаты их работы имеют разный формат представления, что может усложнять их учёт последующими алгоритмами планирования маршрута БТС. Кроме того, не определены правила, как разрешать противоречия о проходимости пространства в полученных результатах алгоритмов.

В данной главе описан программный комплекс построения карты проходимости пространства – модели, описанной в главе 1. Данный комплекс реализует сбор разнородных данных с датчиков или их обработчиков, преобразует данные в единый формат – фрагменты карты проходимости пространства и производит их комплексирование в единую карту проходимости пространства. Глава содержит подробное описание разработанного программного комплекса: структуру, функциональные возможности, формат входных и выходных данных.

3.1 Общие сведения

Полное наименование комплекса программ – «Программный комплекс построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства». Условное обозначение – OssurancуMapFusion. Программный комплекс реализован на языках C++ и CUDA C++. Средством разработки является редактор Microsoft Visual Studio Code.

3.2 Назначение и функциональные возможности

Программный комплекс Ossurancу Map Fusion предназначен для построения карты проходимости пространства – двумерной сетки фиксированного

разрешения, элементы которой описывают возможность проезда БТС в пространстве, ограниченном границами данного элемента. Построение производится на основе сырых показаний датчиков, а также признаков, извлечённых из них.

Программный комплекс предоставляет следующие функциональные возможности:

1. многопоточное обновление карты проходимости пространства на основе различных данных:
 - а) облака точек, принадлежащих препятствиям;
 - б) показаний сонара;
 - в) комплексирования двумерных детекций на RGB-изображении с камеры и облака точек с лидара, принадлежащих препятствиям;
 - г) комплексирования семантической сегментации проезжей части на RGB-изображении с камеры и облака точек с лидара, принадлежащих земле;
 - д) трёхмерных детекций препятствий;
2. расширение поддерживаемых источников данных о проходимости окружающего пространства путём самостоятельной генерации пользователем фрагментов карты проходимости пространства;
3. публикация генерируемой карты проходимости пространства в виде ROS-сообщений (Robot Operating System) [101];
4. опциональное удаление из карты проходимости пространства информации об областях, занимаемых динамическими препятствиями;
5. отключение и подключение отдельных источников информации, учитываемой в карте проходимости пространства, по ходу выполнения программы;

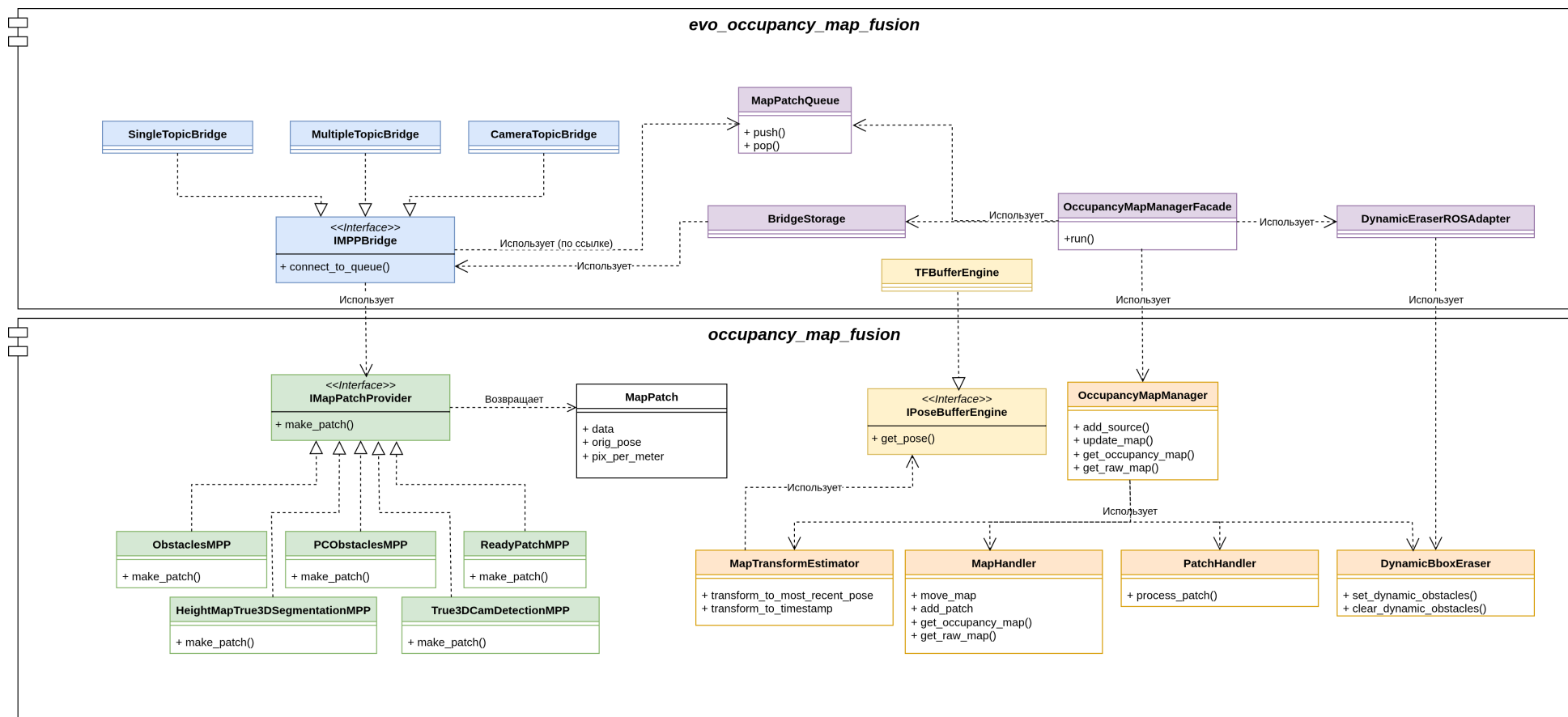
3.3 Структура программного комплекса

На рисунке 3.1 показана диаграмма основных компонентов описываемого программного комплекса. На рисунке 3.2 схематично изображена последовательность шагов при обработке данных приходящих от источников данных (т.е.

датчиков либо их предварительных обработчиков, извлекающих полезные признаки). Комплекс состоит из двух крупных компонентов:

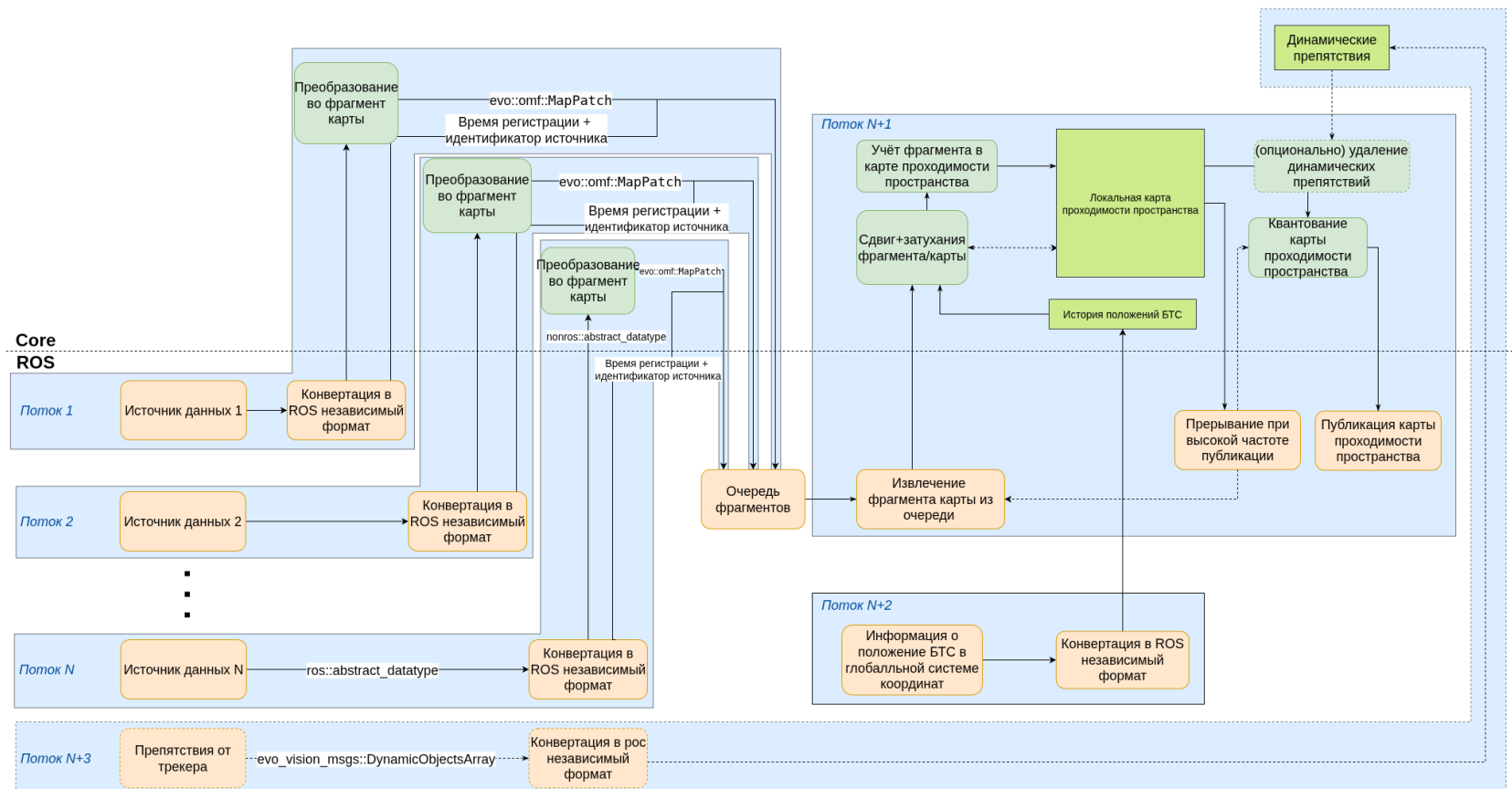
1. `оссирансу_map_fusion` – библиотека, реализующая логику работы с картой проходимости пространства: добавление дополнительных источников данных, актуализация положения карты, обработка и учёт входных данных от источников. Компонент реализован на языках C++ и CUDA C++
2. `evo_оссирансу_map_fusion` – программа, интегрирующая библиотеку `оссирансу_map_fusion` в систему ROS: производит соединение с остальными компонентами систем БТС через механизмы ROS-топиков и ROS-сервисов, преобразует входные данные в формат, соответствующий интерфейсам библиотеки, и преобразует генерируемую карту проходимости в ROS сообщение, передаваемое последующим компонентам ПО БТС, в частности компоненту планирования пути БТС. Компонент реализован на языке C++.

Далее в разделе представлено подробное описание структуры данных компонентов, их назначение и форматы входных и выходных данных.



Цветом выделены «идейно связанные» компоненты: базовый класс и его наследники, либо класс и классы хранимых в нём (путём композиции) объектов.

Рисунок 3.1 — Диаграмма компонентов программного комплекса Occupancy Map Fusion на языке UML(Unified Modeling Language).



Зелёным цветом выделены этапы обработки данных компонентами независимыми от ROS(Robot Operating System),
оранжевым – компонентами, зависящими от ROS.

Рисунок 3.2 — Диаграмма деятельности программного комплекса Occupancy Map Fusion.

3.3.1 Библиотека «occupancy_map_fusion»

Библиотека «occupancy_map_fusion» реализует функционал для построения карты проходимости пространства на основе обратной модели наблюдения. Учёт новых данных в карте происходит в два этапа (рисунок 3.2):

1. Построение фрагмента карты проходимости по данным от одного из источников данных.
2. Добавление построенного фрагмента на общую карту проходимости пространства с учётом:
 - а) положения системы координат фрагмента относительно системы координат карты,
 - б) изменения положения БТС за время между временем зарегистрированных данных и последнего актуального положения карты в глобальной системе координат,
 - в) времени прошедшего с момента регистрации данных.

Первый шаг реализуется через интерфейс «IMapPatchProvider» и наследующие его классы-реализации. Второй – классом «OccupancyMapManager». Опишем их подробнее.

Интерфейс «IMapPatchProvider»

«IMapPatchProvider» – шаблонный класс, параметризуемый типом входных данных $InputT$. Интерфейс задаёт единственный абстрактный метод `make_patch`, реализующий преобразование входных данных во фрагмент карты проходимости пространства.

Инициализация включает параметры, используемые всеми наследуемыми классами:

1. pix_per_meter – разрешение генерируемого фрагмента,
2. $p(TP) \in (0,1)$ – вероятность того, что реализация верно определит наличие препятствия в клетке генерируемого фрагмента,
3. $p(FN) \in (0,1)$ – вероятность того, что реализация ошибочно проигнорирует препятствие в клетке генерируемого фрагмента.

Входом метода «make_patch» являются входные данные источника типа *InputT*.

Выходом метода «make_patch» является фрагмент карты проходимости пространства. Фрагментом является сетка фиксированного разрешения *pix_per_meter*. Те элементы сетки, которые, согласно реализуемому алгоритму, заняты препятствиями, равны логарифму шансов вероятности корректной работы алгоритма:

$$l_+ = \ln \frac{p(TP)}{1 - p(TP)}.$$

Те элементы сетки, которые, согласно реализуемому алгоритму, не заняты какими-либо препятствиями, равны логарифму шансов вероятности ошибки алгоритма:

$$l_- = \ln \frac{p(FN)}{1 - p(FN)}.$$

Все остальные значения сетки заполнены 0, что соответствует вероятности занятости пространства 0,5:

$$p(occ) = \frac{\exp(l)}{1 + \exp(l)} \stackrel{l=0}{=} \frac{1}{2}$$

Помимо самой сетки результирующий фрагмент хранит её разрешение, а также положение источника данных относительно системы координат генерируемой сетки. Всё это описание хранится в виде структуры «MapPatch»:

```
struct MapPatch{
    cv::Mat data;
    float res;
    cv::Point2f origin;
};
```

Здесь сетка хранится в виде структуры «cv::Mat» из библиотеки Open Source Computer Vision Library (OpenCV) [110], это позволяет упростить дальнейшие манипуляции с фрагментом при добавлении на общую карту проходимости: такие как поворот и квантование. Такая форма представления может быть полезна и для классов наследников при построении фрагмента, например для добавления на фрагмент геометрических примитивов.

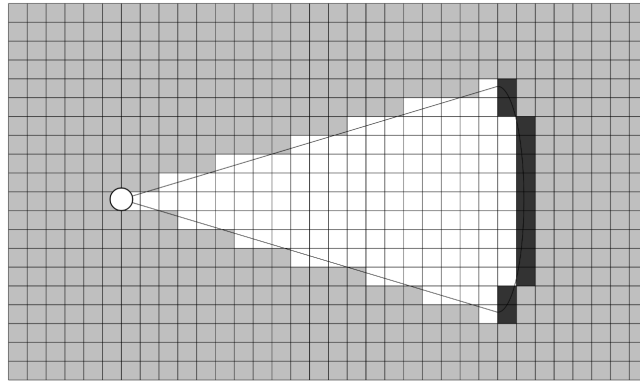


Рисунок 3.3 — Иллюстрация поля зрения, занимаемого препятствием.

Класс «ObstaclesMPP»

Класс реализует интерфейс «IMapPatchProvider». Входным типом для класса является набор полигонов, ограничивающих области, занятые препятствиями. Такие данные могут приходить от сонаров или алгоритмов, представленных в разделе 2.1.

Инициализация класса полностью совпадает с инициализацией «IMapPatchProvider».

Входом метода «make_patch» являются полигоны, ограничивающие области, занимаемые препятствиями. Координаты вершин полигонов описаны в системе координат источника данных.

```
| using InputT = std::vector<std::vector<cv::Point2f>>;
```

Выходом метода «make_patch» является фрагмент карты проходимости пространства в формате «MapPatch». Области ограниченные входными полигонами заполнены значениями l_+ . Дополнительно метод может (если задать $p(FN) \neq 0,5$) заполнить поле зрения, занимаемое входными полигонами, значениями l_- (рисунок 3.3). Это повышает качество генерируемой карты проходимости пространства, если наблюдаемые препятствия не могут заслонять друг друга и, следовательно, можно судить об отсутствии препятствий в пространстве между источником данных и препятствием.

Класс «True3DCamDetectionMPP»

Класс «True3DCamDetectionMPP» реализует интерфейс «IMapPatchProvider». Входной тип «InputT» – облако точек препятствий с лидара, полученное алгоритмом разделения облака точек (раздел 2.3.1) и двумерные детекции с RGB-камеры БТС.

```
struct BBox2D{
    int min_x = 0;
    int min_y = 0;
    int max_x = 0;
    int max_y = 0;
};
using InputT = std::pair<pcl::PointCloud<PointT>, std::vector<
    BBox2D>>;
```

Здесь облако точек хранится в формате «pcl::PointCloud<PointT>» библиотеки Point Cloud Library (PCL) [111]. Данная библиотека широко распространена и предлагает функционал по вычислительно эффективной обработке облаков точек. Также её преимуществом являются готовые библиотеки, предоставляющие возможность конвертации классов библиотеки PCL (в частности, облаков точек) в ROS сообщения и обратно, например, pcl_conversions [112]. В качестве структуры точки, хранимой в облаке точек «PointT» может выступать любая структура с обязательными полями «x», «y» и «z», которые являются числами с плавающей точкой «float».

Метод «make_patch» реализует алгоритм представленный в разделе 2.3 проекции двумерных детекций с камеры БТС в систему координат БТС O_l с использованием комплексирования данных с лидара с последующей растеризацией полученных ограничивающих рамок. В случае отсутствия кластера облака точек препятствий, соответствующего двумерной детекции (см. шаг 6 алгоритма 1), метод опционально может спроецировать детекцию в систему координат, используя наивный алгоритм из раздела 2.1.1, чтобы хоть и с меньшей точностью, но учесть объект в генерируемом фрагменте карты проходимости.

Инициализация класса помимо стандартных параметров «IMapPatchProvider» включает следующие параметры:

1. параметры кластеризации облака точек:

- а) *min_size, max_size* – минимальное и максимальное число точек кластеров (кластеры за пределами этих границ игнорируются);
 - б) *tolerance* – максимальное расстояние Евклида между точками, при котором они считаются точками одного кластера;
 - в) *distance_clip* – максимальное расстояние Евклида от центра системы координат O_l до кластеризуемых точек (точки, расположенные далее удаляются из облака);
 - г) *z_squeeze* – коэффициент сжатия координат вдоль оси Oz_l (ввиду принципа формирования облака точки располагаются реже вдоль этой оси, чем в плоскости Oxy_l);
2. параметры метода RANSAC для вписывания кластера в ограничивающую рамку:
- а) *ransac_tolerance* – максимальное расстояние Евклида от линии-гипотезы RANSAC до точек кластера, чтобы считать их соответствующими гипотезе;
 - б) *iterations_num* – число итераций метода RANSAC;
3. настройки наивного алгоритма проекции детекций в случае отсутствия сопоставленного кластера:
- а) *enable* – булев тип для включения алгоритма;
 - б) *bev_depth* – глубина прямоугольника, строящегося на отрезке после проекции нижней границы препятствия;

Входом метода «make_patch» является пара: облако точек препятствий в формате «pcl::PointCloud» и двумерные детекции с камеры в формате «std::vector<BBox2D>».

Выходом метода «make_patch» является фрагмент карты проходимости в формате «MapPatch», на котором области внутри полученных алгоритмами из разделов 2.3 и 2.1.1 (опционально) ограничивающими рамками заполнены l_+ , остальные элементы фрагмента заполнены 0.

Класс «HeightMapTrue3dSegmentationMPP»

Класс «HeightMapTrue3dSegmentationMPP» реализует интерфейс «IMapPatchProvider». Метод «make_patch» реализует алгоритмы из раздела 2.5 проецирования семантической сегментации на вид сверху в окрестности БТС.

Инициализация класса помимо стандартных параметров «IMapPatchProvider» включает следующие параметры:

1. параметры построения карты высот:

а) *height_map_creator_type* – параметр задающий способ построения карты высот над плоскостью движения БТС. Доступны варианты:

1) *ortho_projection* – использование готовой карты высоты, которая прямоугольно проецируется на плоскость движения БТС;

2) *delanay* – карта высот инициализирована частично, неизвестные значения высоты линейно интерполируются согласно триангуляции Делоне;

2. *use_gpu* – булев тип, указывающий использовать ли алгоритм проекции с использованием параллельных вычислений на архитектуре CUDA.

Входом метода «make_patch» является пара: облако точек земли в формате «pcl::PointCloud» и двумерная семантическая сегментация проезжей части с камеры в формате одноканального изображения «cv::Mat».

```
| using InputT = std::pair<pcl::PointCloud<PointT>, cv::Mat>;
```

Выходом метода «make_patch» является фрагмент карты проходимости «MapPatch», построенный алгоритмом 4. Области, согласно алгоритму соответствующие свободному для проезда БТС пространству, заполнены l_- , занятые препятствиями – l_+ , остальные – 0.

Класс «PCObstaclesMPP»

Класс «PCObstaclesMPP» реализует интерфейс «IMapPatchProvider». Метод «make_patch» производит прямоугольную проекцию облака точек препятствий на плоскость карты проходимости пространства.

Инициализация класса помимо стандартных параметров «IMapPatchProvider» включает следующие параметры:

1. параметры отбора точек:

- а) min_z, max_z – интервал координаты z точек, попадающих на фрагмент карты проходимости пространства (необходим для фильтрации точек, находящихся выше БТС и не препятствующих его движению);
- б) $inplane_max_dist$ – максимальное расстояние Евклида в плоскости Oxy_l от центра координат до точек, попадающих на итоговый фрагмент.

Входом метода «make_patch» является: облако препятствий в формате «pcl::PointCloud».

```
| using InputT = pcl::PointCloud<PointT>;
```

Выходом метода «make_patch» является фрагмент карты проходимости пространства «MapPatch», элементы фрагмента, в которые попали проекции точек инициализированы l_+ , остальные – 0.

Класс «ReadyPatchMPP»

Класс «ReadyPatchMPP» реализует интерфейс «IMapPatchProvider». Класс позволяет расширить количество источников информации, с которыми может взаимодействовать библиотека без изменения содержащегося в ней кода. Это упрощает для пользователей возможность экспериментировать с дополнительными алгоритмами, учитываемыми в карте проходимости пространства БТС.

Инициализация класса полностью совпадает с инициализацией «IMapPatchProvider».

Входом метода «make_patch» является готовый фрагмент карты проходимости пространства «MapPatch». Элементы фрагмента заполнены значениями в интервале $[-1, 1]$, значения в интервале $[-1, 0)$ интерпретируются как области, свободные для проезда БТС, в интервале $(0, 1]$ – занятые препятствиями.

```
| using InputT = MapPatch;
```

Выходом метода «make_patch» является фрагмент карты проходимости пространства «MapPatch» того же размера и разрешения, что и входной фрагмент. Значения элементов определяются по следующей формуле:

$$l_{out} = \begin{cases} l_- & , \text{если } l_{in} < 0 \\ 0 & , \text{если } l_{in} = 0 , \\ l_+ & , \text{если } l_{in} > 0 \end{cases}$$

где l_{out} – итоговое значение итогового фрагмента, l_{in} – оригинальное значение входного фрагмента.

Класс «DynamicEraser»

В некоторых задачах планирования пути БТС важно различать статические и динамические препятствия (люди, машины, велосипедисты и пр.), окружающие БТС и обрабатывать их разными алгоритмами. Чтобы сделать это возможным в рамках библиотеки «occupancy_map_fusion» был создан класс «DynamicEraser», который осуществляет удаление информации об областях, занимаемых динамическими препятствиями.

Инициализация класса осуществляется одним параметром:

bbox_expand_factor – параметр, задающий степень увеличения входящих динамических препятствий, чтобы учесть погрешность в определении их положения.

Класс обладает двумя методами:

1. «set_dynamic_obstacles» – сохраняет в объекте класса наблюдаемые на данный момент динамические препятствия в окрестности БТС. Препятствия описываются повернутыми ограничивающими рамками на плоскости карты проходимости пространства:

```

struct DynamicObstacle{
    struct {
        float x;
        float y;
    } center;
    struct {
        float x;
        float y;
    } dims;
    float heading;
};

```

2. «clear_dynamic_obstacles» – осуществляет удаление динамических препятствий из карты проходимости пространства. На вход алгоритму подаётся текущая карта проходимости пространства: для всех областей, занимаемых сохранёнными динамическими препятствиями, соответствующие значения элементов карты становятся равными 0.

Класс «OccupancyMapManager»

Класс «OccupancyMapManager» осуществляет хранение и обработку итоговой карты проходимости пространства вокруг БТС на основе фрагментов, генерируемых реализациями «IMapPatchProvider», и информации о перемещении БТС в глобальной системе координат.

Класс имеет следующие основные методы:

1. «add_source». Метод, регистрирующий новый источник информации, учитываемый в карте проходимости пространства. Аргументом функции является положение поля генерируемых источником фрагментов «MapPatch». Положение описывается структурой:

```

struct Pose2f{
    float x;
    float y;
    float heading;
};

```

x, y – положение *origin* генерируемых «MapPatch» в системе координат O_i БТС. *heading* – угол поворота генерируемых фрагментов в той же системе координат.

Метод возвращает уникальный идентификатор источника, по которому в дальнейшем будет производиться преобразование фрагмента в общую систему координат перед тем, как он будет добавлен на карту проходимости пространства.

2. **«map_update»**. Существует две перегрузки данного метода. Первая – без каких-либо аргументов. При вызове этой перегрузки, метод находит последнее известное положение БТС в глобальной системе координат и сдвигает карту с учётом положения, в котором в последний раз была сохранена карта. Вторая перегрузка принимает на вход три параметра: фрагмент карты проходимости пространства «MapPatch», идентификатор источника данных, который сгенерировал данный фрагмент, и время, которому этот фрагмент соответствует. При вызове этой перегрузки метод корректирует положение карты проходимости (если время регистрации фрагмента позднее времени последнего сохранения карты), осуществляет сдвиг и поворот фрагмента, чтобы сопоставить его с картой проходимости пространства и добавляет его путём суммирования элементов фрагмента с элементами карты проходимости (см. «Поток N+1» на рисунке 3.2). Дополнительно после этого шага может быть произведено удаление динамических препятствий с карты проходимости. Также в обеих перегрузках данного метода происходит экспоненциальное затухание элементов карты проходимости или фрагмента, а также их отсечение, чтобы уменьшить количество артефактов, полученных в результате ошибок источников информации. Подробнее об этих двух операциях будет рассказано в главе 4.
3. **«get_raw_map»**. Метод не принимает никаких параметров и возвращает последнюю сохранённую карту проходимости пространства без квантования значений элементов карты.
4. **«get_occupancy_map»**. Метод не принимает никаких параметров и возвращает последнюю сохранённую карту проходимости пространства, полученную после квантования:

$$l_{quant} = \begin{cases} -1 & , \text{если } l_{raw} < -l_{thresh} \\ 1 & , \text{если } l_{raw} > l_{thresh} \\ 0 & , \text{в остальных случаях} \end{cases},$$

где l_{quant} итоговое значение элемента после квантования, l_{raw} – оригинальное значение элемента, $l_{thresh} > 0$ – порог квантования, задаваемый параметром класса.

Для реализации описанного функционала класс хранит путём композиции несколько специализированных классов:

1. **«MapTransformEstimator»**. Класс отвечает за сбор, хранение и вычисление положений БТС в глобальной системе координат в течение времени работы класса. Часть функционала класса делегирована интерфейсу «IPoseBufferEngine» для реализации принципа инверсии зависимостей [113], поскольку в системе ROS уже существует механизм для оценки относительных перемещений систем координат, используемых в описании движения БТС [114].
2. **«MapHandler»**. Класс отвечает за хранение, модификацию и извлечение карты проходимости пространства. К модификациям карты относятся: сдвиг, экспоненциальное затухание, клиппинг и добавление фрагментов приведённых в систему координат карты.
3. **«PatchHandler»**. Класс отвечает за преобразование фрагментов карты (сдвиг, поворот, экспоненциальное затухание фрагментов) в систему координат карты.
4. **«DynamicEraser»**. Опциональный класс, реализующий удаление информации об участках, занятых динамическими препятствиями, как описано в предыдущем разделе.

3.3.2 Программа «evo_occupancy_map_fusion»

Программа «evo_occupancy_map_fusion» осуществляет интеграцию функционала в систему ROS, предоставляя классы осуществляющие преобразование ROS-сообщений в классы и структуры, используемые библиотекой «occupancy_map_fusion» (тем самым реализуя структурный шаблон проектирования «Адаптер» [115]). Программа включает следующие основные классы:

1. **«IMPPBridge»**. Класс «Адаптер» класса «IMapPatchProvider». Реализации этого класса в отдельном программном потоке подписываются на один («SingleTopicBridge») или несколько («MultipleTopicBridge») ROS-топиков, в которые поступают сообщения на обработку программой, преобразует их в формат библиотеки «occupancy_map_fusion» и передаёт адаптируемому объекту класса «IMapPatchProvider», к сформированному фрагменту добавляется метайнформация: идентификатор источника данных и время регистрации отражаемых им данных, после чего структура добавляется в очередь «MapPatchQueue». Помимо этого класс предоставляет возможность временно отключить учёт, соответствующего ему источника данных в карте проходимости пространства через ROS-сервис – модель коммуникации компонентов ROS путём двунаправленной синхронной связи между клиентом и сервером.
2. **«MapPatchQueue»**. Класс, реализующий очередь и гарантирующий потоковую безопасность при добавлении и извлечении фрагментов с метайнформацией.
3. **«DynamicEraserROSAdapter»**. Класс, аналогично «IMPPBridge» подписывающийся на сообщения с динамическими препятствиями, конвертирующий их в «std::vector<DynamicObstacle>» и передающий результат в хранимый внутри класса объект «DynamicEraser».
4. **«TFBufferEngine»**. Класс реализует функционал, необходимый для работы «MapTransformEstimator» адаптируемой библиотеки.
5. **«OccupancyMapManagerFacade»**. Класс, исполняющий основной цикл обновления и публикации карты проходимости пространства вокруг БТС («Поток N+1» на рис. 3.2). Метод «run» в цикле пытается извлекать новые фрагменты из очереди «MapPatchQueue», при успехе фрагмент и метайнформация передаются в метод «map_update» хранимого в классе объекта «OccupancyMapManager», после чего, если класс «DynamicEraserROSAdapter» инициализирован, происходит удаление областей, занимаемых динамическими препятствиями. Далее, если с момента последней публикации карты проходимости пространства прошло больше заданного в параметрах класса времени происходит извлечение карты проходимости через метод «get_occupancy_map» класса «OccupancyMapManager», который затем преобразуется в ROS-сообщение.

ние типа «nav_msgs/OccupancyGrid» и публикуется в отдельный топик для сообщений данного типа.

3.4 Заключение к главе 3

В данной главе разработан программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных от датчиков, установленных на БТС, и их обработчиков. Комплекс реализует алгоритмы, описанные в главе 2, а также позволяет пользователям интегрировать собственные алгоритмы построения фрагментов карты проходимости пространства. Для представленного программного комплекса было получено свидетельство о регистрации программы для ЭВМ: № 2025669744 Программное обеспечение «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства».

Глава 4. Параллельный алгоритм построения карты проходимости пространства на основе комплексирования разнородных данных

В главе 3 описан программный комплекс, реализующий построение карты проходимости пространства на основе множества показаний с датчиков, установленных на БТС. Несмотря на показанную работоспособность для реальных БТС, данный комплекс может задерживать добавление сформированных фрагментов на карту в случае большого количества учитываемых в модели источников данных. Данное свойство комплекса ограничивает число датчиков, которые могут участвовать в уточнении карты проходимости пространства, и, следовательно, ограничивает надёжность строящейся карты.

В данной главе предложены модификации алгоритма построения карты проходимости пространства, реализованного в программном комплексе из главы 3, позволяющие снизить время обработки информации об окружающем пространстве при добавлении её на карту проходимости пространства, а также избавиться от необходимости последовательного добавления фрагментов на карту, что позволяет увеличить количество комплексирруемых источников данных с сохранением возможности работы в режиме реального времени.

4.1 Производительность алгоритма построения карты проходимости пространства

Представленный в главе 3 алгоритм построения карты проходимости пространства можно разделить на два этапа:

1. Преобразование входных данных к единому формату представления данных о проходимости пространства. Как правило, это фрагмент карты проходимости, содержащий вероятности занятости клеток, основываясь на последних показаниях данного источника данных.
2. Комплексирование полученных фрагментов с ранее построенной картой проходимости пространства. Для обратной модели наблюдения этот шаг заключается в простом суммировании полученного фрагмента с соответствующей областью карты.

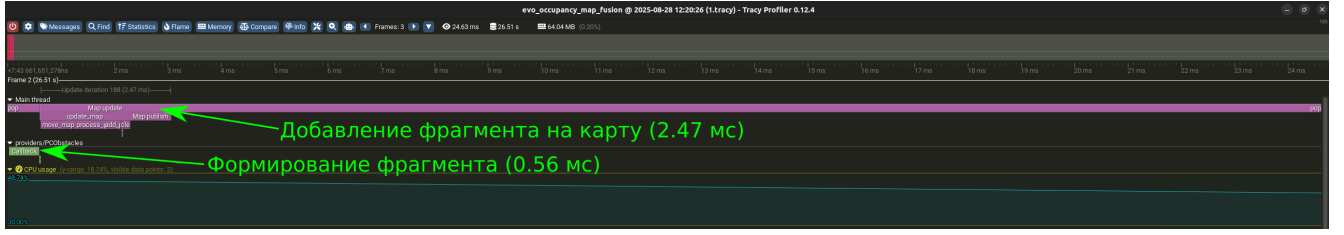
Вычисление первого шага может быть выполнено с помощью асинхронного программирования, так как для построения фрагмента используются данные только одного источника данных, и синхронизация с другими источниками/подсистемами не требуется. Этот шаг может быть вычислительно дорогим в зависимости от сложности используемого алгоритма, однако в рамках данной главы мы предполагаем, что алгоритмы, осуществляющие данный шаг, реализованы оптимальным образом для системы, в которой происходит запуск. Тогда ускорение первого этапа невозможно. В то же время второй шаг является блокирующим: созданный фрагмент добавляется в очередь и включается в карту только после того, как были добавлены все фрагменты, стоящие перед ним в очереди. Таким образом происходит задержка даже в случае, когда добавляемые фрагменты не имеют пересечений и параллельное обновление теоретически возможно. Чем больше источников данных участвует в формировании карты проходимости, тем выше задержки при обновлении карты.

Покажем это на практике: на рисунке 4.1 представлены результаты профилирования «evo_оссирансу_map_fusion» при обработке одного и восьми источников данных. В версии с восемью источниками все провайдеры фрагментов были дубликатами провайдера из запуска с одним источником. Следовательно, все провайдеры обрабатывали одни и те же данные, зарегистрированные в один момент времени. Можно видеть, что из-за последовательного добавления фрагментов на карту проходимости пространства задержки обновления карты проходимости доходят до более чем семикратного замедления.

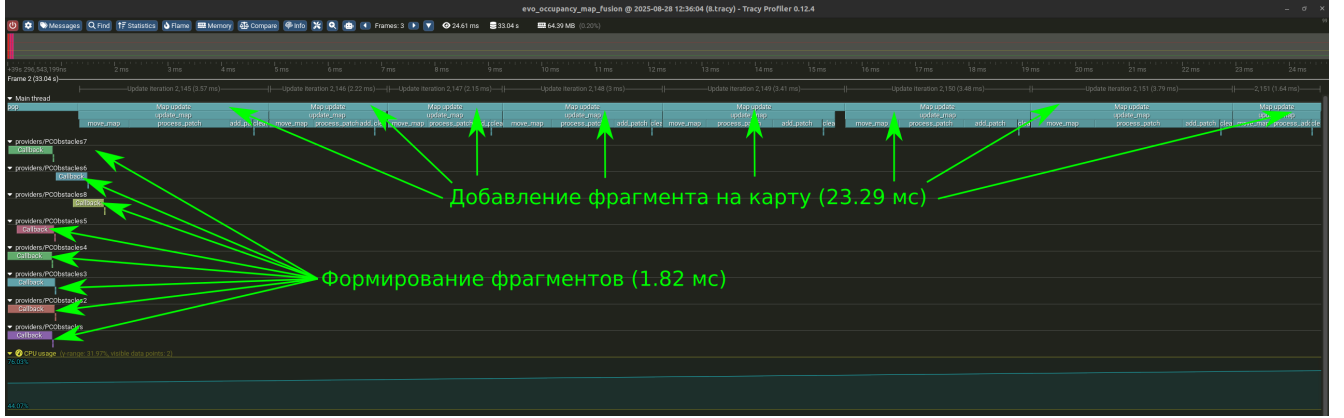
Далее в главе мы предложим модификации базового алгоритма построения карты проходимости, направленные на ускорение второго этапа обновления карты проходимости пространства, а также делающие этот шаг асинхронным для каждого источника данных. Перед тем как приступить к описанию модификаций опишем формально задачу, решаемую алгоритмом.

4.2 Постановка задачи построения карты проходимости пространства

Пусть задана фиксированная относительно земли глобальная система координат, расположенная таким образом, что движение БТС происходит в



а) Один источник данных. Время обновления карты – 3,03 мс.



б) 8 источников данных. Самое долгое время обновления карты – 22,78 мс (замедление на 652 %).

Результаты профилирования получены в программе инструментального профилирования «Трасу» [102]. Приведённые снимки экрана приведены к одному масштабу временной шкалы.

Рисунок 4.1 — Наблюдаемые задержки при обновлении карты проходимости пространства

плоскости $z = 0$, положение БТС в этой системе координат задаётся тремя параметрами: $\mathbf{x} = x, y, \theta$, где x, y координаты фиксированной точки БТС (см. рис 4.2), θ – угол поворота БТС от Ox вокруг вертикальной оси. Введём также локальную систему координат БТС, начало которой находится в той же фиксированной точке x, y , ось Ox направлена вдоль оси движения БТС. Координаты в этой системе координат, мы будем обозначать x_i, y_i . Наконец, пусть у нас есть S источников данных. Каждый источник имеет собственную систему координат, положение которой известно и фиксировано относительно локальной системы координат БТС. Источник s независимо от остальных источников генерирует информацию о занятом и/или свободном пространстве в окрестности БТС в виде наборов $(P_{s,k}, \mathbf{x}_{s,k}, t_{s,k}), s = \overline{1, S}, k \in \mathbb{N}$, где $P_{s,k}$ – фрагмент карты проходимости в виде двумерного массива логарифмов шансов занятости пространства с фиксированным разрешением res_s (pixel per meter, пикселей на метр), оси массива ориентированы вдоль осей системы координат источника s ; $\mathbf{x}_{s,k}$ – положение начала системы координат датчика относительно левого верх-

него угла массива; $t_{s,k}$ – время регистрации сгенерированного фрагмента, k – порядковый индекс сгенерированного набора.

На основе получаемых данных необходимо сформировать локальную карту проходимости пространства в виде набора $(M_i, \mathbf{x}_i, t_i); i \in \mathbb{N}$, где M_i – карта проходимости пространства в момент времени t_i в виде двумерного массива логарифмов шансов занятости пространства с фиксированным разрешением res ; $\mathbf{x}_i$ – координаты левого верхнего угла карты, заданные таким образом, чтобы начало локальной системы координат в момент времени t_i совпадало с центром карты M_i , оси карты параллельны осям глобальной системы координат. Такая ориентация выбрана, чтобы не вносить погрешность в оценку положения свободного/занятого пространства в результате многократных поворотов на малую величину двумерного массива, которая возникала бы при ориентации карты параллельно осям локальной системы координат.

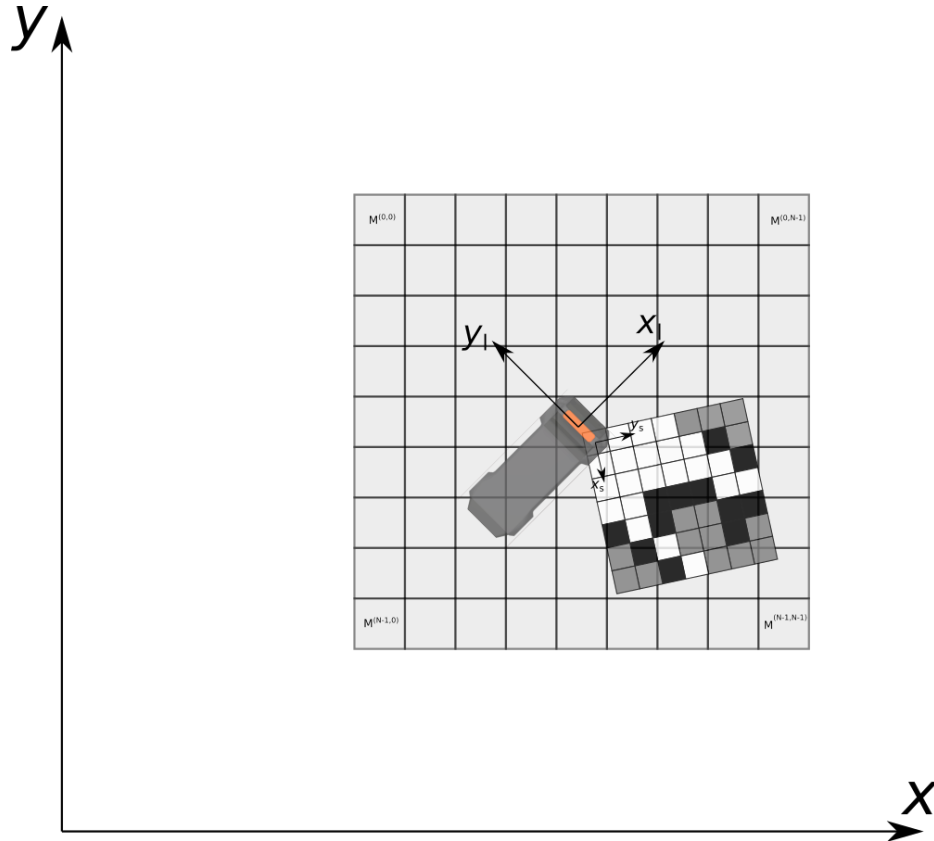


Рисунок 4.2 — Расположение локальной карты проходимости и её фрагмента от одного из источников данных относительно БТС и глобальной системы координат.

Для построения локальной карты проходимости пространства все генерируемые фрагменты P_s карты проходимости пространства добавляются в

очередь в порядке их готовности. Заметим, что время регистрации добавляемых таким образом фрагментов в общем случае не является монотонным (см. рис 4.3). Далее локальная карта проходимости последовательно обновляется добавлением фрагментов из очереди. Алгоритм одной итерации обновления представлен в листинге 5: если $t_{s,k} > t_i$, карта актуализируется к моменту времени фрагмента (строки 2–5): вычисляются новые координаты левого верхнего угла карты, время карты заменяется временем регистрации извлечённого фрагмента, содержимое карты сдвигается и экспоненциально затухает (цель последнего мы опишем далее в этом разделе). В противном случае значения x_i, t_i остаются неизменными: x_{i-1}, t_{i-1} (строки 7–8). Далее извлечённый фрагмент преобразуется: «доворачивается» до ориентации локальной карты проходимости пространства и масштабируется до разрешения локальной карты проходимости, создавая временное представление $P_{aligned}$. Также вычисляется сдвиг левого верхнего угла фрагмента относительно левого верхнего угла локальной карты проходимости пространства *patch_shift* (строка 11). Если время регистрации фрагмента меньше времени карты, к фрагменту применяется экспоненциальное затухание (строка 12). Наконец, фрагмент добавляется к соответствующей ему области на локальной карте проходимости пространства (строка 13).

Недостатком обратной модели наблюдения является предположение о статичности сцены: в случае наличия динамических объектов в сцене, такие объекты оставляют за собой «след», который не удаляется с локальной карты проходимости пространства, если нет источника данных, способного отмечать области, в которых препятствия отсутствуют. Даже при наличии такого источника, «след» может попасть в его слепую зону и остаться на карте. Для решения данной проблемы, мы производим экспоненциальное затухание значений в клетках карты, тем самым постепенно возвращая их в состояние «неизвестно» без повторных обновлений от источников данных (см. листинг 6)

Также на шаге добавления фрагмента на карту мы вводим механизм против «перезарядки» клеток карты: если динамическое препятствие долго стояло неподвижно, а потом сдвинулось, то логарифм шансов освободившегося пространства останется ненулевым ($|l_t| \gg 0$) и при квантовании продолжит отображаться как занятое пространство. Затухание значений в конце концов вернёт клетки освободившегося пространства в состояние «неизвестно» или «свободно», но без ограничения значений хранимых в клетках этот процесс

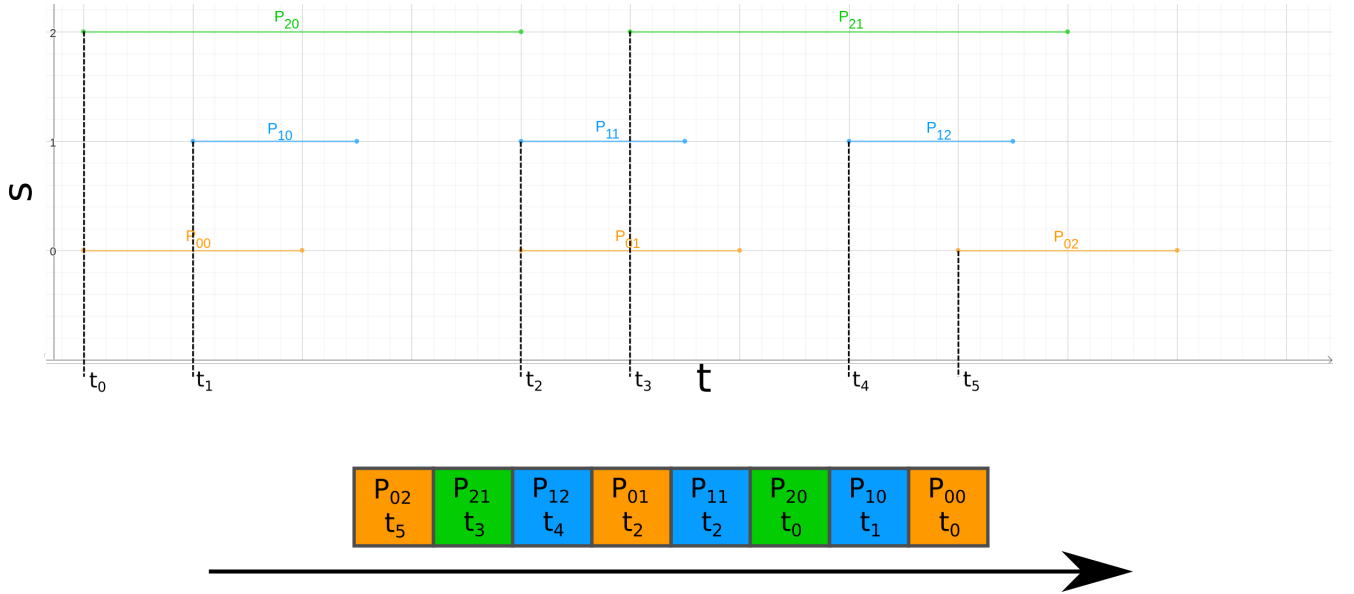


Рисунок 4.3 — Демонстрация формирования очереди фрагментов. Горизонтальные отрезки обозначают время формирования фрагмента каждым источником: начало отрезка соответствует времени регистрации данных $t_{s,k}$, конец отрезка соответствует времени завершения формирования фрагмента и моменту добавления его в очередь фрагментов. Внизу иллюстрации продемонстрирован итоговый порядок фрагментов в очереди: фрагменты извлекаются из очереди в порядке справа налево.

может занять продолжительное время. Чтобы этого избежать, мы ограничиваем абсолютную величину максимального значения, которое может принимать клетка:

$$l_t = \max(\min[\text{inv_obs_model}(t), l_{\text{thresh}}], -l_{\text{thresh}}),$$

где $\text{inv_obs_model}(t)$ – значение логарифма шансов, полученное по формуле 1.1, l_{thresh} – пороговое значение логарифма шансов по абсолютному значению.

4.3 Модификации базовой реализации

В данном разделе мы предлагаем модификации базовой реализации для снижения задержки между появлением информации о проходимости у одного из источников данных и её добавлением на итоговую карту проходимости пространства. Для простоты восприятия, мы упростим задачу и будем считать

Input: $P_{s,k}$ – фрагмент извлечённый из очереди;

$x_{s,k}$ – положение извлечённого фрагмента;

$t_{s,k}$ – время регистрации извлечённого фрагмента;

x_{i-1}, t_{i-1} – положение БТС и время при последнем обновлении карты;

M_{i-1} – карта проходимости пространства после последнего обновления;

res – разрешение карты проходимости;

res_s – разрешение извлечённого фрагмента;

Output: M_i – карта после добавления фрагмента

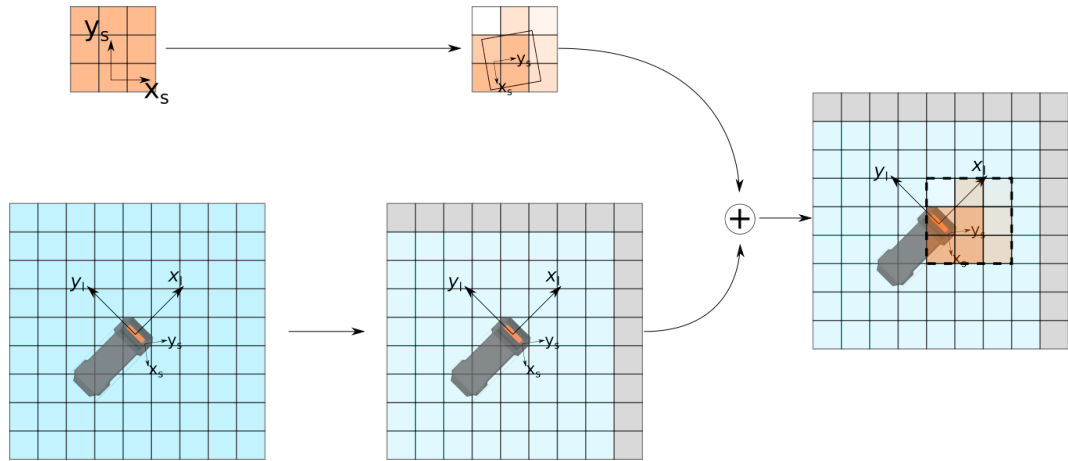
```

1 if  $t_{s,k} > t_{i-1}$  then
2    $x_i \leftarrow get\_map\_pose(t_{s,k});$ 
3    $t_i \leftarrow t_{s,k};$ 
4    $M_{actual} \leftarrow shift\_map(M_{i-1}, x_i - x_{i-1});$ 
5    $M_{actual} \leftarrow dim(M_{actual}, t_i - t_{i-1});$ 
6 else
7    $x_i \leftarrow x_{i-1};$ 
8    $t_i \leftarrow t_{i-1};$ 
9    $M_{actual} \leftarrow M_{i-1};$ 
10 end
11  $P_{aligned}, patch\_shift \leftarrow align\_patch(P_{s,k}, t_{s,k}, t_i, res/res_s);$ 
12  $P_{aligned} \leftarrow dim(P_{aligned}, t_i - t_{s,k});$ 
13  $M_i \leftarrow add\_patch(M_{actual}, P_{aligned}, patch\_shift);$ 
14 return  $M_i$ 

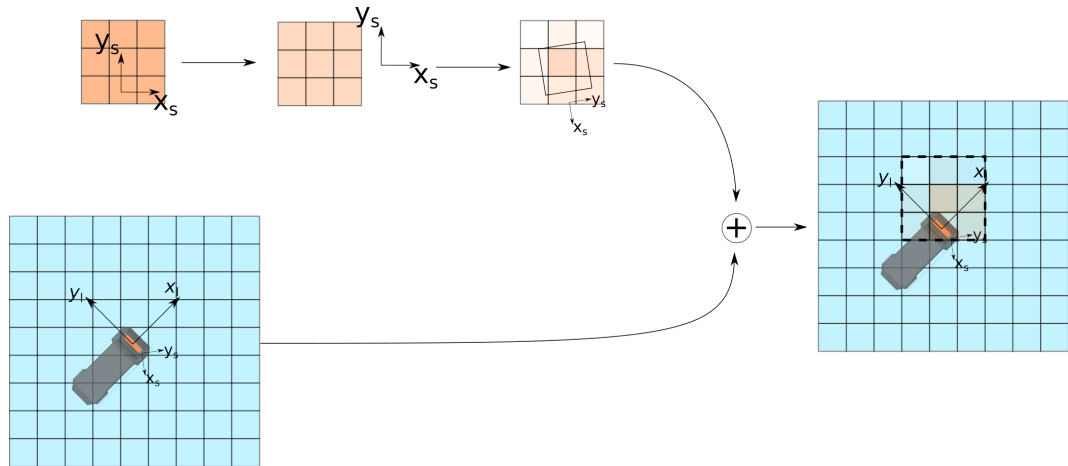
```

Алгоритм 5: Добавление фрагмента на карту: *update_map*.

пространство одномерным: карта в таком случае представляет собой одномерный массив M размера N , а движение возможно только влево или вправо. Также для простоты мы будем считать размер одной клетки равным 1 метру, чтобы опустить операции с учётом разрешения карты res . В прошлом разделе мы показали, что карта должна поддерживать операции сдвига и добавления фрагмента. Помимо этого мы отдельно добавим к описанию реализаций операцию получения карты, поскольку в модификациях данная операция перестаёт быть тривиальным копированием массива и требует отдельного обсуждения.



а) Время регистрации фрагмента новее последнего времени обновления карты.



б) Время регистрации фрагмента старше последнего времени обновления карты.

Рисунок 4.4 — Иллюстрация алгоритма добавления фрагмента от источника данных s на локальную карту проходимости пространства.

4.3.1 Базовая реализация

Для начала опишем исходную реализацию в одномерном случае. Карта представляет собой массив M размера N , текущую координату центра x_i и время t_i , которому текущая карта соответствует (рис. 4.5).

Сдвиг

При сдвиге в новую точку x_{i+1} , в момент времени t_{i+1} массив со значениями логарифма шанса проходимости сдвигается на $\lfloor x_{i+1} - x_i \rfloor$ клеток. Новые

Input: $\widehat{M}_{i-1}$ – карта или фрагмент, элементы которого затухают;
 $\Delta t \geq 0$ – разница текущего времени и времени регистрации $\widehat{M}$;
 a – гиперпараметр

Output: $\widehat{M}_i$ карта или фрагмент после затухания

```

1  $\widehat{M}_i \leftarrow$  Двумерный массив размера:  $rows(\widehat{M}_{i-1}) \times cols(\widehat{M}_{i-1})$ ;
2 for  $j = 0$  to  $rows(M)$  do
3   for  $k = 0$  to  $cols(M)$  do
4      $\widehat{M}_i^{j,k} \leftarrow \exp(-a\Delta t)\widehat{M}_{i-1}^{j,k}$ ;
5   end
6 end
7 return  $\widehat{M}_i$ 

```

Алгоритм 6: Экспоненциальное затухание карты или фрагмента:
 $dim(\widehat{M}_{i-1}, \Delta t)$.

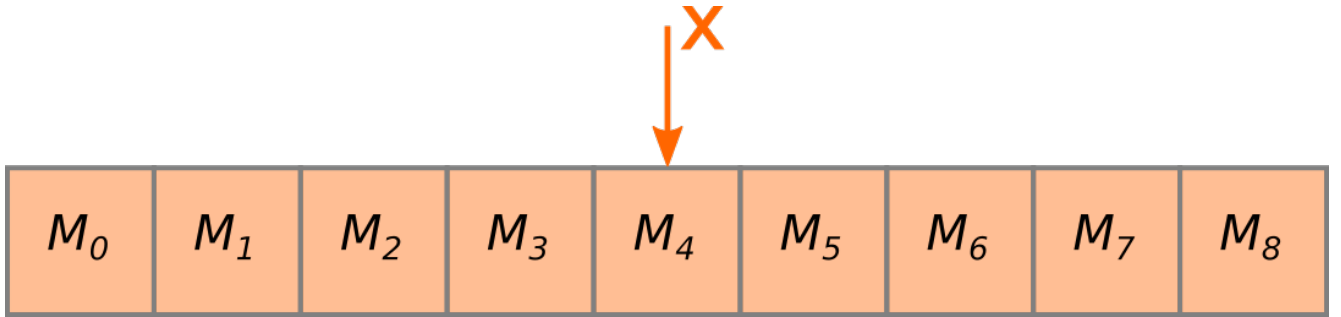


Рисунок 4.5 — Оригинальная реализация локальной карты проходимости пространства.

клетки инициализируются 0. Вычислительная сложность такой операции $\Theta(N)$. Итоговая реализация алгоритма представлена в листинге 7.

Добавление фрагмента

Процесс добавления фрагмента в одномерном случае во многом повторяет алгоритм, представленный в разделе 4.2. Отметим только, что в данном случае «доворачивание» фрагмента не требуется, и необходимо только учесть сдвиг карты за промежуток времени между моментом регистрации фрагмента и итоговым временем регистрации карты после обновления. Дополнительно введём

Input: M – карта до сдвига;
 x_i – положение M до сдвига;
 x_{i+1} – новое положение карты

Output: M после сдвига

```

1  $\Delta x \leftarrow \text{round}(x_{i+1} - x_i);$ 
2  $j \leftarrow 0;$ 
3 for  $j = 0$  to  $N$  do
4   if  $j + \Delta x \in [0; N)$  then
5      $M[j] \leftarrow M[j + \Delta x];$ 
6   else
7      $M[j] \leftarrow 0;$ 
8   end
9 end
10 return  $M$ 

```

Алгоритм 7: Сдвиг карты (базовая реализация): $\text{shift_map}(M, x_{i+1})$.

обозначение размера фрагмента P : N_P . Итоговая реализация алгоритма представлена в листинге 8.

Первая часть алгоритма (до цикла добавления фрагмента на карту) имеет вычислительную сложность $\Theta(1)$, если $t_{s,k} \leq t_{i-1}$, и, как показано в предыдущем пункте, $\Theta(N)$ (операция затухания карты имеет ту же вычислительную сложность) в противном случае. Если фрагмент целиком помещается на карту, во второй части алгоритма будет выполнено $\Theta(N_P)$ операций. Если же фрагмент больше карты и полностью её покрывает, будет выполнено $\Theta(N)$ операций. В то же время вторая часть алгоритма может не выполняться вовсе в случае, если фрагмент не пересекается с картой. Таким образом, вычислительная сложность второй части алгоритма: $O(\min(N_P, N))$, а суммарная вычислительная сложность всего алгоритма: $O(N + \min(N_P, N))$.

Получение карты

Для получения карты необходимо просто вернуть копию текущего массива. Сложность операции: $\Theta(N)$. Копия требуется для того, чтобы не бло-

Input: M – карта проходимости пространства;
 P_s – добавляемый фрагмент;
 $x_{t_{s,k}}$ – положение карты M в момент $t_{s,k}$;
 $x_{s,k}$ – положение источника данных s в момент регистрации фрагмента;
Output: M после добавления фрагмента

```

1 if  $t_{s,k} > t_i$  then
2   |  $M \leftarrow \text{shift\_map}(M, x_{t_{s,k}});$ 
3   |  $M \leftarrow \text{dim}(M, t_{s,k} - t_i);$ 
4 else
5   |  $x_{s,k} \leftarrow x_{s,k} - (x_{t_i} - x_{t_{s,k}});$ 
6   |  $P_s \leftarrow \text{dim}(P_s, t_i - t_{s,k});$ 
7 end
8  $\Delta x_s \leftarrow \text{round}(x_{s,k} - x_i);$ 
9 for  $j \leftarrow \max(0, \Delta x_s)$  to  $\min(N_P; N + \Delta x_s)$  do
10  |  $M[j - \Delta x_s] \leftarrow M[j - \Delta x_s] + P_s[j];$ 
11 end

```

Алгоритм 8: Добавление фрагмента (базовая реализация в одномерном случае): $\text{update_map}(P_s, t_{s,k}, x_{s,k})$.

кировать последующие операции добавления фрагментов и сдвига. В силу тривиальности алгоритма мы не приводим здесь его реализацию.

4.3.2 Расширенная карта проходимости пространства

Первая модификация локальной карты проходимости пространства направлена на снижение стоимости сдвига. Для этого массив для хранения карты увеличивается до размера ($\hat{N} > N$), чтобы описывать область больше, чем отображаемая карта.

В такой реализации к уже упомянутому массиву и координате x добавляется координата, соответствующая некоторой фиксированной точке «полного» массива $\hat{x}$. Возьмём в качестве такой точки центр массива (Рис. 4.6).

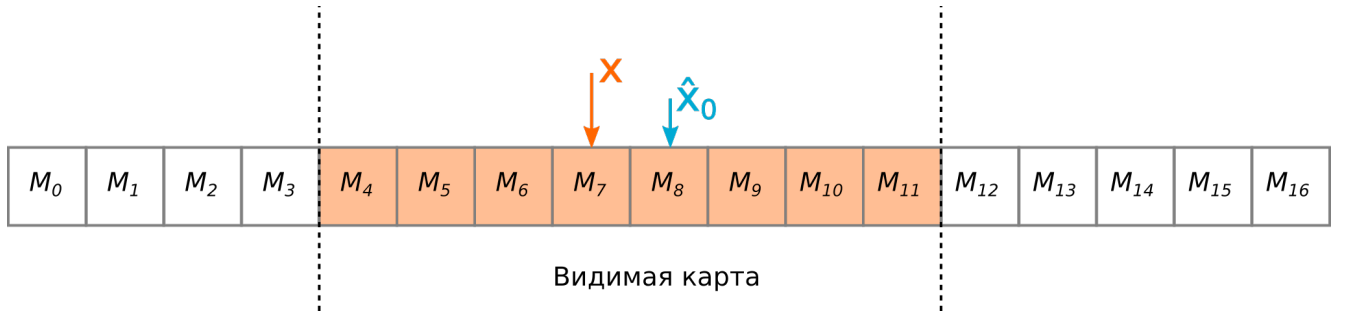


Рисунок 4.6 — Карта с подготовленными окрестностями.

Сдвиг

После описанной модификации для сдвига, в общем случае будет достаточно только обновить значение x , соответственно эта операция требует константное время для исполнения $\Theta(1)$. Если же при сдвиге «отображаемая» часть карты выходит за границы полного массива, его содержимое сдвигается так, чтобы новый $\hat{x}$ совпал с актуальным x . В этом случае сложность, как и в оригинальной реализации, $\Theta(\hat{N})$. Как итог, при правильно выбранном размере массива амортизированная сложность операции: $\Theta(1)$.

```

1 if  $(\hat{x} - x_{i+1}) \in [N/2; \hat{N} - N/2)$  then
2   |   return M;
3 end
4 ...
5 // Аналогично базовой реализации. Применяется ко всем клеткам
   расширенной карты, в том числе за пределами видимой области
6 ...

```

Алгоритм 9: Сдвиг карты (расширенная модификация):
 $shift_map(M, x_{i+1})$.

Добавление фрагмента

Операция добавления фрагмента в точности повторяет аналогичную операцию из оригинальной реализации. Сложность алгоритма: $O(\min(N_P, N))$,

если не происходит затухания карты и $O(\hat{N} + \min(N_P, N))$ в противном случае (мы должны учитывать клетки за границами видимой карты, т.к. они могли быть ранее обновлены и вернуться в область видимости в будущем).

Получение карты

Операция получения карты заключается в копировании подмассива, соответствующего видимой части карты. Как и прежде, сложность операции – $O(N)$. Все операции по-прежнему остаются блокирующими.

4.3.3 Карта с разделяемыми блокировками

Блокировки обновления карты становятся узким местом алгоритма при увеличении числа источников данных, используемых для заполнения карты. Каждый источник вынужден ждать своей очереди для доступа к карте, даже если добавляемые фрагменты не пересекаются между собой. Чтобы снизить задержку в таком случае, мы предлагаем вторую модификацию локальной карты проходимости пространства, изменяющую структуру самой клетки карты. Ранее клетка содержала одно число с плавающей точкой. Теперь это будет атомарная пара чисел: значение логарифма шансов занятости клетки и время, которому оно соответствует (таким образом, мы более не требуем, чтобы все клетки массива соответствовали одному моменту времени). Заметим, что при разработке атомарной структуры данных (т.е. структуры данных, не использующей примитивы блокировки: мьютексы, семафоры, сигналы и т.п. при чтении/записи) важно учитывать её размер и функциональные возможности аппаратного обеспечения. В нашем случае структура состоит из двух 32-битных полей и имеет итоговый размер – 64 бита. Большинство современных архитектур имеют инструкции, обеспечивающие атомарные операции с такими структурами данных [116].

Сдвиг

Операция сдвига не претерпела значительных изменений в сравнении с предыдущей реализацией, с тем уточнением что при копировании клеток во время «полноценного» сдвига копируется не только значение логарифма шансов, но и время, а также в данной реализации необходима инициализация времени клетки (мы используем значение 0, т.к. оно не влияет на последующие операции обновления клетки). Сложность операции остаётся амортизированной $\Theta(1)$ и в худшем случае имеет сложность $\Theta(\hat{N})$. Заметим однако, что реальная константа сложности увеличится из-за использования атомарных структур, копирование и инициализация которых значительно продолжительнее, чем у неатомарных аналогов.

Добавление фрагмента

При добавлении фрагмента на карту многие шаги из предыдущего раздела повторяются. Сперва мы определяем необходим ли сдвиг карты. Если нет, добавление может быть выполнено одновременно с другими такими же операциями, не требующими сдвига. В противном случае операция ждёт возможности взять уникальные права на модификацию карты и произвести требуемый сдвиг.

Далее мы корректируем координаты, куда требуется добавить фрагмент карты проходимости пространства. Для каждой клетки мы атомарно считываем её значения, аналогично предыдущим версиям алгоритма пересчитываем новое значение клетки с учётом затухания и пытаемся атомарно записать в клетку, если её значение не изменилось за то время, что мы вычисляли новое значение клетки (сравнение с обменом, Compare and Swap, CAS). Если клетка поменяла своё значение, мы пересчитываем новое значение клетки с учётом обновившегося состояния обновляемой клетки до тех пор, пока не произведём успешную запись. Описанный алгоритм приведён в листинге 10

Теоретически данная операция может выполняться неограниченно долго, если между попытками сравнения с обменом происходит обновление значения клетки в другой операции. На практике незавершаемость операции практи-

Input: M – карта проходимости пространства;
 P_s – добавляемый фрагмент;
 $x_{t_{s,k}}$ – положение карты M в момент $t_{s,k}$;
 $x_{s,k}$ – положение источника данных s в момент регистрации фрагмента;
Output: M после добавления фрагмента

```

1 // Область с возможностью одновременного добавления фрагментов
2 shared_lock(map_access_mutex);
3  $x_{t_{s,k}} \leftarrow$  local occupancy map pose at  $t_{s,k}$ ;
4 if (Нужен сдвиг  $M$ ) then
5   // Начало критической секции
6   unique_lock(map_access_mutex);
7   if Сдвиг по-прежнему нужен и не произошёл в другом потоке до входа в
      критическую секцию then
8      $M \leftarrow \text{shift\_map}(M, x_{t_{s,k}})$ 
9   else
10     $x_{s,k} \leftarrow x_{s,k} - (x_{t_i} - x_{t_{s,k}})$ ;
11  end
12  // Конец критической секции
13 else
14    $x_{s,k} \leftarrow x_{s,k} - (x_{t_i} - x_{t_{s,k}})$ ;
15 end
16  $\Delta x_s \leftarrow \text{round}(x_{s,k} - x_i)$ ;
17 for  $j \leftarrow \max(0, \Delta x_s)$  to  $\min(N_P; N + \Delta x_s)$  do
18    $t_{prev} \leftarrow 0$ ;
19    $val_{prev} \leftarrow 0$ ;
20    $t_{new} \leftarrow t_{s,k}$ ;
21    $val_{new} \leftarrow P_s[j]$ ;
22   while При CAS( $M[j - \Delta x_s]$ ,  $\{val_{prev}, t_{prev}\}$ ,  $\{val_{new}, t_{new}\}$ ) обмен не произошёл do
23     // Значение  $\{val_{prev}, t_{prev}\}$  актуализируются в случае неудачной
        операции сравнения с обменом
24     if  $t_{prev} < t_{s,k}$  then
25        $t_{new} \leftarrow t_{s,k}$ ;
26        $val_{new} \leftarrow val_{prev} \exp(-a(t_{s,k} - t_{prev})) + P_s[j]$ ;
27     else
28        $t_{new} \leftarrow t_{prev}$ ;
29        $val_{new} \leftarrow val_{prev} + P_s[j] \exp(-a(t_{prev} - t_{s,k}))$ ;
30     end
31   end
32 end

```

Алгоритм 10: Добавление фрагмента (модификация с разделяемыми блокировками): $update_map(P_s, t_{s,k}, x_{s,k})$.

чески невозможна ввиду в первую очередь ограниченного числа потоков, работающих одновременно (в нашем случае их 12, на современных вычислителях, используемых в робототехнике их число варьируется от 8 до 24). Если пренебречь единичными перерасчётами обновляемых значений, сложность такой операции становится $O(\min(N_P, N))$ ($O(\hat{N} + \min(N_P, N))$, если во время выполнения операции произошёл «полноценный» сдвиг карты). Однако константа в этом случае будет заметно больше, чем в предыдущих вариантах из-за использования а) атомарных операций, которые в несколько раз вычислительно дороже неатомарных аналогов и б) `shared_mutex`, который имеет значительно большее время блокировки в сравнении с обычным `mutex`-ом. Поэтому эта реализация становится выгоднее предыдущих вариантов при условии достаточного количества источников, производящих одновременное добавление фрагментов на карту.

Получение карты

При получении карты, копируются все клетки видимой части карты, чтобы избежать состояния гонки с другими операциями. В то же время находится самое позднее значение времени в клетках t_{newest} . После чего создаётся карта «старого» формата – массив чисел с плавающей точкой, куда записываются значения скопированных клеток, приведённых к моменту времени t_{newest} . Сложность такой операции $O(N)$, однако, как и в случае с операцией добавления фрагмента, константа, скрываемая за O , превышает старое значение. Заметим, что эта операция может выполняться одновременно с операциями добавления фрагмента, если допускается частично обновлённое состояние карты. Рассмотрим, например, такой сценарий:

1. Операция получения карты считала значение $M[0]$
2. Источник данных обновил значения клеток $M[0]$, $M[1]$
3. Операция получения карты считала значение $M[1]$

В этом случае только в $M[1]$ будут учтены показания источника данных. Естественно, в последующих вызовах эти показания будут учтены в обеих клетках. Если такая несогласованность данных считается недопустимой, операция получения карты должна быть блокирующей аналогично сдвигу.

4.3.4 Карта проходимости пространства с полным обновлением без блокировок

В рамках последней модификации мы избавляемся от последнего ограничения карты и делаем операцию сдвига также неблокирующей. Для этого мы разделим карту на два равных примыкающих друг к другу фрагмента, добавим к каждому координату, описывающую его положение (как и ранее это будет центр фрагмента) и будем хранить их в умных, разделяемых указателях (рис. 4.7) pM_1, pM_2 . Для нашей реализации важно, чтобы аппаратное обеспечение позволяло работать с такими указателями атомарно. Большинство современных архитектур поддерживают атомарные операции для стандартной C++ реализации `std::shared_ptr`. Примем размер одного фрагмента равным $\hat{N}$.

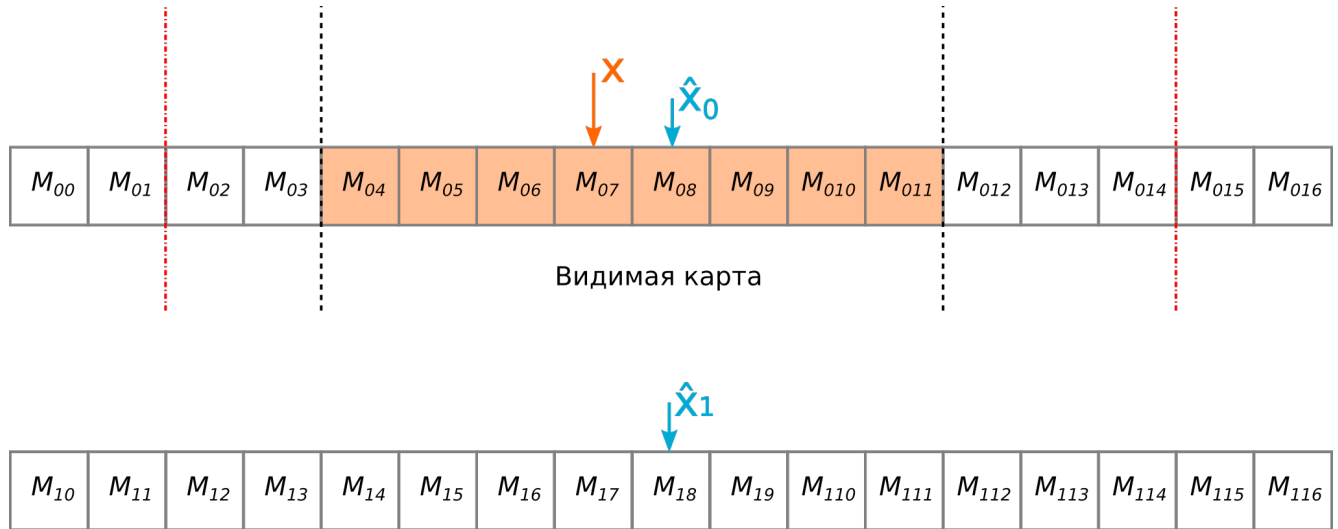


Рисунок 4.7 — Карта с полным обновлением без блокировок.

Сдвиг

Прежде чем приступить к описанию операции, важно сделать следующее замечание: предлагаемая реализация сдвига является неблокирующей относительно добавления фрагмента и получения карты, но в то же время, эта версия не может выполняться одновременно с другими операциями сдвига.

При сдвиге мы определяем положение отображаемой карты после сдвига. Если отображаемая часть карты присутствует целиком на одном фрагменте,

и в результате сдвига отдаляется от его центра более чем на заданный порог Δ_{lim} и при этом так же отдаляется и от второго фрагмента (проще говоря, если дальнейшее движение карты в том же направлении приведет к выходу за границы всех фрагментов), то тот фрагмент, на котором сейчас не находится карта, заменяется на новый: все значения клеток (как и ранее это атомарные пары значения логарифма шансов и времени) инициализируются $\{0,0\}$, а его координата меняется таким образом, чтобы при дальнейшем движении в том же направлении отображаемая часть карты «перешла» на этот фрагмент (см. рис. 4.8). В противном случае, как и ранее, мы просто обновляем положение отображаемой части карты (см. листинг 11). В данном алгоритме важно правильно подобрать Δ_{lim} таким образом, чтобы отображаемая часть карты не могла выйти за границы фрагмента, когда производится замена второго фрагмента (в противном случае это может повлиять на операцию получения карты, выполняемой одновременно со сдвигом).

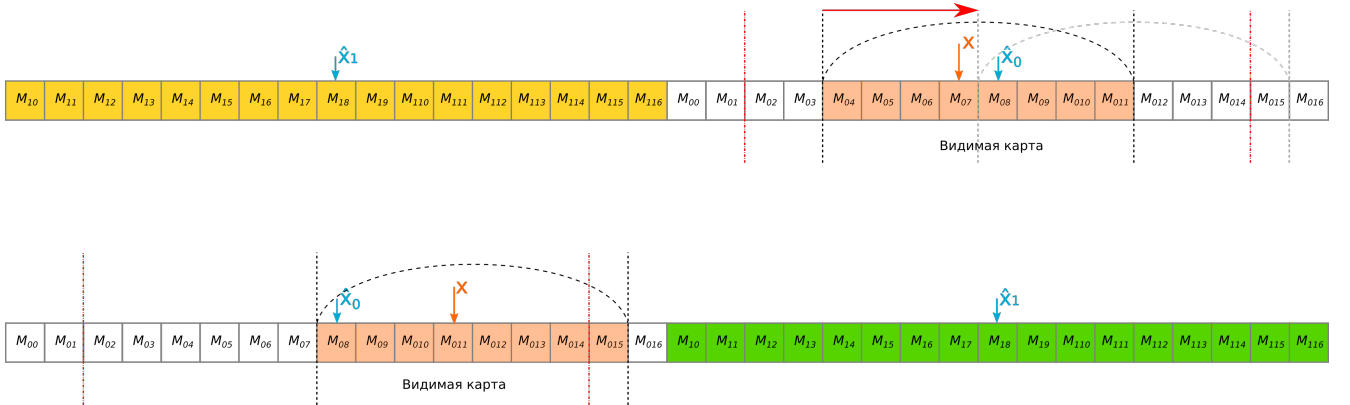


Рисунок 4.8 — Сдвиг карты проходимости пространства с полным обновлением без блокировок.

Вычислительная сложность, как и ранее, $\Theta(1)$, в общем случае, когда обновляется только координата отображаемой части локальной карты проходимости пространства. $\Theta(\hat{N})$, если происходит замена второго фрагмента.

Добавление фрагмента

Добавление фрагмента происходит аналогично предыдущим реализациям за исключением следующих моментов:

Input: M – карта до сдвига;
 x_i – положение M до сдвига;
 x_{i+1} – новое положение карты

Output: M после сдвига

```

1 if  $x_{i+1}$  расположен между центрами  $M_1$  и  $M_2$  then
2   | return  $M$ 
3 end
4  $M_c \leftarrow$  фрагмент ( $M_1$  или  $M_2$ ), на котором находится отображаемая
   карта;
5  $M_o \leftarrow$  оставшийся фрагмент;
6  $\hat{x} \leftarrow$  центр  $M_c$ ;
7 if  $|\hat{x} - x_{i+1}| > \Delta_{lim}$  then
8   |  $M_{new} \leftarrow$  новый фрагмент размера  $\hat{N}$ ;
9   | for  $i \leftarrow 0$  to  $\hat{N}$  do
10  |   |  $M_{new}[i] \leftarrow \{0, 0\}$ ;
11  |   end
12  |  $M_o \leftarrow M_{new}$ ;
13 end
14 return  $M$ 

```

Алгоритм 11: Сдвиг карты (реализация полностью без блокировок):
 $shift_map(M, x_{i+1})$.

1. В самом начале операции происходит копирование указателей на фрагменты M_1, M_2 , чтобы гарантировать их сохранение во время выполнения операции (на случай если одновременно с этой операцией выполняется сдвиг, заменяющий один из фрагментов).
2. Т.к. операция сдвига не может выполняться одновременно с другими операциями сдвига, в добавлении фрагмента эта операция более не происходит. Вместо этого она либо выполняется в отдельном потоке с определённой частотой (гарантирующей, что отображаемая часть карты всегда содержится на одном или обоих фрагментах карты), либо, например, происходит в операции получения локальной карты проходимости пространства (если гарантируется, что эта операция не выполняется одновременно с такой же)
3. Операция применяется последовательно к обоим фрагментам карты

Таким образом вычислительная сложность остаётся $O(\min(N_P, N))$. Реальная константа будет незначительно больше, чем в предыдущей реализации из-за атомарного копирования указателей и применения на двух фрагментах (заметим что последнее может быть выполнено одновременно, но асимптотику операции в общем случае это не изменит).

Получение карты

Аналогично добавлению фрагмента, операция получения почти не отличается от предыдущей реализации, за тем исключением, что первым действием операция копирует указатели, чтобы гарантировать сохранность данных во время работы. Вычислительная сложность, как и раньше $O(N)$.

4.4 Сравнение задержки обновления локальной карты проходимости пространства

Чтобы сравнить представленные модификации между собой мы подготовили их реализации на C++ и синтетические тесты для измерения времени их работы. В рамках этих тестов симулируются следующие сценарии:

- Сдвиг карты на 20 клеток.
- Добавление фрагмента на карту в одном потоке. Размер фрагмента фиксирован и равен 100. Положение фрагмента меняется и берётся из циклической очереди: $-100, -50, -25, -1, 0, 1, 5, 10, 25, 50, 100$. Таким образом моделируются сценарии в которых только часть фрагмента попадает на локальную карту проходимости пространства.
- Добавление фрагмента на карту в нескольких потоках. Тот же сценарий, что и в предыдущем пункте, но запускается в нескольких потоках. Протестированы варианты с разным числом потоков: 1, 5, 9, 13.
- Получение локальной карты проходимости пространства.
- Стандартный цикл «публикации» локальной карты проходимости пространства: В 8 потоках производится сдвиг карты на 10 клеток и

добавления фрагмента размера 100, после завершения работы всех потоков, вызывается операция получения карты, имитирующая передачу карты потребителям.

Все эксперименты повторялись для «видимых областей» разного размера N : 10, 16, 32, 64, 128, 256, 512, 1024, 2048, 4000. Для модификаций, мы брали значение $\hat{N} = 2N$ (в случае с последней модификации с полным обновлением без блокировок полный размер карты с учётом неотображаемых областей составил $4N$). Результаты представленных измерений были получены на ПК с процессором AMD Ryzen 5 5600X с 6 ядрами и тактовой частотой 4,6 ГГц, ОЗУ объёмом 32 Гб.

4.4.1 Сдвиг карты

Результаты замеров представлены на рис. 4.9. Можно видеть линейный рост исходной реализации, как и было показано при обсуждении реализации. Остальные реализации имеют константное (в среднем), время выполнения, поскольку в общем случае требуют только обновления координаты x . Тем не менее константы отличаются между собой. Так, самое быстрое время выполнения у реализации с полным обновлением без блокировок, в которой не используются мьютексы для создания критических областей. Далее идёт расширенная модификация карты, которая использует стандартный `std::mutex`. Наконец, худшее время выполнения среди модификаций имеет версия с разделяемыми блокировками, поскольку здесь используется `std::shared_mutex` манипуляции, с которым более вычислительно интенсивны. Отметим также, что на малых размерах наивная (оригинальная) реализация показывает себя эффективнее модификаций.

4.4.2 Добавление фрагмента в одном потоке

Результаты замеров представлены на рис. 4.10. Как и в прошлом эксперименте, оригинальная реализация имеет линейную сложность. Такую же сложность имеет версия расширенной карты. Такое поведение объясняется необ-

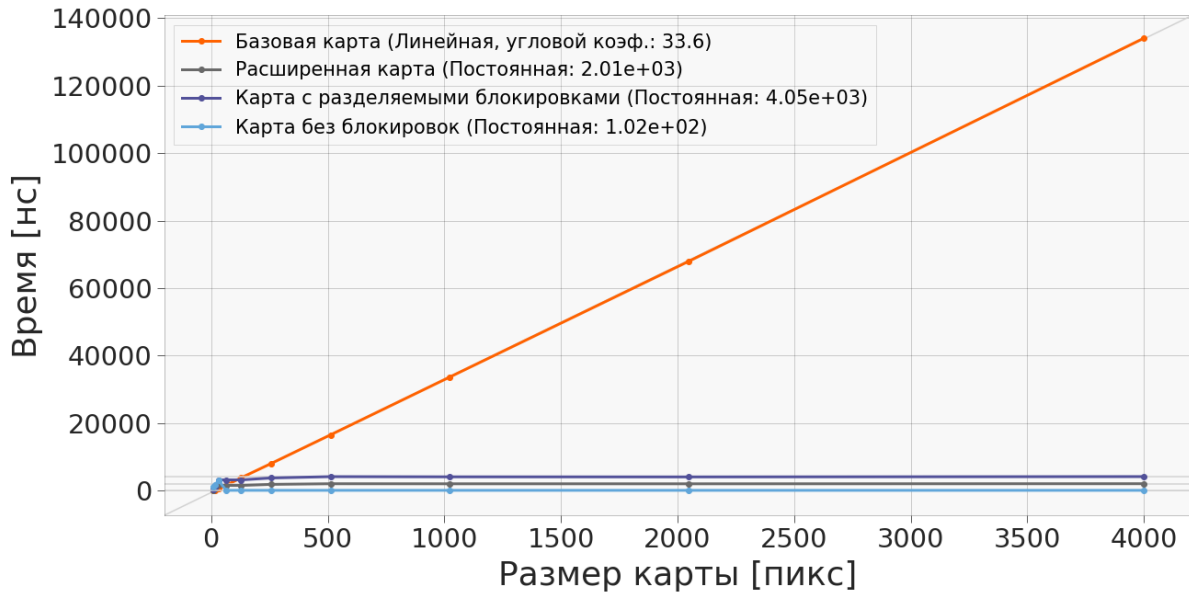


Рисунок 4.9 — График зависимости времени сдвига карты от размера карты.

ходимостью приводить все клетки массива к одному моменту времени. Этим же объясняется большее время, которое требуется на добавление фрагмента в расширенную карту в сравнении с оригинальной версией: ей требуется обновить большее число клеток. В то же время оставшиеся модификации с увеличением карты достигают постоянного числа операций, т.к. не обновляют всю карту целиком, а только те клетки, что пересекаются с фрагментом. При этом у реализации с полным обновлением без блокировок производительность ниже, чем у версии с разделяемой блокировкой. Вероятно, это объясняется необходимостью атомарно копировать указатели на фрагменты карты, обращаться к элементам через указатель и повторению операции на двух массивах (последние два фактора снижают локальность данных и приводят к большему количеству промахов кэша).

4.4.3 Добавление фрагмента в нескольких потоках

Результаты замеров представлены на рис. 4.11, 4.12, 4.13, 4.14. Здесь применимы рассуждения из предыдущего пункта. Можно отметить, что с ростом числа потоков уменьшается размер карты, при котором модификации с

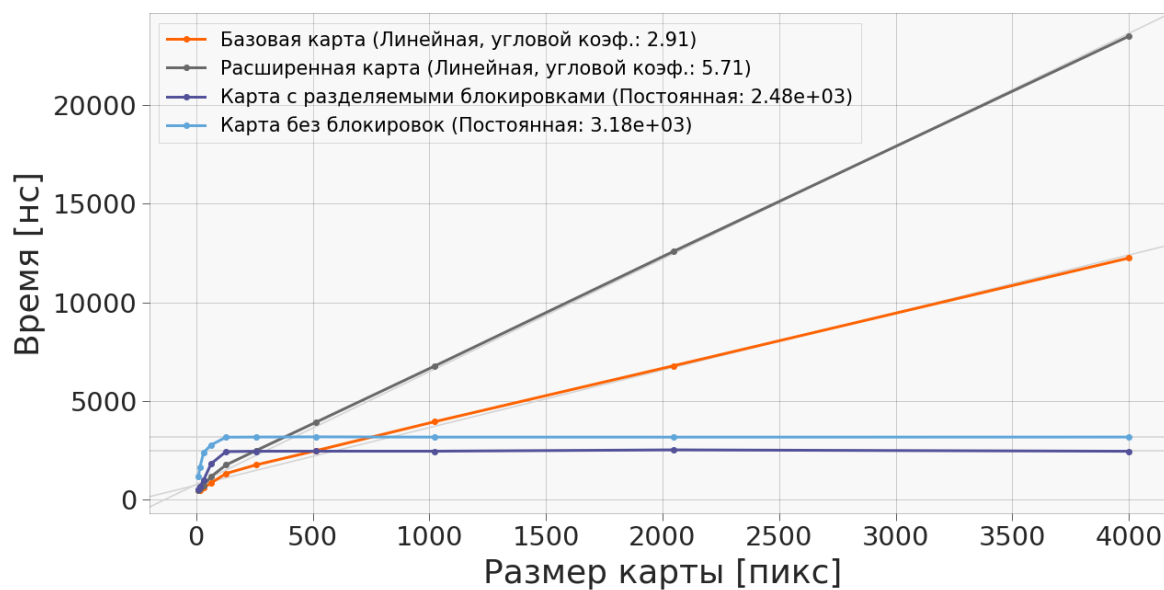


Рисунок 4.10 — График зависимости времени добавления фрагмента на карту от размера карты.

частичным или полным отказом от блокировок становятся вычислительно эффективнее оригинальной реализации.

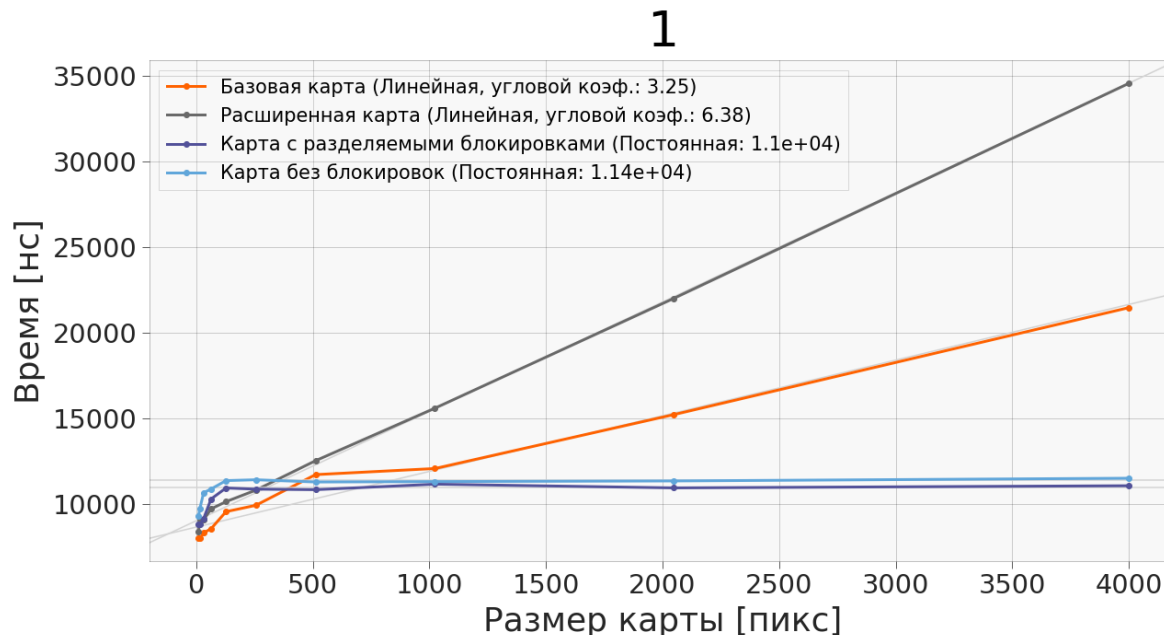


Рисунок 4.11 — График зависимости времени добавления фрагмента на карту в 1 потоке от размера карты.

5

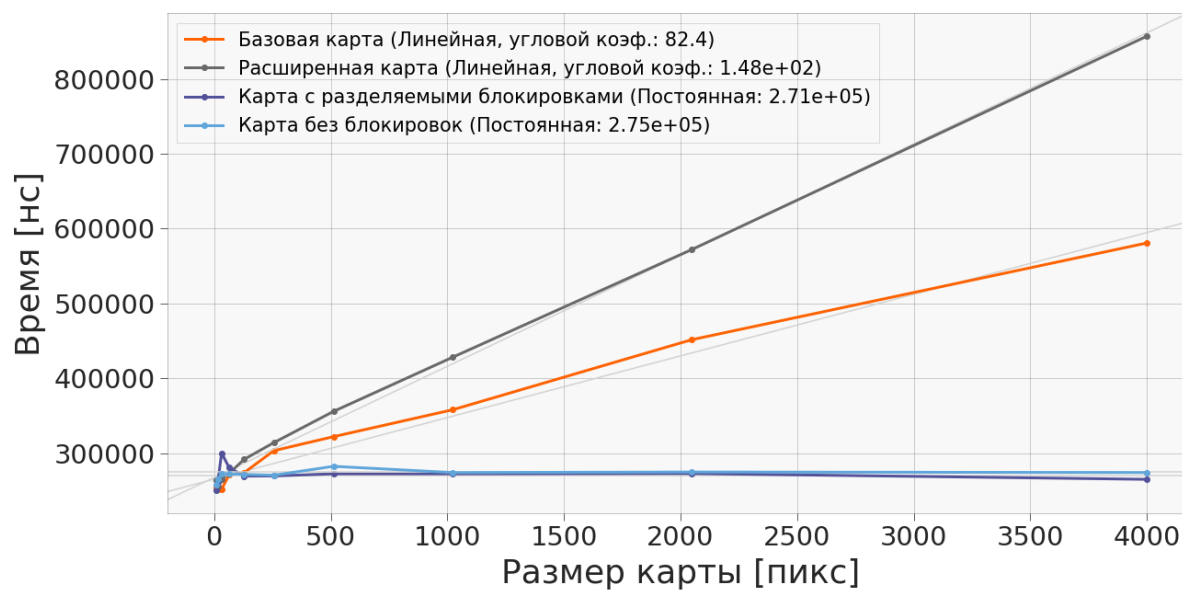


Рисунок 4.12 — График зависимости времени добавления фрагмента на карту в 5 потоках от размера карты.

9

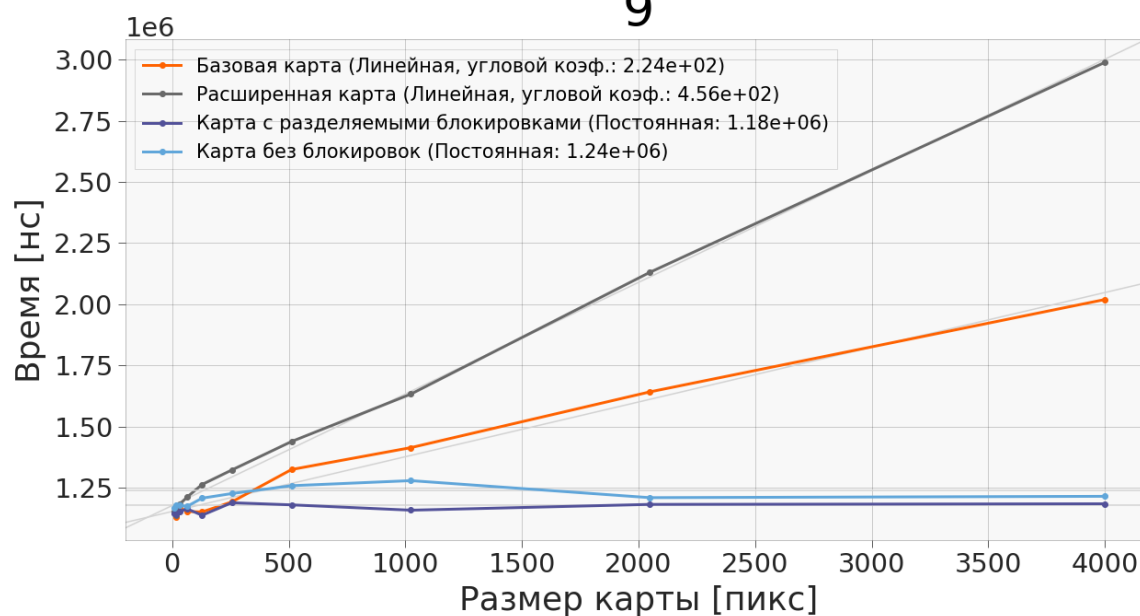


Рисунок 4.13 — График зависимости времени добавления фрагмента на карту в 9 потоках от размера карты.

4.4.4 Получение карты

Результаты замеров представлены на рис. 4.15. Здесь результаты заметно отличаются от предыдущих экспериментов. Время получения оригинальной

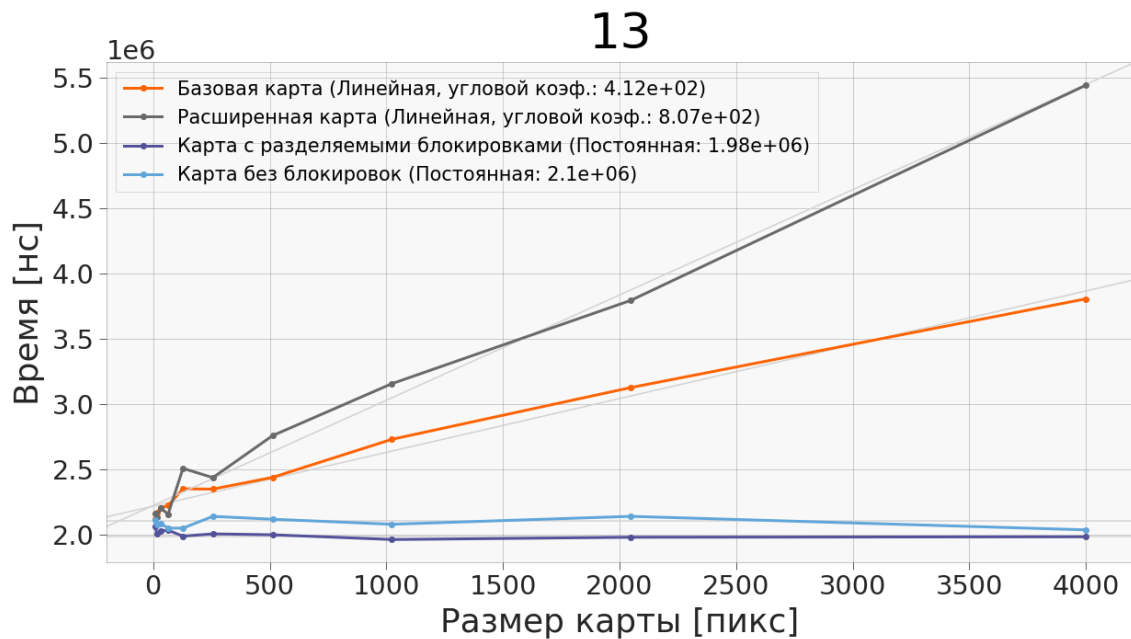


Рисунок 4.14 — График зависимости времени добавления фрагмента на карту в 13 потоках от размера карты.

реализации и расширенной карты с увеличением размера карты возрастает настолько медленно, что кажется константным в сравнении с реализациями с частичным или полным отказом от блокировок (тем не менее оно линейно). В последних же за счёт необходимости атомарно считывать значения клеток, а также приводить их к общему моменту времени работы увеличивается значительно быстрее.

4.4.5 Стандартный цикл «публикации» карты

Наиболее показательная для нас характеристика, показывающая, какая реализация покажет себя лучше всего в реальном сценарии использования. Результаты приведены на рис. 4.16. Можно видеть, что все представленные реализации имеют линейную вычислительную сложность относительно размера карты. Все представленные модификации превосходят по эффективности оригинальную реализацию, причём каждая модификация показывает себя лучше предыдущей (по крайней мере по мере увеличения размера карты). Лучшую производительность показала реализация с полным обновлением без блокировок: её экспериментально установленная константа линейной сложности в 13,89

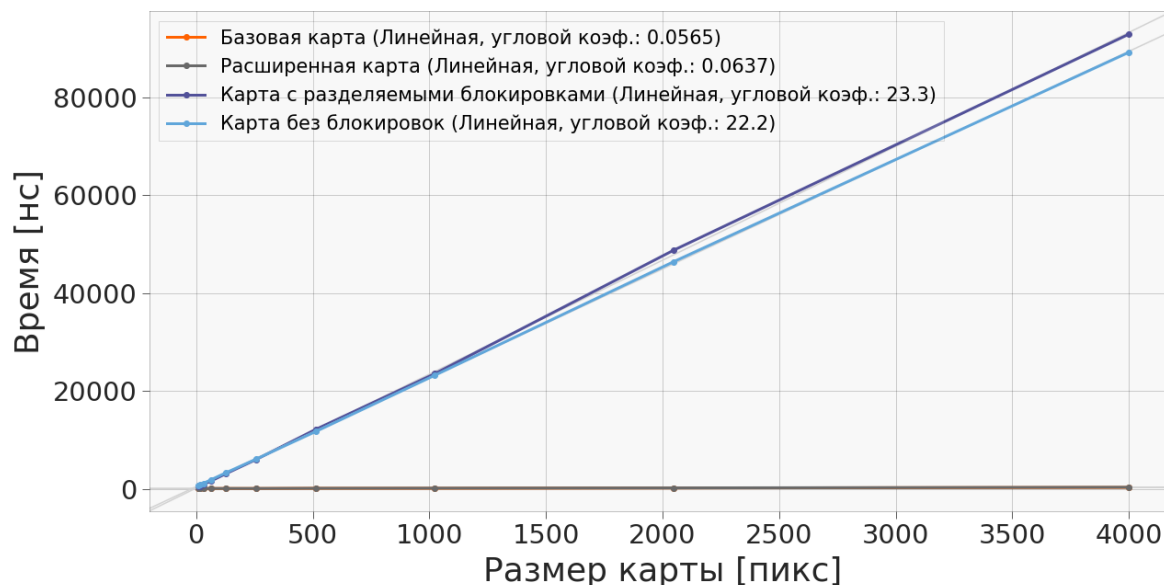


Рисунок 4.15 — График зависимости времени получения карты от её размера. График базовой реализации практически полностью совпадает с расширенной картой и на таком масштабе они не отличимы.

раза меньше, чем у исходной реализации. Далее идёт модификация с разделяемой блокировкой (в 11,52 раза быстрее исходной реализации). Наконец, модификация с расширенной картой, при своей тривиальности показала увеличение производительности в 6,45 раза.

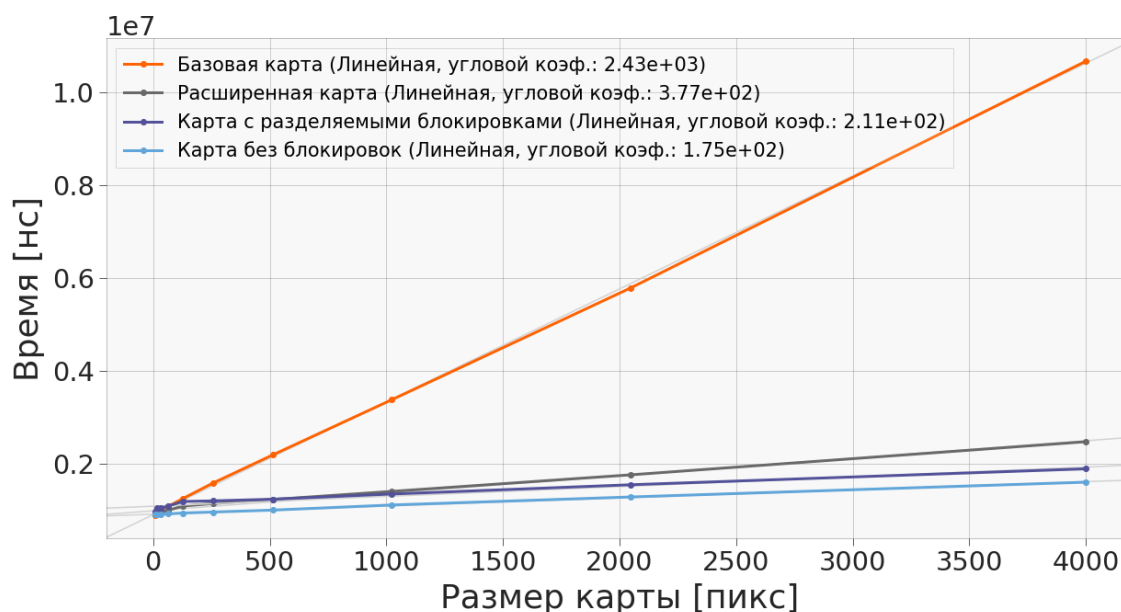


Рисунок 4.16 — График зависимости времени одной итерации «публикации» карты от её размера.

4.5 Реализация финальной модификации для двумерной карты проходимости пространства

Показав работоспособность и лучшую производительность модификации локальной карты проходимости пространства с полным обновлением без блокировок в одномерном случае, представим её расширение для двумерного случая. Как и ранее, мы разбиваем полную карту окружения на несколько, теперь уже двумерных, массивов, состоящих из уже описанных ранее атомарных пар логарифма шансов и времени. Для движения в плоскости потребуется 9 массивов (рис. 4.17). Если видимая область карты пересекает границы, отмеченные штрих-пунктирной линией, создаются новые массивы и заменяют часть предыдущих таким образом, чтобы текущий массив, в котором находится видимая область карты, стал центральным. Важно отметить, что отступ этих линий от краёв массива должен быть подобран так, чтобы за одну итерацию сдвига, видимая область не могла преодолеть расстояние между линией и границей массива. В противном случае если параллельно будет происходить получение карты, то массив, в который «переходит» видимая область, может быть ещё не создан, что приведёт к ошибке выполнения.

Легко видеть, что такая реализация значительно увеличивает размер памяти, требуемой для хранения полученной структуры данных. Кроме того, как и в одномерном случае, операции получения и добавления фрагмента карты должны применяться ко всем 9 массивам. Всё это существенно увеличивает объём занимаемой памяти и коэффициент линейной аппроксимации времени выполнения данных операций. Требуется проведение дополнительных экспериментов для определения условий (средняя скорость движения БТС/средний размер добавляемых фрагментов/количество источников данных и частота их работы/масштаб локальной карты проходимости пространства), при которых такая реализация будет вычислительно эффективнее оригинальной версии.

4.6 Заключение к главе 4

Итак, в данной главе получены следующие результаты:

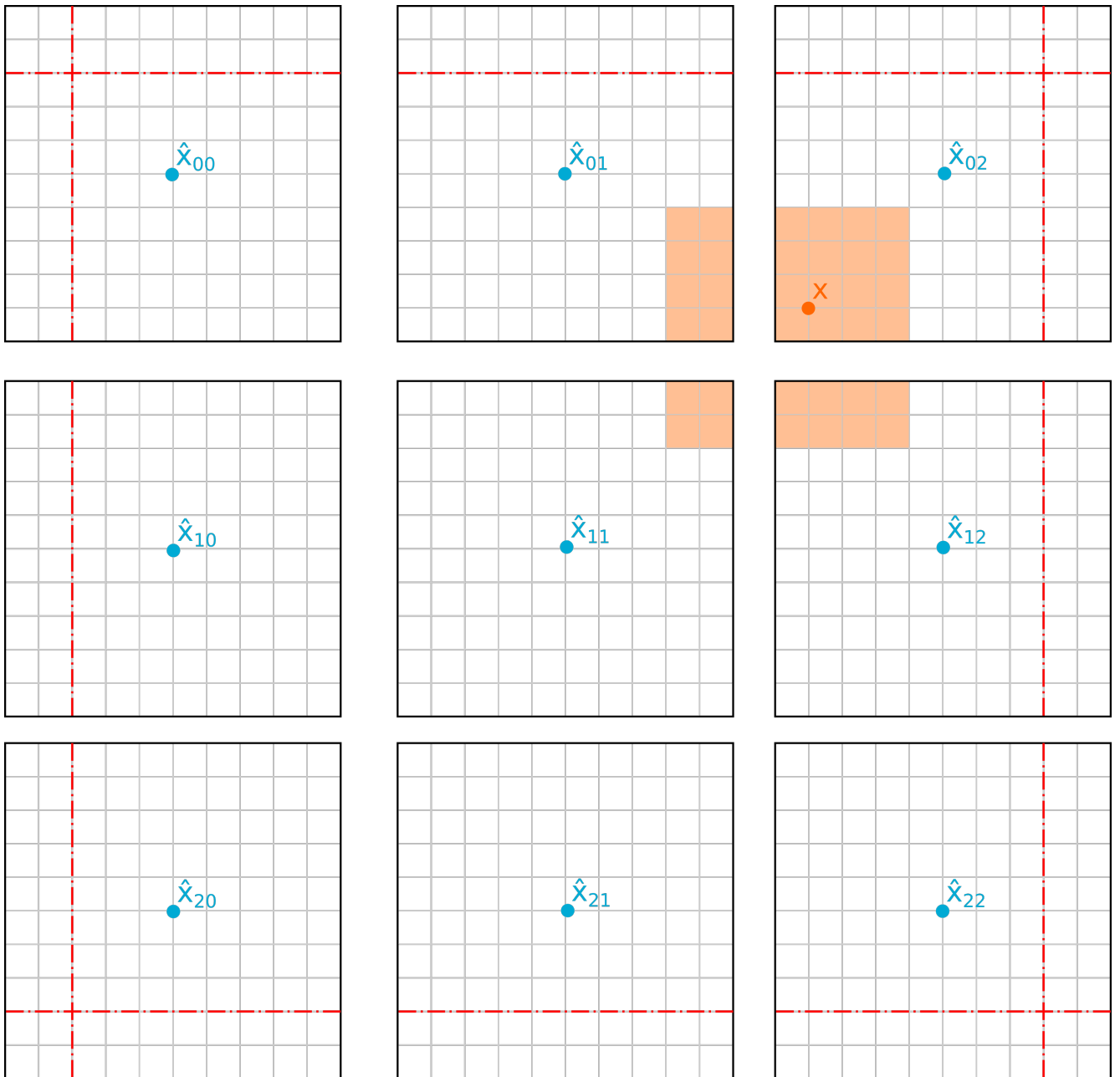


Рисунок 4.17 — Двумерная реализация локальной карты проходимости пространства с полным обновлением без блокировок.

1. предложены три последовательные модификации алгоритма построения локальной карты проходимости пространства, направленные на снижение вычислительной сложности производимых операций;
2. произведена оценка вычислительной сложности предложенных алгоритмов;
3. алгоритмы реализованы в виде программного кода на языке C++ и протестированы для упрощённого сценария одномерной карты проходимости пространства. Полученные при тестировании результаты:

- а) теоретические оценки сложности алгоритмов подтверждены экспериментально;
 - б) лучшую скорость добавления фрагмента на карту в одном и нескольких потоках показала вторая модификация – карта с разделяемыми блокировками;
 - в) лучшую скорость получения карты проходимости показала оригинальная реализация карты;
 - г) лучшую скорость сдвига карты показала последняя модификация – карта проходимости пространства с полным обновлением без блокировок;
 - д) в симуляции одного цикла публикации карты проходимости пространства лучшая производительность продемонстрирована последней модификацией, показав ускорение в 13,89 раза относительно базовой реализации;
4. предложено расширение последней модификации карты проходимости пространства для двумерного пространства.

Перечисленные основные результаты и другие авторские материалы главы опубликованы в работе [117].

Заключение

Основные результаты работы заключаются в следующем:

1. Проведён обзор существующих моделей представления окружающего пространства вокруг БТС. На основании проведённого обзора модель карты проходимости пространства была выбрана в качестве целевой модели, которая будет использована для решения поставленных в работе задач.
2. Предложен алгоритм проекции визуальных детекций с изображения на плоскость движения БТС с использованием модели L-shape. Результаты экспериментов на данных, полученных с реальных БТС, демонстрируют, что L-Shape алгоритм показал наивысший средний коэффициент Жаккара (0,337) среди альтернативных подходов.
3. Предложен альтернативный алгоритм проекции визуальных детекций с изображения на плоскость движения БТС на основе комплексирования данных с камеры и лидаров, установленных на БТС.
4. Предложен алгоритм проекции семантической сегментации проезжей части с изображения на плоскость движения БТС на основе комплексирования данных с камеры и лидаров, установленных на БТС.
5. Разработан программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных от датчиков, установленных на БТС, и их обработчиков.
6. Реализованы три последовательные модификации алгоритма построения локальной карты проходимости пространства, направленные на снижение вычислительной сложности производимых операций. Тестирование на синтетических данных, моделирующих одномерную карту проходимости пространства, показало максимальный прирост в скорости для одного цикла обновления карты проходимости в 13,89 раза относительно базовой реализации для финальной модификации – карты проходимости пространства с полным обновлением без блокировок.
7. Предложено расширение последней модификации карты проходимости пространства для двумерного пространства.

Автор выражает благодарность своему научному руководителю О. С. Шипитько за помощь в постановке задач и обсуждении результатов.

В заключение благодарю коллектив компании «ЭвоКарго». Отдельно хочу поблагодарить К. В. Садовскова, А. П. Рудоманенко, С. А. Скрипичина, П. А. Комиссарову, А. Д. Воронкова за дискуссии, обсуждение результатов и помощь в проведении экспериментов. Благодарю авторов шаблона *Russian-Phd-LaTeX-Dissertation-Template* за помощь в оформлении работы.

Список сокращений и условных обозначений

БТС	Беспилотное транспортное средство
ГПУ	Графическое процессорное устройство
МНК	Метод наименьших квадратов
ОЗУ	Оперативное запоминающее устройство
ПК	Персональный компьютер
ПО	Программное обеспечение
ТС	Транспортное средство
ЦПУ	Центральное процессорное устройство
BEV	Bird's-eye view
CAS	Compare and Swap
CUDA	Compute Unified Device Architecture
FN	False Negative
FP	False Positive
GPR	Gaussian Process Regression
LiDAR	Light Detection and Ranging
PCL	Point Cloud Library
PFC-MSE	PathFinding Cost Mean Squared Error
RADAR	Radio Detection and Ranging
RANSAC	Random Sample Consensus
RGB	Red, Green, Blue
ROS	Robot Operating System
RSS	Residual Sum of Squares
TP	True Positive
UML	Unified Modeling Language

Список литературы

1. Taxonomy and definitions for terms related to on-road motor vehicle automated driving systems [Текст] / S. O.-R. A. V. S. Committee [и др.] // SAE Standard J. — 2014. — Т. 3016. — С. 1.
2. Lane departure warning systems and lane line detection methods based on image processing and semantic segmentation: A review [Текст] / W. Chen [и др.] // Journal of traffic and transportation engineering (English edition). — 2020. — Т. 7, № 6. — С. 748—774.
3. Blind spot monitoring in light vehicles—System performance [Текст] : тех. отч. / G. Forkenbrock [и др.] ; United States. National Highway Traffic Safety Administration. — 2014.
4. Autonomous emergency braking test results [Текст] / W. Hulshof [и др.] // Proceedings of the 23rd International Technical Conference on the Enhanced Safety of Vehicles (ESV). — National Highway Traffic Safety Administration Washington, DC, USA. 2013. — С. 1—13.
5. *Pananurak, W.* Adaptive cruise control for an intelligent vehicle [Текст] / W. Pananurak, S. Thanok, M. Parnichkun // 2008 IEEE International Conference on Robotics and Biomimetics. — IEEE. 2009. — С. 1794—1799.
6. A situation-adaptive lane-keeping support system: Overview of the safelane approach [Текст] / A. Amditis [и др.] // IEEE Transactions on Intelligent Transportation Systems. — 2010. — Т. 11, № 3. — С. 617—629.
7. Research on automatic parking systems based on parking scene recognition [Текст] / S. Ma [и др.] // IEEE Access. — 2017. — Т. 5. — С. 21901—21917.
8. Test procedures traffic jam assist test development considerations [Текст] : тех. отч. / S. J. Rao, G. J. Forkenbrock [и др.] ; United States. Department of Transportation. National Highway Traffic Safety ... — 2019.
9. Waymo public road safety performance data [Текст] / M. Schwall [и др.] // arXiv preprint arXiv:2011.00038. — 2020.
10. *Ingle, S.* Tesla autopilot: semi autonomous driving, an uptick for future autonomy [Текст] / S. Ingle, M. Phute // International Research Journal of Engineering and Technology. — 2016. — Т. 3, № 9. — С. 369—372.

11. 11.2021. — URL: <https://habr.com/ru/companies/yandex/articles/585444/> (дата обр. 30.08.2025).
12. Baidu apollo em motion planner [Текст] / Н. Fan [и др.] // arXiv preprint arXiv:1807.08048. — 2018.
13. *Freedman, I. G.* Autonomous vehicles are cost-effective when used as taxis [Текст] / I. G. Freedman, E. Kim, P. A. Muennig // Injury epidemiology. — 2018. — Т. 5, № 1. — С. 24.
14. Level-5 autonomous driving—Are we there yet? A review of research literature [Текст] / М. А. Khan [и др.] // ACM Computing Surveys (CSUR). — 2022. — Т. 55, № 2. — С. 1—38.
15. An overview of autonomous vehicles sensors and their vulnerability to weather conditions [Текст] / J. Vargas [и др.] // Sensors. — 2021. — Т. 21, № 16. — С. 5397.
16. *Сысоева, С.* Актуальные технологии и применения датчиков автомобильных систем активной безопасности. Часть 6. Радары [Текст] / С. Сысоева // Компоненты и технологии. — 2007. — № 68. — С. 67—76.
17. *Wandinger, U.* Introduction to lidar [Текст] / U. Wandinger // Lidar: range-resolved optical remote sensing of the atmosphere. — Springer, 2005. — С. 1—18.
18. An overview of sensors in Autonomous Vehicles [Текст] / Н. А. Ignatious, М. Khan [и др.] // Procedia Computer Science. — 2022. — Т. 198. — С. 736—741.
19. *Kleeman, L.* Sonar sensing [Текст] / L. Kleeman, R. Kuc // Springer handbook of robotics. — Springer, 2016. — С. 753—782.
20. *Yudin, D.* M3DMap: Object-aware Multimodal 3D Mapping for Dynamic Environments [Текст] / D. Yudin. — 2025. — arXiv: [2508.17044](https://arxiv.org/abs/2508.17044) [cs.CV]. — URL: <https://arxiv.org/abs/2508.17044>.
21. Scene representations for autonomous driving: an approach based on polygonal primitives [Текст] / М. Oliveira [и др.] // Robot 2015: Second Iberian Robotics Conference: Advances in Robotics, Volume 1. — Springer. 2015. — С. 503—515.

22. *Dryanovski, I.* Multi-volume occupancy grids: An efficient probabilistic 3D mapping model for micro aerial vehicles [Текст] / I. Dryanovski, W. Morris, J. Xiao // 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems. — IEEE. 2010. — С. 1553—1559.
23. *Feng, T.* A survey of world models for autonomous driving [Текст] / T. Feng, W. Wang, Y. Yang // arXiv preprint arXiv:2501.11260. — 2025.
24. 3D object detection for autonomous driving: A comprehensive survey [Текст] / J. Mao [и др.] // International Journal of Computer Vision. — 2023. — Т. 131, № 8. — С. 1909—1963.
25. nuscenes: A multimodal dataset for autonomous driving [Текст] / H. Caesar [и др.] // Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. — 2020. — С. 11621—11631.
26. Multi-Object Tracking with Camera-LiDAR Fusion for Autonomous Driving [Текст] / R. Pieroni [и др.]. — 2024. — arXiv: [2403.04112](https://arxiv.org/abs/2403.04112) [[cs.R0](#)]. — URL: <https://arxiv.org/abs/2403.04112>.
27. MCTrack: A Unified 3D Multi-Object Tracking Framework for Autonomous Driving [Текст] / X. Wang [и др.]. — 2024. — arXiv: [2409.16149](https://arxiv.org/abs/2409.16149) [[cs.CV](#)]. — URL: <https://arxiv.org/abs/2409.16149>.
28. *Altché, F.* An LSTM network for highway trajectory prediction [Текст] / F. Altché, A. de La Fortelle // 2017 IEEE 20th international conference on intelligent transportation systems (ITSC). — IEEE. 2017. — С. 353—359.
29. *Thrun, S.* Learning occupancy grid maps with forward sensor models [Текст] / S. Thrun // Autonomous robots. — 2003. — Т. 15, № 2. — С. 111—127.
30. A random finite set approach for dynamic occupancy grid maps with real-time application [Текст] / D. Nuss [и др.] // The International Journal of Robotics Research. — 2018. — Июль. — Т. 37, № 8. — С. 841—866. — URL: <http://dx.doi.org/10.1177/0278364918775523>.
31. *Paskin, M.* Robotic mapping with polygonal random fields [Текст] / M. Paskin, S. Thrun // arXiv preprint arXiv:1207.1399. — 2012.
32. Generating evidential BEV maps in continuous driving space [Текст] / Y. Yuan [и др.] // ISPRS Journal of Photogrammetry and Remote Sensing. — 2023. — Т. 204. — С. 27—41.

33. *Moravec, H.* High resolution maps from wide angle sonar [Текст] / H. Moravec, A. Elfes // Proceedings. 1985 IEEE international conference on robotics and automation. T. 2. — IEEE. 1985. — С. 116—121.
34. *Elfes, A.* Occupancy grids: a probabilistic framework for mobile robot perception and navigation [Текст] : дис. ... канд. / Elfes Alberto. — Ph. D. Thesis. Department of electrical, computer engineering. Carnegie ..., 1989.
35. *Moravec, H. P.* Sensor Fusion in Certainty Grids for Mobile Robots [Текст] / H. P. Moravec // AI Magazine. — 1988. — Т. 9, № 2. — С. 61. — URL: <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/676>.
36. A single LiDAR-based feature fusion indoor localization algorithm [Текст] / Y.-T. Wang [et al.] // Sensors. — 2018. — Vol. 18, no. 4. — P. 1294.
37. *Luo, Z.* A probability occupancy grid based approach for real-time LiDAR ground segmentation [Текст] / Z. Luo, M. V. Mohrenschildt, S. Habibi // IEEE Transactions on Intelligent Transportation Systems. — 2019. — Т. 21, № 3. — С. 998—1010.
38. Data driven prediction architecture for autonomous driving and its application on apollo platform [Текст] / K. Xu [и др.] // 2020 IEEE Intelligent Vehicles Symposium (IV). — IEEE. 2020. — С. 175—181.
39. *Rosenband, D. L.* Inside Waymo's self-driving car: My favorite transistors [Текст] / D. L. Rosenband // 2017 Symposium on VLSI Circuits. — IEEE. 2017. — С. C20—C22.
40. Experience of the ARGO autonomous vehicle [Текст] / M. Bertozzi [и др.] // Enhanced and Synthetic Vision 1998. T. 3364. — SPIE. 1998. — С. 218—229.
41. *Bochare, A.* Camera-Only Bird's Eye View Perception: A Neural Approach to LiDAR-Free Environmental Mapping for Autonomous Vehicles [Текст] / A. Bochare. — 2025. — arXiv: [2505.06113 \[cs.CV\]](https://arxiv.org/abs/2505.06113). — URL: <https://arxiv.org/abs/2505.06113>.
42. Radar/lidar sensor fusion for car-following on highways [Текст] / D. Göhring [и др.] // The 5th International Conference on Automation, Robotics and Applications. — IEEE. 2011. — С. 407—412.
43. Bi-lrfusion: Bi-directional lidar-radar fusion for 3d dynamic object detection [Текст] / Y. Wang [и др.] // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. — 2023. — С. 13394—13403.

44. *Song, J.* Lirafusion: Deep adaptive lidar-radar fusion for 3d object detection [Текст] / J. Song, L. Zhao, K. A. Skinner // 2024 IEEE International Conference on Robotics and Automation (ICRA). — IEEE. 2024. — С. 18250—18257.
45. Lidar-based glass detection for improved occupancy grid mapping [Текст] / H. Tibebu [и др.] // Sensors. — 2021. — Т. 21, № 7. — С. 2263.
46. LIDAR-camera fusion for road detection using fully convolutional neural networks [Текст] / L. Caltagirone [и др.] // Robotics and Autonomous Systems. — 2019. — Т. 111. — С. 125—131.
47. *Liu, H.* Real time object detection using LiDAR and camera fusion for autonomous driving [Текст] / H. Liu, C. Wu, H. Wang // Scientific Reports. — 2023. — Т. 13, № 1. — С. 8056.
48. Radar-camera fusion for object detection and semantic segmentation in autonomous driving: A comprehensive review [Текст] / S. Yao [и др.] // IEEE Transactions on Intelligent Vehicles. — 2023. — Т. 9, № 1. — С. 2094—2128.
49. *Nabati, R.* Centerfusion: Center-based radar and camera fusion for 3d object detection [Текст] / R. Nabati, H. Qi // Proceedings of the IEEE/CVF winter conference on applications of computer vision. — 2021. — С. 1527—1536.
50. *Hasanujjaman, M.* Sensor Fusion in Autonomous Vehicle with Traffic Surveillance Camera System: Detection, Localization, and AI Networking [Текст] / M. Hasanujjaman, M. Z. Chowdhury, Y. M. Jang // Sensors. — 2023. — Т. 23, № 6. — URL: <https://www.mdpi.com/1424-8220/23/6/3335>.
51. Sensor and Sensor Fusion Technology in Autonomous Vehicles: A Review [Текст] / D. J. Yeong [и др.] // Sensors. — 2021. — Т. 21, № 6. — URL: <https://www.mdpi.com/1424-8220/21/6/2140>.
52. Obstacle detection quality as a problem-oriented approach to stereo vision algorithms estimation in road situation analysis [Текст] / A. Smagina [и др.] // Journal of Physics: Conference Series. Т. 1096. — IOP Publishing. 2018. — С. 012035.
53. Deep learning on monocular object pose detection and tracking: A comprehensive overview [Текст] / Z. Fan [и др.] // ACM Computing Surveys. — 2022. — Т. 55, № 4. — С. 1—40.

54. Manipulator grabbing position detection with information fusion of color image and depth image using deep learning [Текст] / D. Jiang [и др.] // Journal of Ambient Intelligence and Humanized Computing. — 2021. — Т. 12, № 12. — С. 10809—10822.
55. *Sasiadek, J. Z.* Sensor fusion [Текст] / J. Z. Sasiadek // Annual Reviews in Control. — 2002. — Т. 26, № 2. — С. 203—228.
56. *Duong, T.* Autonomous navigation in unknown environments with sparse bayesian kernel-based occupancy mapping [Текст] / T. Duong, M. Yip, N. Atanasov // IEEE Transactions on Robotics. — 2022. — Т. 38, № 6. — С. 3694—3712.
57. A Fusion of Dynamic Occupancy Grid Mapping and Multi-object Tracking Based on Lidar and Camera Sensors [Текст] / Y. Wang [и др.] // 2020 3rd International Conference on Unmanned Systems (ICUS). — 2020. — С. 107—112.
58. BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation [Текст] / Z. Liu [и др.]. — 2024. — arXiv: [2205.13542](https://arxiv.org/abs/2205.13542) [cs.CV]. — URL: <https://arxiv.org/abs/2205.13542>.
59. *Durrant-Whyte, H. F.* Sensor models and multisensor integration [Текст] / H. F. Durrant-Whyte // The International Journal of Robotics Research. — 1988. — Т. 7, № 6. — С. 97—113.
60. Object Detection in Autonomous Driving Scenarios Based on an Improved Faster-RCNN [Текст] / Y. Zhou [и др.] // Applied Sciences. — 2021. — Т. 11, № 24. — URL: <https://www.mdpi.com/2076-3417/11/24/11630>.
61. *Bernardin, K.* Evaluating multiple object tracking performance: the clear mot metrics [Текст] / K. Bernardin, R. Stiefelwagen // EURASIP Journal on Image and Video Processing. — 2008. — Т. 2008, № 1. — С. 246309.
62. Performance measures and a data set for multi-target, multi-camera tracking [Текст] / E. Ristani [и др.] // European conference on computer vision. — Springer. 2016. — С. 17—35.
63. Hota: A higher order metric for evaluating multi-object tracking [Текст] / J. Luiten [и др.] // International journal of computer vision. — 2021. — Т. 129, № 2. — С. 548—578.

64. Grid-Centric Traffic Scenario Perception for Autonomous Driving: A Comprehensive Review [Текст] / Y. Shi [и др.] // IEEE Transactions on Neural Networks and Learning Systems. — 2025. — Т. 36, № 7. — С. 11814—11834.
65. UniOcc: A Unified Benchmark for Occupancy Forecasting and Prediction in Autonomous Driving [Текст] / Y. Wang [и др.]. — 2025. — arXiv: [2503.24381](https://arxiv.org/abs/2503.24381) [cs.CV]. — URL: <https://arxiv.org/abs/2503.24381>.
66. *Matsushita, S.* Machine learning-based object movement prediction method using occupancy grid maps from roadside sensor [Текст] / S. Matsushita, O. Alparslan, K. Sato // IEICE Communications Express. — 2025. — Т. 14, № 5. — С. 189—192.
67. Fiery: Future instance prediction in bird’s-eye view from surround monocular cameras [Текст] / A. Hu [и др.] // Proceedings of the IEEE/CVF International Conference on Computer Vision. — 2021. — С. 15273—15282.
68. Video panoptic segmentation [Текст] / D. Kim [и др.] // Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. — 2020. — С. 9859—9868.
69. A navigation-based evaluation metric for probabilistic occupancy grids: Pathfinding cost mean squared error [Текст] / J.-B. Horel [и др.] // 2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC). — IEEE. 2023. — С. 3148—3153.
70. Алгоритмы. Построение и анализ:[пер. с англ.] [Текст] / Т. Кормен [и др.]. — Издательский дом Вильямс, 2009.
71. *Li, P. Z. X.* GMMMap: Memory-Efficient Continuous Occupancy Map Using Gaussian Mixture Model [Текст] / P. Z. X. Li, S. Karaman, V. Sze // IEEE Transactions on Robotics. — 2024. — Т. 40. — С. 1339—1355. — URL: <http://dx.doi.org/10.1109/TRO.2023.3348305>.
72. *O’Callaghan, S.* Continuous occupancy mapping with integral kernels [Текст] / S. O’Callaghan, F. Ramos // Proceedings of the AAAI conference on artificial intelligence. Т. 25. — 2011. — С. 1494—1500.
73. *Vido, C. E.* From grids to continuous occupancy maps through area kernels [Текст] / C. E. Vido, F. Ramos // 2016 IEEE International Conference on Robotics and Automation (ICRA). — 2016. — С. 1043—1048.

74. *Fischler, M. A.* Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography [Текст] / M. A. Fischler, R. C. Bolles // Commun. ACM. — New York, NY, USA, 1981. — Июнь. — Т. 24, № 6. — С. 381—395. — URL: <https://doi.org/10.1145/358669.358692>.
75. *СП 34.13330.2012.* Автомобильные дороги [Текст]. — Введ. 01.07.2013. — М. : Стандартинформ, 2013. — 107 с. — (Свод правил).
76. *Himmelsbach, M.* Fast segmentation of 3D point clouds for ground vehicles [Текст] / M. Himmelsbach, F. v. Hundelshausen, H.-J. Wuensche // 2010 IEEE Intelligent Vehicles Symposium. — IEEE, 06.2010. — С. 560—565. — URL: <http://dx.doi.org/10.1109/IVS.2010.5548059>.
77. *Zermas, D.* Fast segmentation of 3D point clouds: A paradigm on LiDAR data for autonomous vehicle applications [Текст] / D. Zermas, I. Izzat, N. Papanikolopoulos // 2017 IEEE International Conference on Robotics and Automation (ICRA). — IEEE, 05.2017. — С. 5067—5073. — URL: <http://dx.doi.org/10.1109/ICRA.2017.7989591>.
78. *Lim, H.* Patchwork: Concentric Zone-Based Region-Wise Ground Segmentation With Ground Likelihood Estimation Using a 3D LiDAR Sensor [Текст] / H. Lim, M. Oh, H. Myung // IEEE Robotics and Automation Letters. — 2021. — Окт. — Т. 6, № 4. — С. 6458—6465. — URL: <http://dx.doi.org/10.1109/LRA.2021.3093009>.
79. *Rasmussen, C. E.* Gaussian processes in machine learning [Текст] / C. E. Rasmussen // Summer school on machine learning. — Springer, 2003. — С. 63—71.
80. Gaussian-Process-Based Real-Time Ground Segmentation for Autonomous Land Vehicles [Текст] / T. Chen [и др.] // Journal of Intelligent & Robotic Systems. — 2013. — Т. 76, № 3/4. — С. 563—582. — URL: <http://dx.doi.org/10.1007/s10846-013-9889-4>.
81. *Phillion, J.* Lift, Splat, Shoot: Encoding Images from Arbitrary Camera Rigs by Implicitly Unprojecting to 3D [Текст] / J. Phillion, S. Fidler // Computer Vision – ECCV 2020. — Springer International Publishing, 2020. — С. 194—210. — URL: http://dx.doi.org/10.1007/978-3-030-58568-6_12.

82. BEVFormer: Learning Bird's-Eye-View Representation from Multi-camera Images via Spatiotemporal Transformers [Текст] / Z. Li [и др.] // Computer Vision – ECCV 2022. — Springer Nature Switzerland, 2022. — С. 1–18. — URL: http://dx.doi.org/10.1007/978-3-031-20077-9_1.
83. Bevfusion: Multi-task multi-sensor fusion with unified bird's-eye view representation [Текст] / Z. Liu [и др.] // 2023 IEEE international conference on robotics and automation (ICRA). — IEEE. 2023. — С. 2774–2781.
84. *Petrovskaya, A.* Model based vehicle tracking for autonomous driving in urban environments [Текст] / A. Petrovskaya, S. Thrun // Proceedings of robotics: science and systems IV, Zurich, Switzerland. — 2008. — Т. 34.
85. *Arnon, D. S.* A linear time algorithm for the minimum area rectangle enclosing a convex polygon [Текст] / D. S. Arnon, J. P. Giesermann. — 1983.
86. Optimal Vehicle Pose Estimation Network Based on Time Series and Spatial Tightness with 3D LiDARs [Текст] / H. Wang [и др.] // Remote Sensing. — 2021. — Окт. — Т. 13. — С. 4123.
87. Efficient L-shape fitting for vehicle detection using laser scanners [Текст] / X. Zhang [и др.] // 2017 IEEE Intelligent Vehicles Symposium (IV). — IEEE. 2017. — С. 54–59.
88. The Marathon 2: A Navigation System [Текст] / S. Macenski [и др.] // 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). — Las Vegas, NV, USA : IEEE, 10.2020.
89. *Fankhauser, P.* A Universal Grid Map Library: Implementation and Use Case for Rough Terrain Navigation [Текст] / P. Fankhauser, M. Hutter // Robot Operating System (ROS). — Springer International Publishing, 2016. — С. 99–120. — URL: http://dx.doi.org/10.1007/978-3-319-26054-9_5.
90. OctoMap: an efficient probabilistic 3D mapping framework based on octrees [Текст] / A. Hornung [и др.] // Autonomous Robots. — 2013. — Февр. — Т. 34, № 3. — С. 189–206. — URL: <http://dx.doi.org/10.1007/s10514-012-9321-0>.
91. Efficient occupancy grid computation on the GPU with lidar and radar for road boundary detection [Текст] / F. Himm [и др.] // 2010 IEEE Intelligent Vehicles Symposium. — IEEE, 06.2010. — С. 1006–1013. — URL: <http://dx.doi.org/10.1109/IVS.2010.5548091>.

92. *Michael, M. M.* Simple, fast, and practical non-blocking and blocking concurrent queue algorithms [Текст] / M. M. Michael, M. L. Scott // Proceedings of the fifteenth annual ACM symposium on Principles of distributed computing - PODC '96. — ACM Press, 1996. — С. 267—275. — (PODC '96). — URL: <http://dx.doi.org/10.1145/248052.248106>.
93. *Herlihy, M.* The Art of Multiprocessor Programming [Текст] / M. Herlihy, N. Shavit. — Morgan Kaufmann, 04.2008.
94. Yolop: You only look once for panoptic driving perception [Текст] / D. Wu [и др.] // Machine Intelligence Research. — 2022. — Т. 19, № 6. — С. 550—562.
95. *Kuhn, H. W.* The Hungarian method for the assignment problem [Текст] / H. W. Kuhn // Naval research logistics quarterly. — 1955. — Т. 2, № 1/2. — С. 83—97.
96. *ГОСТ 21.701-2013.* Система проектной документации для строительства. Правила выполнения рабочей документации автомобильных дорог [Текст]. — Введ. 01.01.2015. — М. : Стандартинформ, 2014. — 32 с. — (Межгосударственный стандарт).
97. *NVIDIA.* CUDA, release: 10.2.89 [Текст] / NVIDIA, P. Vingelmann, F. H. Fitzek. — 2020. — URL: <https://developer.nvidia.com/cuda-toolkit>.
98. Fast euclidean cluster extraction using gpus [Текст] / A. Nguyen [и др.] // Journal of Robotics and Mechatronics. — 2020. — Т. 32, № 3. — С. 548—560.
99. *Stein, M. L.* Interpolation of spatial data: some theory for kriging [Текст] / M. L. Stein. — Springer Science & Business Media, 1999.
100. *Jones, A.* The matérn class of covariance functions [Текст] / A. Jones. — 07.2021. — URL: <https://andrewcharlesjones.github.io/journal/maternal-kernels.html> (дата обр. 15.04.2025).
101. ROS: an open-source Robot Operating System [Текст] / M. Quigley [и др.] // ICRA workshop on open source software. Т. 3. — Kobe. 2009. — С. 5.
102. *Kyöstilä, S.* Tracy: A debugger and system analyzer for cross-platform graphics development [Текст] / S. Kyöstilä, K. J. Kangas, K. Pulli // Proceedings of the 23rd ACM SIGGRAPH/EUROGRAPHICS symposium on Graphics hardware. — 2008. — С. 1—11.

103. *Moller, K.* Overcoming Blind Spots: Occlusion Considerations for Improved Autonomous Driving Safety [Текст] / K. Moller, R. Trauth, J. Betz // 2024 IEEE Intelligent Vehicles Symposium (IV). — 2024. — С. 819—826.
104. *Rennich, S.* Cuda C/C++ streams and concurrency [Текст] / S. Rennich. — URL: <https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf> (дата обр. 17.08.2025).
105. 08.2025. — URL: <https://docs.nvidia.com/nsight-systems/index.html> (дата обр. 17.08.2025).
106. *Chekanov, M.* L-shape fitting algorithm for 3D object detection in bird's-eye-view in an autonomous driving system [Текст] / M. Chekanov, O. Shipitko. — 04.2024. — URL: <http://dx.doi.org/10.1117/12.3023477>.
107. *М. О. Чеканов С. В. Кудряшова, О. С. Ш.* Трехмерная детекция объектов на основе L-shape модели в автономных системах движения [Текст] / О. С. Ш. М. О. Чеканов С. В. Кудряшова // Сенсорные системы. — 2025. — Т. 39, № 1. — С. 66—78.
108. *Chekanov, M.* Camera and lidar sensor fusion for projection of 2d detections into 3d coordinate space [Текст] / M. Chekanov, O. Shipitko, D. Nikolaev // ECMS 2026 Proceedings edited by Filippo Sanfilippo, Florenc Demrozi, Fabio Sgarbossa, Mohammad Poursina. — ECMS, 06.2026. — С. 259—266. — URL: <http://dx.doi.org/10.7148/2026-0259>.
109. *Воронков, А. Д.* Алгоритм проекции семантической сегментации проезжей части на карту проходимости пространства с использованием лидарной оценки геометрии поверхности [Текст] / А. Д. Воронков, А. П. Рудоманенко, М. О. Чеканов // Информационные процессы. — 2026. — Т. 26, № 2. — С. 511—530.
110. *Bradski, G.* The OpenCV Library [Текст] / G. Bradski // Dr. Dobb's Journal of Software Tools. — 2000.
111. *Rusu, R. B.* 3D is here: Point Cloud Library (PCL) [Текст] / R. B. Rusu, S. Cousins // IEEE International Conference on Robotics and Automation (ICRA). — Shanghai, China : IEEE, 05.2011.
112. *Woodall, W.* pcl-conversions [Текст] / W. Woodall. — URL: https://wiki.ros.org/pcl_conversions (дата обр. 17.08.2025).

113. *Martin, R. C.* The dependency inversion principle [Текст] / R. C. Martin // C Report. — 1996. — Т. 8, № 6. — С. 61—66.
114. *Marder-Eppstein, E.* tf2-ros [Текст] / E. Marder-Eppstein, W. Meeussen. — URL: https://wiki.ros.org/tf2_ros (дата обр. 17.08.2025).
115. Head First Design Patterns: A Brain-Friendly Guide [Текст] / E. Freeman [и др.]. — "O'Reilly Media, Inc.", 2004.
116. *Fog, A.* Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD and VIA CPUs [Текст] / A. Fog // Agner Fog Homepage. — 2014.
117. *Чеканов, М. О.* Многопоточные неблокирующие алгоритмы построения карты проходимости пространства [Текст] / М. О. Чеканов // Информационные технологии и вычислительные системы. — 2026. — № 2. — С. 40—51.

Список рисунков

1.1	Пример эталонной карты и предсказанных карт с одинаковым качеством согласно метрикам, не учитывающим контекст решаемой задачи.	26
2.1	Наивный подход к проецированию ограничивающей рамки в 2D на вид сверху.	42
2.2	Неверная проекция грузовика. Зона слева от грузовика воспринимается, как часть объекта, и мешает объехать его.	42
2.3	Проекция двумерной ограничивающей рамки на вид сверху с учётом сегментации проезжей части.	43
2.4	Примеры сцен из набора данных, использованного для сравнения алгоритмов.	46
2.5	Пример неправильно построенной ограничивающей рамки.	48
2.6	Примеры работы сравниваемых алгоритмов построения ограничивающих рамок.	50
2.7	График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий.	52
2.8	Примеры оформления поперечного профиля из ГОСТ 21.701-2013 [96].	53
2.9	Демонстрация работы алгоритма 1 на данных, полученных с БТС.	54
2.10	Сравнение регрессии на основе гауссовских процессов с использованием в качестве ядра радиальной базисной функции и ядра Матерна [100].	57
2.11	Пример работы алгоритма разделения облака точек земли и препятствий.	58
2.12	График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий для алгоритма комплексирования и алгоритмов, не использующих комплексирование.	62
2.13	Среднее время выполнения этапов алгоритма 1	63
2.14	Демонстрация влияния окклюзии на безопасность движения БТС [103].	64

2.15	Пример семантической сегментации проезжей части моделью YOLOP на изображении с камеры БТС.	65
2.16	Пример заслонённой области свободной для проезда.	65
2.17	Иллюстрация алгоритма проецирования сегментации проезжей части на вид сверху. Для облегчения восприятия проецируемая сетка представлена одним рядом.	67
2.18	Профилирование в Nsight Systems изменившейся части программы, реализующей алгоритм 2, при добавлении дополнительных точек для аппроксимации.	69
2.19	График полного времени работы алгоритма 2 с дополнительными тестовыми точками при последовательном и параллельном вычислении.	70
2.20	Эталонные карты проходимости пространства и карты, построенные сравниваемыми алгоритмами на тестовых данных. . . .	77
3.1	Диаграмма компонентов программного комплекса Occupancy Map Fusion на языке UML(Unified Modeling Language).	83
3.2	Диаграмма деятельности программного комплекса Occupancy Map Fusion.	84
3.3	Иллюстрация поля зрения, занимаемого препятствием.	87
4.1	Наблюдаемые задержки при обновлении карты проходимости пространства	100
4.2	Расположение локальной карты проходимости и её фрагмента от одного из источников данных относительно БТС и глобальной системы координат.	101
4.3	Демонстрация формирования очереди фрагментов. Горизонтальные отрезки обозначают время формирования фрагмента каждым источником: начало отрезка соответствует времени регистрации данных $t_{s,k}$, конец отрезка соответствует времени завершения формирования фрагмента и моменту добавления его в очередь фрагментов. Внизу иллюстрации продемонстрирован итоговый порядок фрагментов в очереди: фрагменты извлекаются из очереди в порядке справа налево.	103
4.4	Иллюстрация алгоритма добавления фрагмента от источника данных s на локальную карту проходимости пространства.	105

4.5	Оригинальная реализация локальной карты проходимости пространства.	106
4.6	Карта с подготовленными окрестностями.	109
4.7	Карта с полным обновлением без блокировок.	114
4.8	Сдвиг карты проходимости пространства с полным обновлением без блокировок.	115
4.9	График зависимости времени сдвига карты от размера карты.	119
4.10	График зависимости времени добавления фрагмента на карту от размера карты.	120
4.11	График зависимости времени добавления фрагмента на карту в 1 потоке от размера карты.	120
4.12	График зависимости времени добавления фрагмента на карту в 5 потоках от размера карты.	121
4.13	График зависимости времени добавления фрагмента на карту в 9 потоках от размера карты.	121
4.14	График зависимости времени добавления фрагмента на карту в 13 потоках от размера карты.	122
4.15	График зависимости времени получения карты от её размера. График базовой реализации практически полностью совпадает с расширенной картой и на таком масштабе они не отличимы.	123
4.16	График зависимости времени одной итерации «публикации» карты от её размера.	123
4.17	Двумерная реализация локальной карты проходимости пространства с полным обновлением без блокировок.	125

Список таблиц

1	Сравнение характеристик датчиков, используемых в колёсной робототехнике	16
2	Изменение предельного числа одновременно учитываемых источников данных с ростом рабочей скорости БТС	36
3	Статистики полученных коэффициента Жаккара для сравниваемых алгоритмов. Наилучшие значения выделены полужирным.	51
4	Статистики полученных коэффициентов Жаккара для предложенного алгоритма и алгоритмов, не использующих комплексирование. Наилучшие значения выделены полужирным. . .	61
5	Сценарии, представленные в наборе тестовых данных.	73
6	Значения метрики PFC-MSE (меньше – лучше) для предложенного и базового лидарного алгоритмов. Наилучшие значения выделены полужирным.	75
7	Параметры сравниваемых алгоритмов Б, П, А1, А2 (обозначения введены выше) и усреднённые по пяти сценариям значения метрик. Стрелка ↓ означает, что меньшие значения метрики соответствуют лучшему качеству, ↑ — большие. Наилучшие значения по каждой метрике выделены полужирным.	75

Приложение А

Вывод решения задачи вписывания в L-Shape модель

В разделе 2.1 задача вписывания набора точек $p_i = (x_i, y_i)$, $i \in \{1, \dots, N\}$, в ориентированную ограничивающую рамку сведена к поиску двух перпендикулярных прямых $l : y = kx + d_1$ и $l' : x = -ky + d_2$, минимизирующих сумму квадратов отклонений вписываемых точек от соответствующей прямой. Согласно L-Shape модели, существует номер m , разделяющий точки между прямыми:

$$\begin{aligned} \exists m \in \{1, \dots, N-1\} \hookrightarrow & \forall i \in \{1, \dots, m\}, p_i = (x_i, y_i) \in l; \\ & \forall i \in \{m+1, \dots, N\}, p_i = (x_i, y_i) \in l' \end{aligned}$$

При фиксированном m задача минимизации записывается в виде:

$$\operatorname{argmin}_{k, d_1, d_2} \left(\begin{pmatrix} x_1 & 1 & 0 \\ x_2 & 1 & 0 \\ \vdots & \vdots & \vdots \\ x_m & 1 & 0 \\ -y_{m+1} & 0 & 1 \\ \vdots & \vdots & \vdots \\ -y_N & 0 & 1 \end{pmatrix} \begin{pmatrix} k \\ d_1 \\ d_2 \end{pmatrix} - \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \\ x_{m+1} \\ \vdots \\ x_N \end{pmatrix} \right)^2$$

Обозначим через $\hat{x}$ и $\hat{y}$ столбцы координат точек, отнесённых к прямой l , а через $\tilde{y}$ и $\tilde{x}$ – столбцы координат точек, отнесённых к прямой l' . Тогда выражение переписывается как

$$\operatorname{argmin}_a \left(\begin{pmatrix} \hat{x} & 1 & 0 \\ -\hat{y} & 0 & 1 \end{pmatrix} a - \begin{pmatrix} \tilde{y} \\ \tilde{x} \end{pmatrix} \right)^2 = \operatorname{argmin}_a (Xa - y)^2,$$

где $a = (k, d_1, d_2)^T$. Решение этой задачи и соответствующее ему значение суммы квадратов отклонений даются формулами (2.1) и (2.2) основного текста.

Покажем, что при переходе от m к $m+1$ входящие в эти формулы матрицы пересчитываются за фиксированное, не зависящее от N , число операций. Для удобства чтения введём обозначение $\bar{m} = N - m$. Выпишем промежуточные матрицы в явном виде:

$$\begin{aligned}
X^T X &= \begin{pmatrix} \sum \hat{x}^2 + \sum \hat{y}^2 & \sum \hat{x} & -\sum \hat{y} \\ \sum \hat{x} & m & 0 \\ -\sum \hat{y} & 0 & \bar{m} \end{pmatrix} \\
\Delta_{X^T X} &= m\bar{m}(\sum \hat{x}^2 + \sum \hat{y}^2) - m\left(\sum \hat{y}\right)^2 - \bar{m}\left(\sum \hat{x}\right)^2 \\
(X^T X)^{-1} &= \frac{1}{\Delta_{X^T X}} \\
&\begin{pmatrix} m\bar{m} & -\bar{m}\sum \hat{x} & m\sum \hat{y} \\ -\bar{m}\sum \hat{x} & m(\sum \hat{x}^2 + \sum \hat{y}^2) - (\sum \hat{y})^2 & -\sum \hat{x}\sum \hat{y} \\ m\sum \hat{y} & -\sum \hat{x}\sum \hat{y} & m(\sum \hat{x}^2 + \sum \hat{y}^2) - (\sum \hat{x})^2 \end{pmatrix} \\
X^T y &= \begin{pmatrix} \hat{x}^T \tilde{y} - \hat{y}^T \tilde{x} \\ \sum \tilde{y} \\ \sum \tilde{x} \end{pmatrix}
\end{aligned}$$

Все элементы этих матриц выражаются через семь величин: $\sum \hat{x}^2 + \sum \hat{y}^2$, $\sum \hat{x}$, $\sum \hat{y}$, $\sum \tilde{x}$, $\sum \tilde{y}$, $\hat{x}^T \tilde{y}$, $\hat{y}^T \tilde{x}$, а также через m и $\bar{m}$. Если вычислить их один раз для $m = 1$ (за $\Theta(N)$ операций), то при «перемещении» очередной точки (x, y) с прямой l' на прямую l перечисленные величины пересчитываются так:

$$\begin{aligned}
\sum \hat{x}^2 + \sum \hat{y}^2 + &= x^2 - y^2 \\
\sum \hat{x} + &= x \\
\sum \hat{y} - &= y \\
\sum \tilde{x} - &= x \\
\sum \tilde{y} + &= y \\
\hat{x}^T \tilde{y} + &= xy \\
\hat{y}^T \tilde{x} - &= xy
\end{aligned}$$

Таким образом, каждый шаг перебора m требует фиксированного числа операций, и итоговая временная сложность L-Shape алгоритма составляет $\Theta(N) + (N - 1)\Theta(1) = \Theta(N)$ вместо $\Theta(N^2)$ у прямой реализации.

Приложение Б

Свидетельства о государственной регистрации программ для ЭВМ

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2025617212

Программное обеспечение «Система восприятия N1 V3»

Правообладатель: *Общество с ограниченной
ответственностью «ЭвоКарго» (RU)*Авторы: *см. на обороте*

Заявка № 2025614166

Дата поступления 03 марта 2025 г.

Дата государственной регистрации

в Реестре программ для ЭВМ 24 марта 2025 г.

Руководитель Федеральной службы
по интеллектуальной собственностидокумент подписан электронной подписью
Сертификат 0692e7c1af300b54f2401670bca2026
Владелец **Зубов Юрий Сергеевич**
Действителен с 10.07.2024 по 03.10.2025

Ю.С. Зубов

Авторы: *Бабатаев Ильдар Хамзаевич (RU), Бойко Дмитрий Викторович (RU), Боровикова Полина Александровна (RU), Валуков Александр Дмитриевич (RU), Зайцева Елена Александровна (RU), Калинин Валерий Геннадьевич (RU), Козловцев Константин Анатольевич (RU), Комиссарова Полина Александровна (RU), Ларин Александр Витальевич (RU), Лысухин Даниил Дмитриевич (RU), Мозжухин Евгений Игоревич (RU), Осиповский Роман Владиславович (RU), Погодина Кристина Валерьевна (RU), Протасов Саян Константинович (RU), Прошкина Екатерина Юрьевна (RU), Рудоманенко Антон Павлович (RU), Скоробогатова Ольга Александровна (RU), Скрипичин Сергей Анатольевич (RU), Скублов Алексей Сергеевич (RU), Фокин Иван Игоревич (RU), Чалков Артём Анатольевич (RU), Чеканов Михаил Олегович (RU)*

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2025669744

**Программное обеспечение «Способ построения карты
проходимости в окрестности
высокоавтоматизированного беспилотного грузового
транспортного средства»**

Правообладатель: **Общество с ограниченной
ответственностью «ЭвоКарго» (RU)**

Авторы: **Чеканов Михаил Олегович (RU), Рудоманенко
Антон Павлович (RU), Садовсков Кирилл Викторович
(RU)**



Заявка № **2025668746**

Дата поступления **25 июня 2025 г.**

Дата государственной регистрации

в Реестре программ для ЭВМ **30 июля 2025 г.**

*Руководитель Федеральной службы
по интеллектуальной собственности*

документ подписан электронной подписью

Сертификат 0692e701af300b54f2401670bca2026

Владелец **Зубов Юрий Сергеевич**
Действителен с 10.07.2024 по 03.10.2025

Ю.С. Зубов

Приложение В

Патенты

РОССИЙСКАЯ ФЕДЕРАЦИЯ

**ПАТЕНТ**

НА ИЗОБРЕТЕНИЕ

№ 2853062

**Способ построения карты проходимости в окрестности
высокоавтоматизированного беспилотного грузового
транспортного средства**

Патентообладатель: *Общество с ограниченной
ответственностью "Эвокарго" (RU)*

Авторы: *Чеканов Михаил Олегович (RU), Садовсков Кирилл
Викторович (RU), Рудоманенко Антон Павлович (RU)*

Заявка № **2025112125**

Приоритет изобретения **12 мая 2025 г.**

Дата государственной регистрации

в Государственном реестре изобретений

Российской Федерации **18 декабря 2025 г.**

Срок действия исключительного права

на изобретение истекает **12 мая 2045 г.**



*Руководитель Федеральной службы
по интеллектуальной собственности*

ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 00a570e4f7ad48d531b4b8818e75f29506
Владелец **Зубов Юрий Сергеевич**
Действителен с 04.02.2025 по 28.11.2026

Ю.С. Зубов

Приложение Г

Акты о внедрении результатов диссертационного исследования

EVOCARGO

ООО «ЭВОКАРГО»
ИНН/КПП: 7714459603/771701001
ОГРН 1207700132804
Расчетный счет: 40702810038000257848
Банк: ПАО СБЕРБАНК БИК: 044525225
Корр. счет: 30101810400000000225

Адрес: 129085, г. Москва г, ул. Годовикова,
д. 9, стр. 4 Под.4.15, этаж 3, помещение. 3.9
тел.: +7 (495) 926 26 67
e-mail: info@evocargo.com

Акт о внедрении результатов диссертационного исследования

Данный акт подтверждает, что результаты исследовательской работы, полученные Чекановым М.О. и приведенные в диссертации «Алгоритмы комплексирования разнородных данных при построении модели окружающей сцены беспилотного транспортного средства», представленной на соискание ученой степени кандидата технических наук по специальности 2.3.8 Информатика и информационные процессы, были внедрены в продукт «тележка высокоавтоматизированная электрическая торговой марки EVOCARGO моделей: BATC ЭВО 1.1, BATC ЭВО-1.2.», предназначенную для оказания услуг автономной логистики (перемещение материально-технических ресурсов).

Генеральный директор
ООО «ЭвоКарго»



М.И. Нигметзянов